

Azure Blob Storage troubleshooting documentation

Welcome to Azure Blob Storage troubleshooting. These articles explain how to determine, diagnose, and fix various issues that you might encounter when you use Azure Blob Storage. In the navigation pane on the left, browse through the article list or use the search box to find issues and solutions.

Azure Blob Storage documentation

OVERVIEW

[Azure Storage documentation](#)

Blob Storage and Azure Data Lake Store Gen2

HOW-TO GUIDE

[Azure Storage data protection, backup, and recovery](#)

[Copy blobs between storage accounts with network restriction](#)

[Invalid blob or block content, or invalid block list](#)

[Optimize CreateFile function](#)

[Request body is too large error when write to file](#)

[Troubleshoot high CPU or disk usage issues](#)

Best practices for Azure Storage data protection, backup, and recovery

The article provides Azure Storage data protection and backup options, self-serve recovery scenarios, and Microsoft-assist recovery possibilities.

Data protection, backup, and recovery options

Azure Storage data protection refers to strategies for:

- Protecting the storage account and data within it from being deleted or modified.
- Restoring data after it has been deleted or modified.

This section introduces available data protection, backup, and recovery options. For more information, see [data backup and protection options](#).

Data protection and backup options

The following sections introduce data protection scenarios and recommended protection options:

- [Scenario 1: Storage account protection](#)
- [Scenario 2: Blob container protection](#)
- [Scenario 3: Blob file protection](#)

Scenario 1: Storage account protection

Enable Azure Resource Manager (ARM) lock to lock all your storage accounts and prevent the deletion of the storage account. For more information about ARM lock, see [Apply an Azure Resource Manager lock to a storage account](#).

Benefits and limitations:

- Protects the storage account against deletion or configuration changes.
- Doesn't protect containers or blobs in the account from being deleted or overwritten.
- It supports Azure Data Lake Storage (ADLS) Gen 2.

Scenario 2: Blob container protection

- Enable immutability policies on a container to protect business-critical documents, such as to meet legal or regulatory compliance requirements.

Benefits and limitations:

- Protects a container and its blobs from all deletes and overwrites.
- When a legal hold or a locked time-based retention policy is in effect, the storage account is also protected from deletion. Containers for which no immutability policy has been set aren't protected from deletion.
- It supports ADLS Gen 2 in preview.

For more information about immutability policies on a container, see [Store business-critical blob data with immutable storage](#).

- Enable container soft-delete to restore a deleted container within a specified interval.

Benefits and limitations:

- A deleted container and its contents may be restored within the retention period. The best practice for a minimum retention interval is seven days.
- Only container-level operations, like "Delete container", can be restored. Container soft delete doesn't enable you to restore an individual blob in the container if that blob is deleted.
- It supports ADLS Gen 2.

For more information on container soft delete, see [Soft delete for containers](#).

Scenario 3: Blob file protection

- Enable immutability policies on a blob version to prevent a blob version from being deleted for an interval you control.

Benefits and limitations:

- Protects a blob version from being deleted and its metadata from being overwritten. An overwrite operation creates a new version.
- If at least one container has version-level immutability enabled, the storage account is also protected from deletion.
- Container deletion fails if at least one blob exists in the container.
- It's *not* available for ADLS Gen2.

For more information on immutability policies on a blob version, see [Store business-critical blob data with immutable storage](#).

- Enable blob soft delete to restore a deleted blob or blob version within a specified interval.

Benefits:

- A deleted blob or blob version may be restored within the retention period. The best practice for a minimum retention interval is seven days.
- It supports ADLS Gen 2.

For more information on blob soft delete, see [Soft delete for blobs](#).

- Enable blob snapshot to manually save the state of a blob at a given point in time.

Benefits and limitations:

- A blob may be restored from a snapshot if the blob is overwritten. However, if the blob is deleted, snapshots are also deleted.
- It supports ADLS Gen 2 in preview.

For more information on blob snapshots, see [Blob snapshots](#).

- Enable blob versioning to automatically save the state of a blob in a previous version when it's overwritten.

Benefits and limitations:

- Every blob write operation creates a new version. The current version of a blob may be restored from a previous version if the current version is deleted or overwritten.
- It's not available for ADLS Gen2.

For more information on blob versioning, see [Blob versioning](#).

- Enable Point-in-time restore to restore a set of block blobs to a previous point in time.

Benefits and limitations:

- A set of block blobs may be reverted to their state at a specific point in the past.
- Only operations performed on block blobs are reverted.
- Any operations performed on containers, page blobs, or append blobs aren't reverted.
- It's not available for ADLS Gen2.

For more information on point-in-time restore, see [Point-in-time restore for block blobs](#).

- Copy data to a second account via Azure Storage object replication or tools like AzCopy or Azure Data Factory.

Benefits and limitations:

- Data can be restored from the second storage account if the primary account is compromised in any way.
- AzCopy and Azure Data Factory are supported.
- Object replication isn't supported.

Data recovery options

The following sections introduce data recovery scenarios and possible recovery options:

- [Scenario 1: Storage account recovery](#)
- [Scenario 2: Blob container recovery](#)
- [Scenario 3: Blob file recovery](#)

You can recover data after [data protection and backup options](#) are enabled.

Scenario 1: Storage account recovery

Refers to [Recover deleted storage accounts from the Azure portal](#).

Scenario 2: Blob container recovery

- Recover the soft-deleted container and its contents.

Requirements for recovery:

- Container soft delete is enabled.
- The container soft delete retention period hasn't yet expired.

For more information, see [Enable and manage soft delete for containers](#).

- Recovery from a second storage account.

Requirements for recovery: All container and blob operations have been replicated to a second storage account.

Scenario 3: Blob file recovery

- Recover blobs to previous versions via blob versioning.

Requirements for recovery:

- Blob versioning is enabled.
- The blob has one or more previous versions.

For more information, see [Enable and manage blob versioning](#).

This option is currently not supported for ADLS workloads.

Recovery procedures:

1. Go to the affected blob from the Azure portal.
2. Select the ellipses (...) for the blob you want to recover.
3. Select **View versions**.
4. Select the version that's required to restore from.
5. Select **Make current version**.

- Recover blobs via blob soft delete.

Requirements for recovery:

- Blob soft delete is enabled.
- The soft delete retention interval hasn't expired.

For more information, see [Manage and restore soft-deleted blobs](#).

- Recover a set of block blobs via point-in-time.

Requirements for recovery:

- Point-in-time restore is enabled.
- The restore point is within the retention interval.
- The storage account hasn't been compromised or corrupted.

For more information, see [Perform a point-in-time restore on block blob data](#).

- Recover blobs via snapshots.

Requirements for recovery: The blob has one or more snapshots. For more information, see [Create and manage a blob snapshot in .NET](#).

Recovery procedures:

1. Go to the affected blob from the Azure portal.
2. Select the ellipses (...) for the blob you want to recover.
3. Select **View snapshots**.
4. Select the snapshot that's required to restore from.

5. Select **Promote**.

Best practice for Azure RBAC

Another best practice to avoid accidental account deletion is to limit the number of users who have permissions to delete an account via role-based access control (Azure RBAC).

Here are some recommended methods:

- Only grant the access users need.
- Limit the number of subscription owners.
- Use Microsoft Entra Privileged Identity Management.
- Assign roles to groups, not users.
- Assign roles using the unique role ID instead of the role name.

For more information, see [Best practices for Azure RBAC](#).

Non-supported storage recovery

Microsoft doesn't support the following storage recovery scenarios:

- Azure Storage Queue recovery isn't supported.
- Azure Storage Table entries recovery isn't supported, while deleted table recovery is supported. For more information, see [Supported Storage Recovery](#).
- Azure Blob files recovery without enabling blob file protection isn't supported.

Supported storage recovery

This section describes several supported storage recovery scenarios when some prerequisites are met:

- [Scenario 1: Storage account recovery \(ARM storage account recovery\)](#)
- [Scenario 2: Classic storage account recovery](#)
- [Scenario 3: ADLS Gen 2 data and file system recovery](#)
- [Scenario 4: Table recovery](#)
- [Scenario 5: Disk recovery](#)

Microsoft is making every effort to recover the data but cannot guarantee the amount of data that can be restored.

Scenario 1: Storage account recovery (ARM storage account recovery)

Prerequisites:

- The storage account was deleted within the past 14 days.
- The storage account was created with the Azure Resource Manager deployment model.
- A new storage account with the same name hasn't been created since the original account was deleted.
- The user recovering the storage account must be assigned an Azure RBAC role that provides the **Microsoft.Storage/storageAccounts/write** permissions. For information about built-in Azure RBAC roles that provide this permission, see [Azure built-in roles](#).
- Make sure the resource group that deleted the storage account exists. If the resource group was deleted, you must recreate it manually.
- [For specific cases only] If the deleted storage account used customer-managed keys with Azure Key Vault and the key vault has also been deleted, you must restore the key vault before you restore the storage account. For more information, see [Azure Key Vault recovery overview](#).

Suggestions:

- Recover a storage account from an existing storage account.
- Recover a storage account via a support ticket.

For more information, see [Recover deleted storage accounts from Azure portal](#).

Scenario 2: Classic storage account recovery

Prerequisites:

- A new storage account with the same name hasn't been created since the original account was deleted.
- The storage account was deleted within the past 14 days.

Suggestions:

- Seek help from support engineers to evaluate the situation.

Scenario 3: ADLS Gen 2 data and file system recovery

Prerequisites:

- Storage account with Hierarchical Namespace (HNS) enabled.
- The ADLS Gen2 file or folder was deleted within three days.

Suggestions:

- Seek help from support engineers to evaluate the situation.

Scenario 4: Table recovery

Prerequisites:

- The whole table is deleted using the "DELETE Table" operation without modifying the table entry data.

Suggestions:

- Seek help from support engineers to evaluate the situation.

Scenario 5: Disk recovery

Prerequisites:

- The prerequisites of disk recovery vary depending on a few factors. For instance, is soft delete enabled? Or does the recovery disk refer to managed disk or unmanaged disk?

Suggestions:

- Seek help from support engineers to evaluate the situation.

Recover deleted storage accounts from the Azure portal

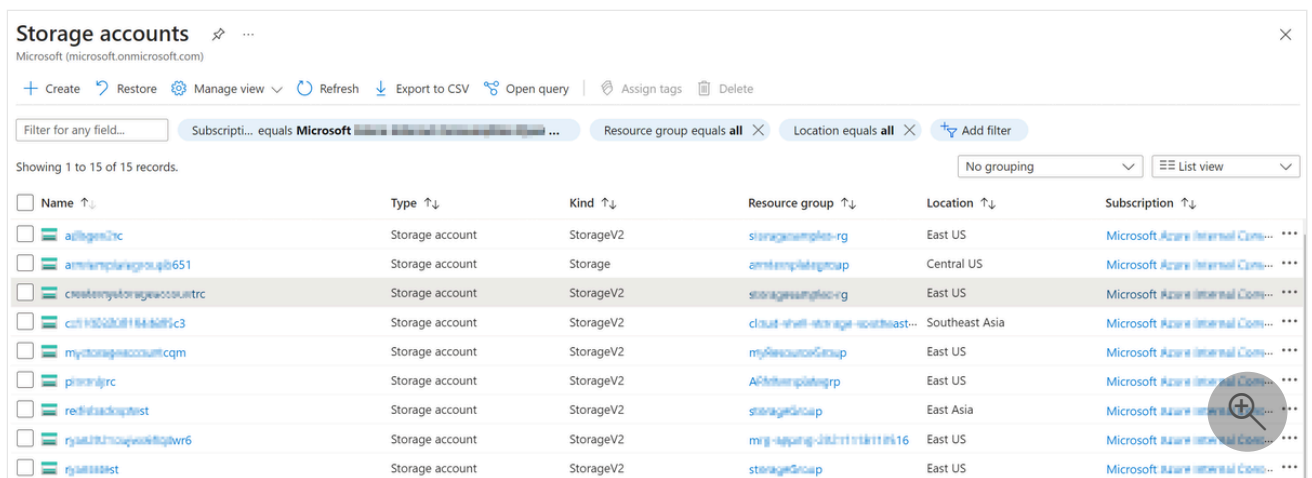
There are two ways for end users to recover a deleted storage account from the Azure portal:

- [Recover the deleted account from an existing storage account.](#)
- [Recover the account via a support ticket.](#)

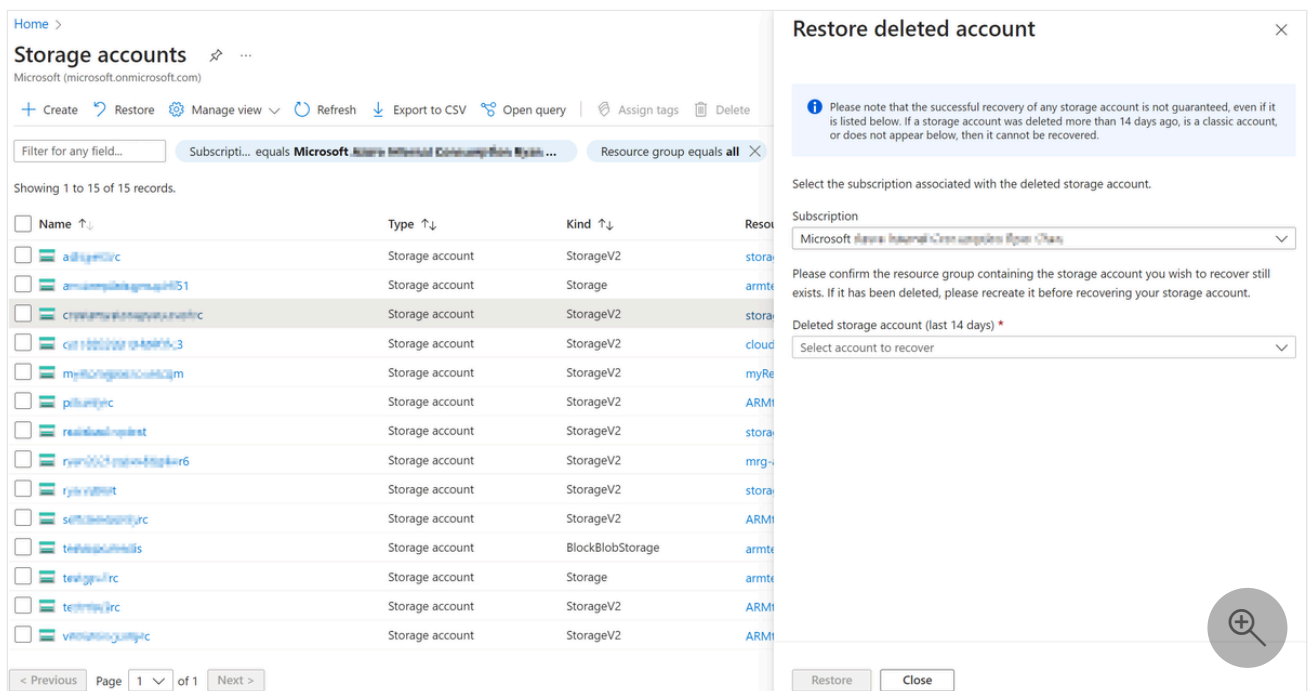
Recover a deleted storage account from another storage account

To recover a deleted storage account from within another storage account, follow these steps:

1. Navigate to the list of your storage account in the Azure portal.
2. Select the **Restore** button to open the **Restore deleted account** pane.



3. Select the subscription for the account you want to recover from the **Subscription** drop-down list.



4. From the drop-down list, select the account to recover. If the storage account you want to recover isn't in the drop-down list, it can't be recovered.
5. Select the **Restore** button to recover the account. The portal displays a notification that the recovery is in progress.

For more information, see [Recover a deleted account from the Azure portal](#).

Recover storage accounts via a support ticket

1. In the Azure portal, navigate to **Help + support**.
2. Select **Create a support request**.
3. On the **Problem description** tab, in the **Issue type** field, select **Technical**.
4. In the **Subscription** field, select the subscription that contained the deleted storage account.
5. In the **Service** field, select **Storage Account Management**.
6. In the **Resource** field, select any storage account resource. The deleted storage account doesn't appear in the list.
7. Add a brief summary of the issue.
8. In the **Problem type** field, select **Deletion and Recovery**.
9. In the **Problem subtype** field, select **Recover deleted storage account**.

The following screenshot shows an example of the **Problem description** tab being filled out:

The screenshot shows the 'New support request' form in the Azure portal. The breadcrumb navigation at the top is 'Home > Help + support >'. The title is 'New support request' with a three-dot menu icon. Below the title are four tabs: '1. Problem description' (active), '2. Recommended solution', '3. Additional details', and '4. Review + create'. The main heading is 'Tell us your issue, and we'll help you resolve it.' followed by a subtext: 'Provide information about your billing, subscription, quota management, or technical issue (including requests for technical advice).' The form fields are as follows: 'Issue type *' is a dropdown menu with 'Technical' selected; 'Subscription *' is a dropdown menu with a blurred selection; below it is a link 'Can't find your subscription? Show more ⓘ'; 'Service' has two radio buttons, 'My services' (selected) and 'All services'; 'Service type *' is a dropdown menu with 'Storage Account Management' selected; 'Summary *' is a text input field with the placeholder 'Briefly describe your issue'; 'Problem type *' is a dropdown menu with 'Deletion and Recovery' selected; and 'Problem subtype *' is a dropdown menu with 'Recover deleted storage account' selected.

10. Navigate to the **Recommended solution** tab, and select **Customer-Controlled Storage Account Recovery**.

New support request ...

1. Problem description 2. Recommended solution 3. Additional details 4. Review + create

Fix this problem now using our expert solution

Expert solutions are written by Azure engineers to help you quickly resolve the problem on your own.

Recommended Steps

Azure Storage now provides users a self-service storage account recovery capability.

Conditions for a storage account to be recoverable:

- A new storage object with the same name has not been re-created since deletion.
- The storage account was deleted in the last 14 days, including today. If the storage account was deleted prior to that, it can't be recovered.
- It is not a classic storage account.

Prerequisites:

- Ensure that the Resource Group is created first, if it has been deleted.
- Ensure that the KeyVault key is recovered first for storage accounts with CMK.

Disclaimer:

- There's no guarantee that recovery will always succeed. Recovery is a best effort rather than a guarantee.
- We strongly recommend using a combination of [ARM resource locks](#) and [Azure RBAC](#) permissions to prevent accidental account deletion.

Customer-Controlled Storage Account Recovery

11. From the drop-down list, select the account to recover. If the storage account you want to recover isn't in the drop-down list, it can't be recovered.

Recover deleted account

✕

Review a list of storage accounts deleted in the past 14 days below for the following subscription:

Azure Storage content development and testing.

Please confirm the resource group containing the storage account you wish to recover still exists. If it has been deleted, please recreate it before recovering your storage account.

Deleted storage accounts (last 14 days) *

Select account to recover

Resource group: st-rg

Storage account: s-per

Location: West US, Creation: 12/10/2020, 12:04:56 PM, Deletion: 12/10/2020, 12:43:09 PM

If a storage
be recovered.

Recover

Close

12. Select **Recover** to restore the account. The portal displays a notification that the recovery is in progress.

Troubleshoot issues in AzCopy v10

This article discusses common issues that you might encounter when you use AzCopy. This article also helps you identify the causes of the issues, and suggests how to resolve them.

Identifying issues

You can determine whether a job succeeds by looking at the exit code.

If the exit code is `0-success`, the job finished successfully.

If the exit code is `1-error`, examine the log file. After you understand the exact error message, you can more easily search for the right keywords and determine the solution. To learn more, see [Find errors and resume jobs by using log and plan files in AzCopy](#).

If the exit code is `2-panic`, check whether the log file exists. If the file doesn't exist, file a bug or reach out to support.

Any other nonzero exit code (such as `OOMKilled`) might be generated by the system. Check your operating system documentation for special exit codes.

"403" errors

"403" errors are common. Sometimes they're benign and don't cause a failed transfer. For example, in AzCopy logs, you might see that a `HEAD` request received "403" errors. Those errors appear when AzCopy checks whether a resource is public. In most cases, you can ignore those instances.

In some cases, "403" errors can cause a failed transfer. If this issue occurs, other attempts to transfer files are likely to fail until you resolve the issue. "403" errors can be caused by authentication and authorization issues. They can also occur if requests are blocked by the storage account firewall configuration.

This section covers a few common causes for 403 errors. For a comprehensive troubleshooting guide, see [Troubleshoot 403 errors](#).

Authentication and authorization issues

"403" errors that prevent data transfer occur because of issues that involve SAS tokens, role-based access control (Azure RBAC) roles, and access control list (ACL) configurations.

SAS tokens

If you use a shared access signature (SAS) token, make sure that the following statements are true:

- The expiration and start times of the SAS token are appropriate.
- You selected all the necessary permissions for the token.
- You generated the token by using an official SDK or tool. Try Storage Explorer if you haven't already.

For a complete checklist of SAS issues that can cause 403 errors, see [SAS tokens](#).

Azure RBAC

If you use Azure RBAC roles through the `azcopy login` command, verify that you have the appropriate Azure roles assigned to your identity (for example, the Storage Blob Data Contributor role).

To learn more about Azure roles, see [Assign an Azure role for access to blob data](#).

ACLs

If you use access control lists (ACLs), verify that your identity appears in an ACL entry for each file or directory that you intend to access. Also, make sure that each ACL entry reflects the appropriate permission level.

To learn more about ACLs and ACL entries, see [Access control lists \(ACLs\) in Azure Data Lake Storage Gen2](#).

To learn more about how to incorporate Azure roles together with ACLs and how the system evaluates them to make authorization decisions, see [Access control model in Azure Data Lake Storage Gen2](#).

Firewall and private endpoint issues

If the storage firewall configuration doesn't allow access from the hosting component on which AzCopy is running, AzCopy operations return an HTTP "403" error code.

Note

In this article, the term *hosting component* refers to a physical computer, virtual machine (VM), or container.

Permitted scope for copy operations

The `AllowedCopyScope` property of a storage account is used to specify the environments from which you can copy data to the destination account. This property is displayed in the Azure portal as the **Permitted scope for copy operations (preview)** configuration setting. By default, the property isn't given a value. The property doesn't return a value until you explicitly set it. The `AllowedCopyScope` property has three possible values, as shown in the following table.

 [Expand table](#)

Value	Description
<code>(null)</code>	(Default value) Allows copying from any storage account to the destination account.
<code>Microsoft Entra ID</code>	Permits copying from only accounts that are within the same Microsoft Entra tenant as the destination account.
<code>PrivateLink</code>	Permits copying from only storage accounts that have private links to the same virtual network as the destination account.

For more information about this property and its associated configuration setting, see [Restrict the source of copy operations to a storage account](#).

Transfer data from or to a local hosting component

If you're uploading or downloading data between a storage account and an on-premises hosting component, make sure that the hosting component that runs AzCopy can access either the source or destination storage account. You might have to use IP network rules in the firewall settings of either the source or destination accounts to allow access from the public IP address of the hosting component. For a checklist of common causes, see [Public network endpoint](#).

Transfer data between storage accounts

"403" authorization errors can prevent you from transferring data between accounts by using the client hosting component on which AzCopy is running.

If you're copying data between storage accounts, make sure that the hosting component that runs AzCopy can access both the source and the destination account. You might have to use IP

network rules in the firewall settings of both the source and destination accounts to allow access from the public IP address of the hosting component. The service uses the IP address of the AzCopy client hosting component to authorize the source to destination traffic. To learn how to add a public IP address to the firewall settings of a storage account, see [Grant access from an internet IP range](#).

For a checklist of other issues to consider, see [Public network endpoint](#).

In case your VM doesn't or can't have a public IP address, consider using a private endpoint. See [Use private endpoints for Azure Storage](#).

Use Private Link

[Private Link](#) is at the virtual network/subnet level. If you want AzCopy requests to go through Private Link, then AzCopy must make those requests from a VM that's running in that virtual network/subnet. For example, suppose that you configure Private Link in VNet1/Subnet1, but the VM on which AzCopy runs is in VNet1/Subnet2. In this scenario, AzCopy requests don't use Private Link, and the requests are expected to fail. For a checklist of other issues to consider, see [Private endpoints](#).

Proxy-related errors

If you encounter TCP errors such as "dial tcp: lookup proxy.x.x: no such host," this means that your environment isn't configured to use the correct proxy or you use an advanced proxy that AzCopy doesn't recognize.

You have to update the proxy settings to reflect the correct configurations. See [Configure proxy settings](#).

You can also bypass the proxy by setting the environment variable `NO_PROXY="*"`.

Here are the endpoints that AzCopy requires:

[Expand table](#)

Sign-in endpoints	Azure Storage endpoints
<code>login.microsoftonline.com</code> (global Azure)	<code>(blob file dfs).core.windows.net</code> (global Azure)
<code>login.chinacloudapi.cn</code> (Azure China)	<code>(blob file dfs).core.chinacloudapi.cn</code> (Azure China)
<code>login.microsoftonline.de</code> (Azure Germany)	<code>(blob file dfs).core.cloudapi.de</code> (Azure Germany)

Sign-in endpoints	Azure Storage endpoints
<code>login.microsoftonline.us</code> (Azure US Government)	<code>(blob file dfs).core.usgovcloudapi.net</code> (Azure US Government)

x509: certificate signed by unknown authority

This error is often related to the use of a proxy that's using a Secure Sockets Layer (SSL) certificate that isn't trusted by the operating system. Verify your settings and make sure that the certificate is trusted at the operating system level.

We recommend that you add the certificate to your hosting component's root certificate store because that's where the trusted authorities are kept.

Unrecognized parameters

If you receive an error message that states that your parameters aren't recognized, make sure that you're using the correct version of AzCopy. AzCopy v8 and earlier versions are deprecated. [AzCopy v10](#) is the current version, and it's a complete rewrite that doesn't share any syntax with the previous versions. Refer to [AzCopy Migration Guide for v8 to v10](#).

Also, make sure to use built-in help messages by using the `-h` switch together with any command (for example, `azcopy copy -h`). See [Get command help](#). To view the same information online, see [azcopy copy](#).

To find example commands, see [Transfer data](#). If you have a question, try searching through existing [GitHub issues](#) first to see whether it was already answered.

Conditional access policy error

You might receive the following error when you invoke the `azcopy login` command:

```
Failed to perform login command: failed to login with tenantID "common", Azure directory endpoint "https://login.microsoftonline.com", autorest/adal/devicetoken: -REDACTED- AADSTS50005: User tried to log in to a device from a platform (Unknown) that's currently not supported through Conditional Access policy. Supported device platforms are: iOS, Android, Mac, and Windows flavors. Trace ID: -REDACTED- Correlation ID: -REDACTED- Timestamp: 2021-01-05 01:58:28Z
```

This error means that your administrator configured a conditional access policy that specifies the kind of device that you can sign in from. AzCopy uses the device code flow. The device

code flow can't guarantee that the hosting component on which you're using the AzCopy tool is also where you're signing in from.

If your device is among the list of supported platforms, then you might be able to use Storage Explorer. Storage Explorer integrates AzCopy for all data transfers (it passes tokens to AzCopy through the secret store) but provides a sign-in workflow that supports passing device information. AzCopy itself also supports managed identities and service principals as a sign-in alternative.

If your device isn't on the list of supported platforms, contact your administrator for help.

Server busy, network errors, or time-outs

If you see a large number of failed requests that have the "503 Server Busy" status, then the storage service is throttling your requests. If you see network errors or time-outs, you might be trying to push too much data for your infrastructure to handle. In all cases, the workaround is similar.

If you see a large file repeatedly fail to copy because certain chunks fail each time, try to limit the concurrent network connections or throughput limit depending on your specific case. We suggest that you lower the performance drastically at first, observe whether this action solved the initial problem, and then ramp up the performance again until you achieve an overall balance.

For more information, see [Optimize the performance of AzCopy with Azure Storage](#).

If you're copying data between accounts by using AzCopy, the quality and reliability of the network from where you run AzCopy might affect the overall performance. Even though data transfers from server to server, AzCopy does initiate calls for each file to copy between service endpoints.

Known constraints in AzCopy

- Copying data from government clouds to commercial clouds isn't supported. However, copying data from commercial clouds to government clouds is supported.
- Asynchronous service-side copy isn't supported. AzCopy performs synchronous copy only. In other words, by the time the job finishes, the data has been moved.
- When you copy to an Azure File share, if you forgot to specify the `--preserve-smb-permissions` flag and you don't want to transfer the data again, consider using Robocopy to bring over the permissions.

- Azure Functions has a different endpoint for MSI authentication. AzCopy doesn't yet support MSI authentication.

See also

- [Get started with AzCopy](#)
- [Find errors and resume jobs by using log and plan files in AzCopy](#)
- [Troubleshoot 403 errors](#)

Last updated on 10/13/2025

Use AzCopy to copy blobs between Azure storage accounts with network restrictions

Summary

This article introduces how to copy blobs between storage accounts by using the AzCopy command-line utility. It also explains how to implement the copy operation when network restrictions are configured for the storage accounts.

Background

Copying blob files between two storage accounts is a common requirement for many Azure users. Azure Storage supports directly copying blobs from one storage account to another, which you can implement by using the AzCopy command-line utility. You don't need to download files to local disks or buffers and then upload them again.

Copying blobs between two storage accounts by using AzCopy doesn't rely on the network bandwidth of your local computer. This method can take advantage of the performance of storage accounts and Azure Virtual Network to achieve better throughput than downloading and uploading files. If both storage accounts are in the same region, you don't pay any bandwidth charges.

AzCopy commands for copying blobs between storage accounts

- If you provide authorization credentials by using Microsoft Entra ID, use the following command:

AzCopy

```
azcopy copy 'https://<source-storage-account-name>.blob.core.windows.net/<container-name>/<blob-path>' 'https://<destination-storage-account-name>.blob.core.windows.net/<container-name>/<blob-path>'
```

In this scenario, you need to make sure your Microsoft Entra identity has the proper role assignments for both source and destination accounts.

- If you use a Shared Access Signature (SAS) token, use the following command:

AzCopy

```
azcopy copy 'https://<source-storage-account-name>.blob.core.windows.net/<container-name>/<blob-path><SAS-token>'  
'https://<destination-storage-account-name>.blob.core.windows.net/<container-name>/<blob-path><SAS-token>'
```

In this scenario, you need to append a SAS token to both the source and destination URL used in your AzCopy commands.

For more information, see [Copy blobs between Azure storage accounts with AzCopy v10](#).

Copy blobs between storage accounts with access restriction

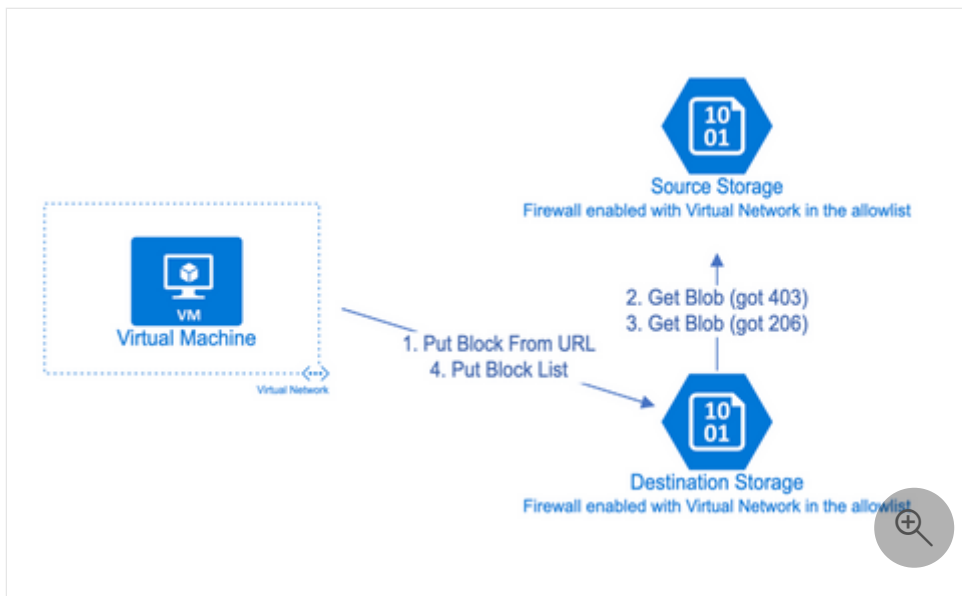
If you need to restrict access to both source and destination storage accounts through the storage firewall, you might need additional configurations for copying blobs between storage accounts by using AzCopy. This restriction exists because the copy request between two storage accounts uses private IP addresses, and the IP addresses are dynamic.

Two supported scenarios exist:

Scenario 1: The client uses a public endpoint to access storage accounts

In this scenario, you must add the client's public IP address or virtual network (VNet) to the firewall allowlist in the source and destination storage accounts.

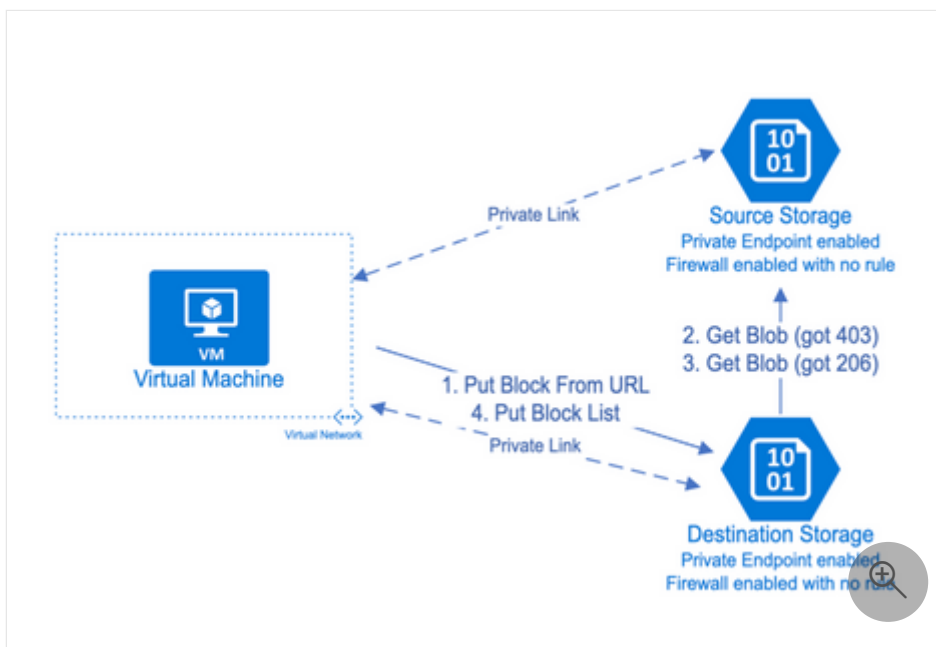
The following image shows the process of copying blobs between storage accounts in this scenario:



Scenario 2: The client's VNet has private links configured, and it uses a private endpoint to access storage accounts

In this scenario, you don't need a firewall allowlist.

The following image shows the process of copying blobs between storage accounts in this scenario:



Here's the full process of this mechanism for the two scenarios:

1. The client sends a PutBlockfromURL request to the destination storage.
2. The destination storage receives the requests and tries to get blocks from the given source URL. However, since the destination storage isn't allowed by the source firewall, it receives a **403 Forbidden** error.

3. After the destination storage receives the **403 Forbidden** error, it sends another GetBlob request on behalf of the client. If the client has access to the source storage, the destination can get the blocks from the source and return a success response code to the client.
4. The client sends PutBlockList to the destination storage to commit the blocks and finish the process after receiving a success response code from the request.

Scenario 3: The client uses public endpoints with network security perimeter enforced

In this scenario, you need to configure network security perimeter access rules so the request succeeds for all three paths: client-to-destination, client-to-source, and destination-to-source. If there's no authorized access across any of the three paths, the copy operation fails and usually shows a **403 Forbidden** error. For more information about network security perimeters, see [What is a network security perimeter?](#)

Client, source, and destination accounts in the same perimeter

When the client, source account, and destination account are all within the same perimeter, traffic between them is allowed as intra-perimeter communication. This configuration is recommended, as intra-perimeter communication ensures that all three paths are authorized without requiring extra access rules.

Source and destination accounts in the same perimeter

When source and destination accounts are in the same perimeter, traffic between them is allowed as intra-perimeter communication. In this scenario, you need to ensure that inbound access is granted to the client from both accounts.

1. The client sends a Put Block From URL request to the destination storage account.

To permit this request, ensure that inbound access is granted to the client from the destination account.

2. The destination account sends a Get Block request to the source storage account.

The destination storage account receives the request and tries to retrieve blocks from the given source URL. Because the destination retrieves blocks from the source on behalf of the client, inbound access from the client must be granted on both the source and destination accounts.

To permit this request, ensure that inbound access is granted to the client from the source account.

Client, source account, and destination account in different perimeters

When the client, source account, and destination account are all in different perimeters, you can allow cross-perimeter communication and simplify access rule requirements by using perimeter links. For more information, see [az network perimeter link](#).

A perimeter link is the only supported method for granting inbound access to a client that resides in a different perimeter than the source or destination account. If you don't use a perimeter link to allow communication between source and destination account perimeters, you must configure the following access rules:

1. The client sends a Put Block From URL request to the destination storage account.
2. The destination account sends a Get Block request to the source storage account.

To permit this request, ensure that:

- Outbound access is granted to the source account from the destination account.
- Inbound access is granted to the destination account from the source account.

Add outbound access rule

A profile can allow external outbound access based on fully qualified domain names. Resources associated within the same network security perimeter do not require an access rule. By default access is denied to external destinations when there's no matching access rule. [Learn more](#) 

Rule name *	allowSource
Destination type	FQDNs
Allowed destinations *	sourcesa.blob.core.windows.net 

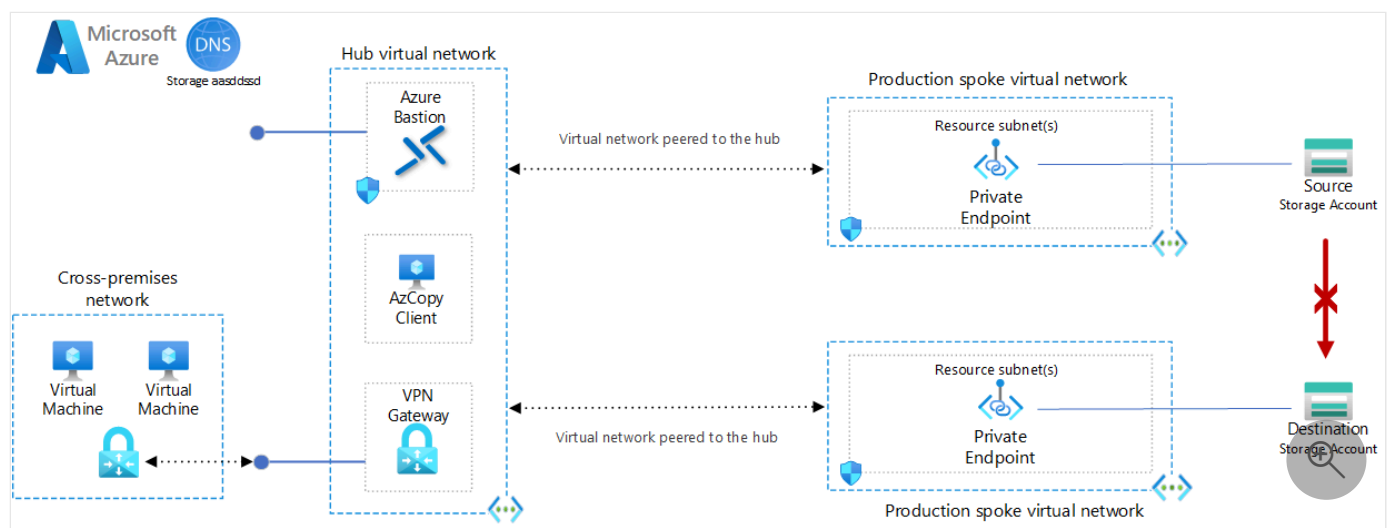
Note

This image is provided as an example for reference and only depicts the portal UI experience.

For more information about managing inbound and outbound access, see [Network Security Perimeter Access Rules](#) and [az network perimeter profile access-rule](#).

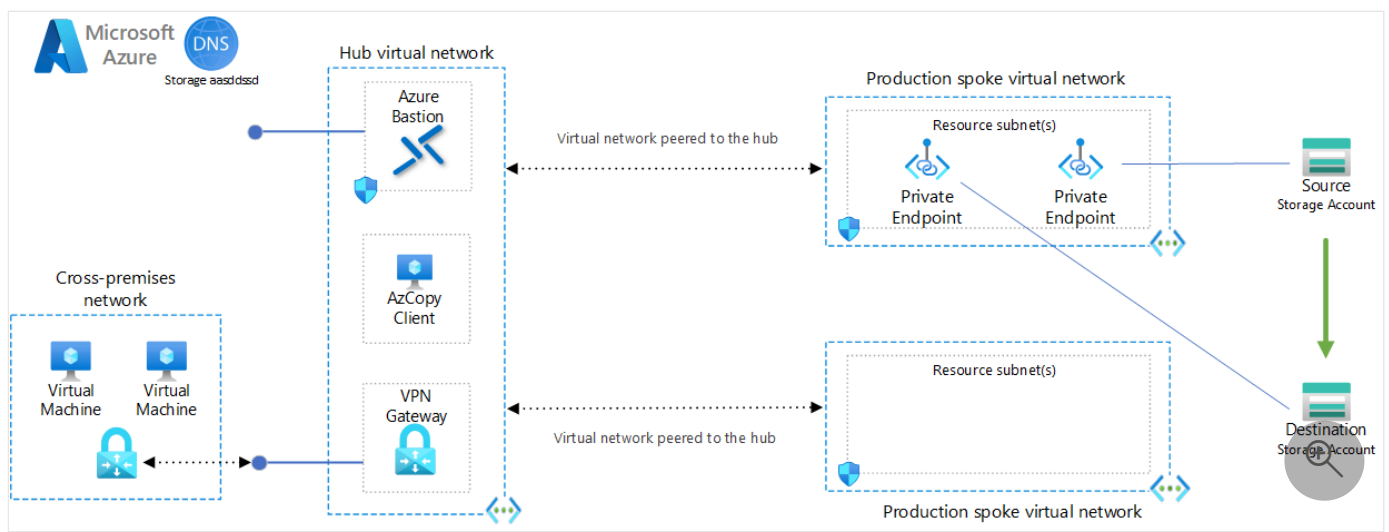
Copy blobs between storage accounts in a Hub-spoke architecture using private endpoints

A 403 error occurs when you use AzCopy to copy blobs between storage accounts connected to private endpoints in different spoke VNETs from a VM in a hub VNet. You see a `403 This request is not authorized to perform this operation - CannotVerifyCopySource` error in the AzCopy logs or in the Azure Storage logs. The following architecture diagram shows the scenario in which the error occurs.



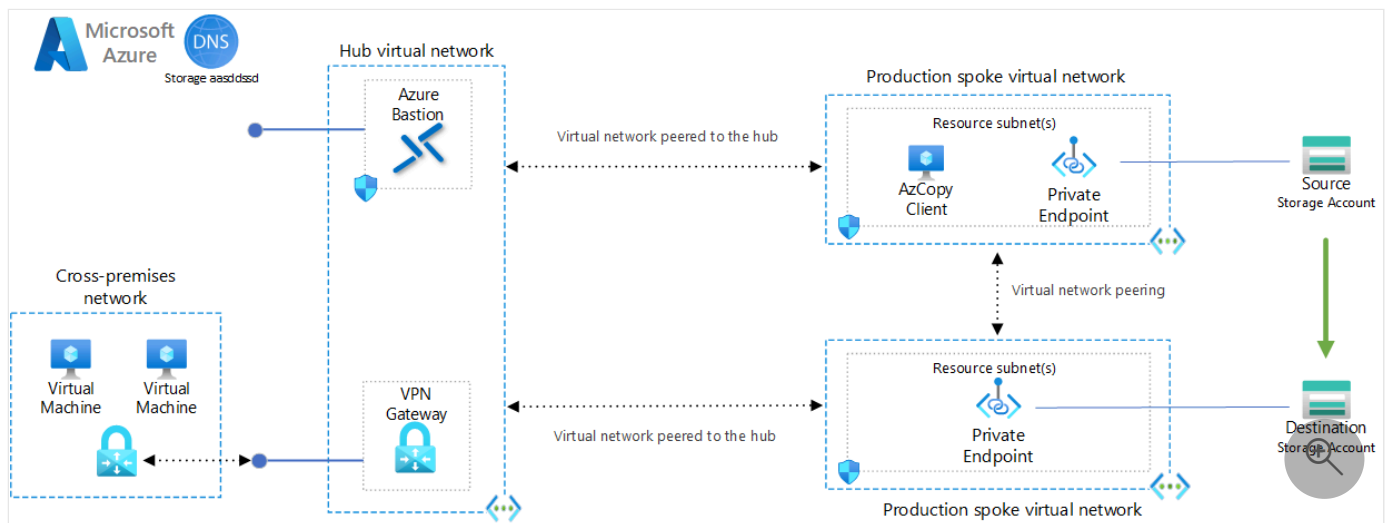
Workaround 1: Create a private endpoint for the destination storage account in the source VNet

Create a private endpoint for the destination storage account in the source VNet. This configuration allows the VM to successfully copy the blobs between the storage accounts by using AzCopy. The following architecture diagram shows the process of copying blobs between storage accounts in Workaround 1.



Workaround 2: Place the VM in the same VNet as the source storage account and configure VNet peering between the source and destination VNets

Place the VM within the same VNet as the source storage account and set up [virtual network peering](#) between this VNet and the destination VNet. This peering needs to be directly between the two VNets and can't be done through the hub VNet. The following architecture diagram shows the process of copying blobs between storage accounts in Workaround 2.



For more information, see [Virtual network peering FAQ](#).

Workaround 3: Use a temporary staging account to copy the data

If you can't implement the previously mentioned workarounds or are restricted from changing the existing network configuration of the storage account or VNet, use a temporary staging account to copy the data:

1. Create a temporary storage account in the same region as the source storage account and the destination storage account.
2. Use AzCopy to copy the data from the source storage account to the temporary storage account.
3. Copy the data from the temporary storage account to the destination storage account.
Make sure that the temporary storage account has a private endpoint in the same VNet as the destination storage account before you perform the data transfer.

Workaround 4: Use a VM to download and upload data between storage accounts

Use this workaround only if other methods aren't feasible. Use a VM to download the data from the source storage account, and then upload it to the destination storage account. You can use AzCopy for this process. Make sure that the VM's size and disk capacity are suitable for the data transfer process.

Last updated on 05/05/2026

Error when you write more than 4 MB of data to Azure Storage: Request body is too large

Original product version: Storage Account Management

Original KB number: 4016806

Symptoms

Assume that you try to write more than 4 megabytes (MB) of data to a file on Azure Storage, or you try to upload a file that's larger than 4 MB by using the SAS url (REST API) or HTTPS PUT method. In this scenario, the following error is returned:

Code : RequestBodyTooLarge

Message: The request body is too large and exceeds the maximum permissible limit.

RequestId:<ID>

MaxLimit : 4194304

Cause

This error can occur if the block size limit for the service version you run is exceeded. For example, versions prior to `2016-05-31` have a 4-MB block size limit per call to the Azure Storage service. Newer service versions support larger block sizes. For more information, see the following articles:

- [Scale targets for Blob storage](#)
- [Versioning for Azure Storage](#)

Workaround

If your file is too large, you must split it into chunks. For more information, see [Scalability and performance targets for standard storage accounts](#).

Uploading blob or block content fails in Azure Blob Storage

This article discusses how to resolve failures that might occur when you use Microsoft Azure Blob Storage together with your cloud applications to upload blob or block content.

Prerequisites

- A storage account from one of the following storage services:
 - [Azure Blob Storage](#)
 - [Azure Data Lake Storage Gen2](#)
 - [General-purpose v2 storage](#)
- [Azure Storage SDK](#)
- [Azure Storage Explorer](#)
- [PowerShell](#)

Symptoms

You receive one of the following error messages.

 Expand table

Error code	Error message
<code>BlockCountExceedsLimit</code>	"The uncommitted block count cannot exceed the maximum limit of 100,000 blocks."
<code>InvalidBlobOrBlock</code>	"The specified blob or block content is invalid."
<code>InvalidBlock</code> or <code>InvalidBlockList</code>	"The specified block list is invalid."

Cause 1: The block length that was specified in the Put Block call isn't valid

The block length that was specified in the [Put Block](#) URI request isn't valid for one or more of the following reasons:

- The application or client specified a block size that isn't supported.
- The size of the block is larger than the maximum allowed block size. To find the block size limits for different versions of the Blob Service REST API, see the [Remarks](#) section of the `Put Block` reference article.
- When you tried to upload blocks of data by using more than one application, there were uncommitted blocks that had inconsistent block lengths. This situation occurs because different applications use different lengths to upload data, or because a previous upload failed.
- The blob has too many uncommitted blocks because a previous upload operation was canceled. The maximum number of uncommitted blocks that can be associated with a blob is 100,000.

Remove the uncommitted blocks by implementing one of these solutions.

Solution 1: Wait for garbage collection to pick up the uncommitted data

Wait seven days for the uncommitted blocklist to be cleaned up by garbage collection.

Solution 2: Use a dummy blob to make the data transfer

Use the Azure Storage SDK to transfer the data by using a dummy blob. To do so, follow these steps:

1. Create a dummy blob that has the same blob name and is in the same container. This blob can have a length of zero.
2. Transfer the blob by using an unblocked transfer.

Solution 3: Commit the uncommitted blocklist by using the Azure Storage SDK

Use the Azure Storage SDK to commit the uncommitted block list and clean up the blob. To do so, follow these steps:

1. Retrieve the uncommitted blocklist by making a [Get Block List](#) URI request in which the `blocklisttype` URI parameter is set to `uncommitted`.
2. Commit the blocklist by using the [Put Block List](#) URI request.

3. Delete the blob.

The following PowerShell function is an example of how to retrieve an uncommitted block list and then delete it. The function requires the following parameters.

 Expand table

Parameter name	Description
<code>-StorageAccountName</code>	The name of the storage account.
<code>-SharedAccessSignature</code>	A shared access signature (SAS) token that uses the URI parameters <code><ss=b;srt=sco;sp=rwldc></code> . These parameters are described in Construct an account SAS URI .
<code>-ContainerName</code>	The name of the storage container.
<code>-BlobName</code>	The name of the blob.

PowerShell

```
[CmdletBinding()] Param(
    [Parameter(Mandatory=$true, Position=1)] [string] $StorageAccountName,
    [Parameter(Mandatory=$true, Position=1)] [string] $SharedAccessSignature,
    [Parameter(Mandatory=$true, Position=1)] [string] $ContainerName,
    [Parameter(Mandatory=$true, Position=1)] [string] $BlobName
)

# Build the URI strings in the REST API for GET and DELETE.
$uriDelete = (
    "https://$StorageAccountName.blob.core.windows.net/",
    "$ContainerName",
    "/",
    "$BlobName",
    "$SharedAccessSignature"
) -Join ""
$uriGet = (
    "$uriDelete",
    "&comp=blocklist",
    "&blocklisttype=uncommitted"
) -Join ""
Write-Host "The Delete URI is $uriDelete."
Write-Host "The Get URI is $uriGet."

# Make a REST API call to get the uncommitted block list.
$listFileURI = Invoke-WebRequest -Uri $uriGet -Method Get
$FileSystemName = $listFileURI.Content
$string = $FileSystemName -replace 'ï»¿', ''
$string |
    Select-Xml -XPath "/BlockList/UncommittedBlocks/Block" |
    Select-Object -Expand Node
```

```

$Count = $String.Count

# Delete the blob and the uncommitted block.
if ($Count.Count -gt 0) {
    $listFileURI1 = Invoke-WebRequest -Uri $uriDelete -Method Delete
    $FileSystemName1 = $listFileURI1.StatusCode
    Write-Host "The deletion was successful. The API returned status code
$FileSystemName1."
}

Write-Host "Check whether the uncommitted blocks are still present."
Try {
    $listFileURI2 = Invoke-WebRequest -Uri $uriGet -Method Get
} Catch {
    # $err = $_.Exception
    Write-Host "StatusCode:" $_.Exception.Response.StatusCode.value__
    Write-Host "StatusDescription:" $_.Exception.Response.StatusDescription
}

Write-Host (
    "In this error message, we can verify that the",
    "uncommitted blocks and their respective blob have been deleted.",
    "The name and size of the uncommitted blocks that have been deleted are shown."
)

```

Cause 2: PUT operations occur simultaneously for a blob

A timing or concurrency issue occurs. This causes multiple PUT (Put Block) operations to occur at about the same time for a single blob. The [Put Block List](#) operation writes a blob by specifying the list of block IDs that make up the blob. To be written as part of a blob, a block must have been successfully written to the server in a previous [Put Block](#) operation.

⚠ Note

This error can occur during concurrent upload commits after you start the upload but before you commit. In this case, the upload fails. The application can retry the upload when the error occurs, or it can try another recovery action that's based on the required scenario.

Solution: Use leases

Instead of using optimistic concurrency, try to implement pessimistic concurrency (leases) by using the Azure Storage SDK or a GUI-based tool, such as Azure Storage Explorer. For more

information about optimistic and pessimistic concurrency, see [Managing concurrency in blob storage](#).

If the error is caused by concurrency issues, you might also have to clean up the uncommitted blocks by following one of the solutions in [Cause 1](#).

Last updated on 01/15/2026

Troubleshoot Azure Blob Storage access issues in the Azure portal

When accessing Azure Blob Storage through the Azure portal, you may encounter errors when performing operations on containers or blobs. This article describes common error messages, their causes, and steps to resolve them.

General troubleshooting

When accessing blob containers or blobs in the Azure portal, requests are authorized using either your Microsoft Entra ID account or the storage account access key.

Verify required permissions

- If using **Microsoft Entra ID**:
 - You must have at least the **Reader** role for control plane access.
 - You must also have a data access role such as:
 - **Storage Blob Data Reader**
 - **Storage Blob Data Contributor**
 - **Storage Blob Data Owner**
- If using **Access Key**:
 - You must have an Azure Resource Manager role that includes the following control plane permission:

```
Microsoft.Storage/storageAccounts/listkeys/action
```

- Built-in roles that include this permission include:
 - **Reader and Data Access**
 - **Storage Account Contributor**
 - **Contributor**
 - **Owner**

For more information, see [Data access from the Azure portal](#).

Validate browser LocalNetworkAccessAllowedForUrl policy

If you're using **Microsoft Edge** or **Google Chrome** to access storage through a **private endpoint**, configure the `LocalNetworkAccessAllowedForUrl` browser policy to allow the Azure portal:

```
https://portal.azure.com
```

For more information, see:

- [Microsoft Edge policy: LocalNetworkAccessAllowedForUrls](#)
- [Chrome policy: LocalNetworkAccessAllowedForUrls](#) ↗

Use the Connectivity check tool

Use the built-in connectivity diagnostics in the Azure portal:

1. Go to your storage account in the Azure portal.
2. Expand **Help**.
3. Select **Connectivity check**.

This tool can help identify whether an issue is related to permissions, network security configuration, or other causes.

Error: This request is not authorized to perform this operation

You may see the following error when accessing a blob container or blob in the Azure portal:

```
This request is not authorized to perform this operation
Error code: 403
```

Cause

Common causes include:

- Public network access is disabled.
- Your client IP address is not included in the allowed IP ranges.
- Access is restricted to selected virtual networks or subnets.

Solution

- Verify the storage account network configuration. For more information, see [Azure Storage network security overview](#).
- If public network access is disabled and you are using a private endpoint, verify that DNS is configured correctly. For more information, see [Use private endpoints for Azure Storage](#).

Error: This request is not authorized to perform this operation using this permission

You may see the following error when accessing a blob container or performing an operation on a blob in the Azure portal:

```
This request is not authorized to perform this operation using this permission
Error code: 403
```

Cause

Your Microsoft Entra identity doesn't have sufficient permissions for the requested operation.

Solution

If the error occurs when viewing containers

The following control plane permissions are required at the storage account level:

```
Microsoft.Storage/storageAccounts/read
Microsoft.Storage/storageAccounts/blobServices/read
Microsoft.Storage/storageAccounts/blobServices/containers/read
```

The built-in **Reader** role includes these permissions.

If the error occurs when performing a data plane operation (creating a blob, deleting a blob, etc.)

Your identity must be assigned a role that includes the required data plane permissions, such as:

 Expand table

Role	Description
Storage Blob Data Reader	Read access to blob containers and blob data.
Storage Blob Data Contributor	Read, write, and delete access to blob containers and blob data.
Storage Blob Data Owner	Full access to blob containers and blob data, including assigning POSIX ACLs.

For more information, see [Data access from the Azure portal](#).

Error: You do not have permissions to list the data using your user account with Microsoft Entra ID

You may see the following error when accessing a blob container or blob in the Azure portal:

You do not have permissions to list the data using your user account with Microsoft Entra ID. Click to learn more about authenticating with Microsoft Entra ID. This request is not authorized to perform this operation using this permission.

Cause

Your Microsoft Entra identity is missing the following data plane permission:

Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read

Solution

Your identity must be assigned a role that includes the required data plane permissions, such as:

 Expand table

Role	Description
Storage Blob Data Reader	Read access to blob containers and blob data.
Storage Blob Data Contributor	Read, write, and delete access to blob containers and blob data.

Role	Description
Storage Blob Data Owner	Full access to blob containers and blob data, including assigning POSIX ACLs.

For more information, see [Data access from the Azure portal](#).

Error: You do not have permissions to use the access key to list data

You may see the following error when accessing a blob container or blob in the Azure portal:

You do not have permissions to use the access key to list data.

Cause

This error can occur when:

- Your Microsoft Entra identity doesn't have sufficient control plane permissions.
- Your browser blocks local network access when you use a private endpoint. A network trace may show the following error: `ERR_BLOCKED_BY_LOCAL_NETWORK_ACCESS_CHECKS`.

Solution

If you're using Access Key

Assign a role that includes the following permission:

Microsoft.Storage/storageAccounts/listkeys/action

Built-in roles that include this permission include:

- Reader and Data Access
- Storage Account Contributor
- Contributor
- Owner

If you're using Microsoft Entra ID

Verify that you have the following control plane permissions at the storage account scope:

```
Microsoft.Storage/storageAccounts/read
Microsoft.Storage/storageAccounts/blobServices/read
Microsoft.Storage/storageAccounts/blobServices/containers/read
```

The built-in **Reader** role includes these permissions.

If you're using a private endpoint

Verify that the `LocalNetworkAccessAllowedForUr1` browser policy allows the Azure portal URL:

```
https://portal.azure.com
```

For more information, see:

- [Data access from the Azure portal](#)
- [Microsoft Edge policy: LocalNetworkAccessAllowedForUrls](#)
- [Chrome policy: LocalNetworkAccessAllowedForUrls](#) ↗

Error: You don't have access

You may see the following error when accessing a blob container in the Azure portal:

```
You don't have access
Error code: 403
```

Cause

Your Microsoft Entra identity doesn't have the required control plane permissions.

Solution

The following control plane permissions are required at the storage account level:

```
Microsoft.Storage/storageAccounts/read
Microsoft.Storage/storageAccounts/blobServices/read
Microsoft.Storage/storageAccounts/blobServices/containers/read
```

The built-in **Reader** role includes these permissions.

For more information, see [Data access from the Azure portal](#).

Error: Network request failed – cannot access storage endpoint

You may see the following error when accessing a blob container or blob in the Azure portal:

```
Network request failed – cannot access storage endpoint
Your firewall settings may be preventing network access from this client to the
storage data endpoint.
CORS settings on your storage account may be preventing network access from this
client.
```

Cause

Common causes include:

- Public network access is disabled.
- Your client IP address is not included in the allowed IP ranges.
- Access is restricted to selected virtual networks or subnets.
- CORS rules are blocking access.

Solution

- Verify the storage account network configuration. For more information, see [Azure Storage network security overview](#).
- If public network access is disabled and you are using a private endpoint, verify that DNS is configured correctly. For more information, see [Use private endpoints for Azure Storage](#).
- If CORS rules are configured on the storage account, verify they are correct. For more information, see [Cross-origin resource sharing \(CORS\) support for Azure Storage](#).

Troubleshoot PublicAccessNotPermitted error

When accessing Azure Blob storage, you may encounter a `PublicAccessNotPermitted` error. This article describes how to resolve this error.

Symptom

When attempting to access an Azure Blob container or blob, you may receive the following error:

text

```
Public access is not permitted on this storage account.  
ErrorCode: PublicAccessNotPermitted
```

Cause

This error occurs when an unauthenticated (anonymous) request is made to a blob or container that does not allow public access.

Resolution

Option 1: Use authenticated access (recommended)

Instead of enabling anonymous access, use a supported authentication method to securely access Azure Blob storage:

- **Microsoft Entra ID (recommended)**
[Authorize Blob Access with Microsoft Entra ID](#)
- **Shared access signature (SAS)**
[Grant limited access to data with shared access signatures \(SAS\)](#)
- **Storage account key**
[Authorize with Shared Key](#)

For more information: [Authorize access to data in Azure Storage](#)

Option 2: Enable anonymous access (only if required)

 Warning

When a container is configured for anonymous access, any client can read data in that container. This can introduce significant security risks. Only enable anonymous access when explicitly required.

If your scenario requires anonymous access, see: [Configure anonymous read access for containers and blobs](#)

Last updated on 06/25/2026

Troubleshoot 403 errors in Azure Blob Storage

Summary

This article provides guidance for troubleshooting HTTP 403 errors in Azure Blob Storage. It includes a comprehensive checklist of potential causes and solutions, covering authentication, authorization, network restrictions, encryption policies, and more.

When you access Azure Blob Storage, you might encounter HTTP 403 errors that prevent clients from reading, writing, or managing data. These errors can result from a variety of configuration or authorization issues, including role assignments, token settings, network restrictions, encryption policies, and more. This article provides a comprehensive checklist and troubleshooting guidance to help you identify the root cause of 403 errors and resolve them efficiently. If you already know the error code or cause category, use the links in the [diagnostic checklist](#) to jump directly to the relevant section.

About 403 errors

A *403 error* is an HTTP status code that includes the string `403` (for example, `Forbidden (403)`). Each 403 error is accompanied by a more specific error code, such as `UnauthorizedBlobOverwrite` or `QueryParameterSasMandatory`. These detailed error codes can help you narrow down the cause of the issue. The following articles describe the most common types of 403 errors:

- [Azure Blob Storage error codes](#)
- [Common REST API error codes](#)
- [Common Error Codes for Storage Resource Provider](#)
- [Shared access signature error codes](#)

A 403 error can result from either *control plane* or *data plane* operations. Control plane operations are Azure Resource Manager requests, such as creating a storage account or updating its properties. Data plane operations involve requests to the storage service endpoint, such as uploading or downloading blobs. You can use Azure resource logs to find error details, including information about the security principal, key, or SAS used for authorization.

Common 403 errors related to authentication failures

Authentication failures occur when there are issues with credentials, tokens, or authentication methods.

 Expand table

Error	Causes and solutions
Error message Server failed to authenticate the request. Make sure the value of Authorization header is formed correctly including the signature. Error code AuthenticationFailed Error detail Signed expiry time <code><date></code> has to be after signed start time <code><date></code>	Expired token: The SAS token has passed its expiration time. Generate a new SAS token with a future expiration time. Clock skew: Time differences between systems can make tokens appear expired early. Consider adding buffer time to expiration. Start time in future: Avoid setting a start time if the SAS is for immediate use.
Error message Server failed to authenticate the request. Make sure the value of Authorization header is formed correctly including the signature. Error code AuthenticationFailed Error detail The MAC signature found in the HTTP request 'xyz...' is not the same as any computed signature.	Wrong account key: Verify you're using the correct primary or secondary key for the storage account. Regenerated keys: If storage access keys were recently regenerated, update your application with the new key. Encoding issues: Ensure the account key is properly encoded and formatted.
Error message Server failed to authenticate the request. Make sure the value of Authorization header is formed correctly including the signature. Error code AuthenticationFailed Error detail SAS field 'sv' is not well formed.	Invalid SAS format: Use official Azure SDKs or tools like Storage Explorer to generate SAS tokens. Manual construction errors: If you construct SAS tokens manually, verify all required fields and their formats. Version mismatch: Ensure the <code>sv</code> (version) parameter matches your Storage Client Library version.
Error message Server failed to authenticate the request. Make sure the value of Authorization header is formed correctly including the signature.	Service vs Account SAS: You're using a service SAS (for specific blobs or containers) for account-level operations.

Error	Causes and solutions
Error code AuthenticationFailed	Scope mismatch: Generate the appropriate SAS type for your operation. Use an account SAS for account-level operations (list containers), and use a service SAS for specific blob or container operations.
Error detail The specified signed resource is not allowed for the this resource level	
Error message Key based authentication is not permitted on this storage account.	Policy restriction: The storage account is configured to disallow shared key authentication.
Error code KeyBasedAuthenticationNotPermitted	Switch to Microsoft Entra ID: Use Microsoft Entra ID authentication instead of account keys.
	Enable shared key: If needed, re-enable shared key access in the storage account configuration.

Common 403 errors related to authorization failures

Authorization failures occur when a client has valid credentials but insufficient permissions for the requested operation.

 Expand table

Error	Causes and solutions
Error message This request isn't authorized to perform this operation using this permission.	Missing permissions: The SAS token doesn't include the required permission (read, write, delete, list).
Error code AuthorizationPermissionMismatch	Operation requirements: Some operations require multiple permissions (for example, overwriting a blob requires both write and delete).
	Check SAS permissions: Verify the <code>sp</code> (permissions) field in your SAS token includes all necessary permissions.
	Insufficient Azure RBAC role: The user or service principal lacks the required role for the operation.
	For read operations, use Storage Blob Data Reader. For write operations, use Storage Blob Data Contributor. For management operations, use Storage Blob Data Owner.
	Role assignment scope: Ensure roles are assigned at the

Error	Causes and solutions
	appropriate scope (subscription, resource group, storage account, or container).
Error message This request isn't authorized to perform this operation using this service.	Wrong service type: The SAS token was created for a different Azure Storage service.
Error code AuthorizationServiceMismatch	Service field mismatch: Check the <code>ss</code> (services) field in your SAS token. Use <code>b</code> for Blob service Use <code>f</code> for File service Use <code>q</code> for Queue service Use <code>t</code> for Table service.
Error message This request isn't authorized to perform this operation	Public network access disabled: The storage account doesn't allow public network access.
Error code AuthorizationFailure	IP address restrictions: Your IP address isn't included in the allowed IP ranges. Virtual network restrictions: Access is restricted to specific virtual networks or subnets. By default, traffic to all subnets is blocked. Requests to resources in a subnet receive a 403 error until you add a rule that allows traffic to that subnet. Firewall rules: Storage account firewall is blocking your request.

Comprehensive 403 error diagnostic checklist

If you receive an error other than the errors described in the previous sections, or if the causes and solutions described in those sections don't help you resolve your error, use this checklist as a guide to identify the cause and find a solution.

- ✓ **Secure file transfer:** If secure transfer is required, make sure that requests are made over HTTPS. ([Learn more](#)).
- ✓ **Azure role-based access control:** Verify the security principal has the correct Azure RBAC role. ([Learn more](#)).
- ✓ **Shared access signature (SAS) tokens:** Check token expiration, permissions, and generation method. ([Read more](#)).

- ✓ **User delegation SAS:** Confirm required fields, permissions, and key status for user delegation SAS. ([Read more](#)).
- ✓ **Stored access policies:** Review stored access policy settings and the propagation delay. ([Read more](#)).
- ✓ **Access control lists (ACLs):** Ensure ACL entries and permissions are correct for the client. ([Read more](#)).
- ✓ **Shared Key authorization:** If you're using an account key, confirm that Shared Key authorization is allowed. ([Learn more](#)).
- ✓ **Public network endpoint:** Check public endpoint access, firewall rules, and network security perimeter. ([Read more](#)).
- ✓ **Private endpoints:** Validate private endpoint subnet and resource configuration. ([Read more](#)).
- ✓ **Encryption scopes:** Review encryption scope settings and customer-managed key status. ([Read more](#)).
- ✓ **Disabled accounts:** Confirm the storage account or subscription isn't disabled. ([Read more](#)).
- ✓ **Azure Databricks and Azure RBAC:** Scope roles to the resource group, account, or container. ([Read more](#)).
- ✓ **Check scope of copy operations:** Check if the account settings restrict the scope of copy operations. ([Learn more](#)).

SAS tokens

Common causes of 403 errors when you use SAS tokens include:

- **Inappropriate SAS token expiration or start time:** The SAS token's expiration and start times aren't set correctly for the intended usage.
- **Missing permissions:** Not all necessary permissions are selected for the SAS token.
- **Insufficient permissions:** The client attempts an operation that isn't permitted by the SAS token. For example, the client attempts to write to a blob with a SAS that permits only read operations. Some actions require extra permission. For example, for a client to overwrite a blob, the SAS token must permit both write and delete permission.

- **Token generation method:** You didn't generate the SAS token by using an official SDK or tool (such as Storage Explorer). To avoid errors in the signature string, consider using official SDKs to generate SAS tokens. If you construct SAS tokens manually, use the latest storage service version and libraries. See [Delegating Access with a Shared Access Signature](#).
- **Expired SAS token:** The client is using a SAS token that is expired.
- **Clock skew:** Small time differences between the host generating the SAS and the storage service cause the SAS to be invalid or expire earlier than expected.
- **Incorrect SAS start time:** Avoid setting a start time if the SAS is for immediate use, as clock differences can make the SAS not yet valid.
- **Short expiry time:** Setting a short expiry time can result in premature expiration due to clock differences.
- **Version mismatch:** The `sv` (version) parameter in the SAS token should match the version of the Storage Client Library in use. Always use the latest library version.
- **Regenerated access keys:** If you regenerate storage access keys, you might invalidate existing SAS tokens, especially those with long expiry times.
- **Malformed signature:** The SAS or shared key signature is incorrectly formed, encoded, or contains invalid parameters.
- **Unauthorized IP address:** The request originates from an IP not authorized by the SAS token. Verify the "sip" field matches the client's IP address.
- **Interference by a stored access policy:** A container with stored access policies can override the permission given by a SAS. For example, a stored access policy on a container might revoke permission earlier than the SAS expiry date and time.

See [Best practices when using SAS](#).

User delegation SAS tokens

Common causes of 403 errors when associated with user delegation SAS tokens include:

- **Required fields missing:** Ensure all mandatory fields specific to User delegation SAS are present: `skoid`, `sktid`, `skt`, and `ske`. If any are missing, authentication fails.
- **Unsupported API version:** Only REST API versions later than "2018-11-09" are supported for User delegation SAS.

- **Invalid token:** The SAS token's start (`st`) and end (`se`) times must be within the delegation key's start (`skt`) and end (`ske`) times.
- **Request timing:** The request start time must be within the SAS token's valid time window.
- **Signature mismatch:** The SAS signature provided by the client must match the signature generated by the storage service. For example, a SAS created for container X fails if used to access container Y.
- **Insufficient SAS permission:** The SAS token must permit the requested operation. For example, a "Read" permission can't be used for a "Write" request. Check the "sp" field.
- **Insufficient Azure RBAC role:** The user must have the necessary Azure role for the requested operation. For example, a write operation requires the Storage Blob Data Contributor role.
- **OAuth authentication:** Ensure OAuth (Azure RBAC) authorization passes for the requested operation.
- **Revoked key:** The request is made after the user delegation key is revoked or expired.

See [Create a user delegation SAS](#).

Stored access policies

When you use stored access policies, review the following items:

- **Policy settings:** Ensure the permissions, start time, and expiration time defined in the stored access policy are correct for your scenario.
- **SAS token:** Ensure the SAS token associated with the policy matches the intended permissions and time window.

After you apply or update a stored access policy, allow up to 30 seconds for the policy to take effect. During this interval, any SAS associated with the policy might fail with a 403 (Forbidden) error until the policy becomes active.

To learn more, see [Define a stored access policy](#).

Access control lists (ACLs)

When you use ACLs to authorize access to files and directories, review the following items.

- **Security principal membership:** The security principal used by the client must appear in an ACL entry for the target file or directory. If the ACL entry includes only security groups, the principal must be a member of one of those groups.
- **Correct permissions:** The ACL entry associated with the security principal grants the appropriate permission level for the intended operation.
- **Sticky bit isn't set:** ACLs have a setting called the sticky bit, which can also cause 403 errors. If the sticky bit is enabled on a directory, only the child item's owner, the directory's owner, or the Superuser (\$superuser) can delete or rename child items. To learn more, see [403 Access Denied authorization error when the sticky bit is enabled in ADLS Gen2](#).

For more information about ACLs, see [Access control lists \(ACLs\) in Azure Data Lake Storage Gen2](#).

Public network endpoint

If the client receives a 403 error when accessing a storage account through its public endpoint, review the following items.

- **Firewall rules:** If firewall rules are enabled, ensure the client is making requests from an allowed IP address or subnet. If the storage account allows traffic only from a specific virtual network subnet, any attempt to access data from a public network is forbidden. For example, if a user tries to access the storage account from the Azure portal (which uses the browser's network) or from a local machine, a 403 error appears unless you add the IP address of the browser network or local machine to the IP address rules. See [Azure Storage firewall rules](#).
- **Source and destination access:** For operations that copy data between storage accounts, the client must have network access to both the source and destination accounts.
- **Azure service access:** If the 403 error originates from other services which need to interact with your storage account's backend, confirm the service is added as an allowed [resource instance](#) or that a network security exception for trusted Azure services is configured. See [Trusted Azure services](#).
- **Network security perimeter rules:** If the storage account is part of a network security perimeter, those rules override firewall settings. Even if the client is allowed by the firewall, a restrictive network security perimeter rule can still result in a 403 error. Enable resource logs

to identify which rule is causing the issue. See [Diagnostic logs for Network Security Perimeter](#).

- **Service endpoint policies:** If a service endpoint policy is defined for the resource group, subscription, or storage account, review the policy rules. Either remove the policy or ensure the storage account is included. Changes to service endpoint policies can take up to 15 minutes to take effect. See [Virtual network service endpoint policies for Azure Storage](#).

Private endpoints

If the client receives a 403 error when accessing a storage account through a private endpoint, review the following items.

- **Domain Name System (DNS) settings:** Make sure that the DNS resolution of the storage account name resolves to a private IP address. A DNS configuration might be resolving the storage account to the public endpoint. Update the DNS configuration so that clients in the network actually get to the private endpoint. See [DNS changes for private endpoints](#).
- **Hub virtual network connectivity:** For copying blobs between storage accounts with private endpoints in different spoke virtual networks from a VM in a Hub virtual network, ensure network connectivity and review error logs for `CannotVerifyCopySource`. See [Copy blobs between storage accounts in a Hub-spoke architecture using private endpoints](#).
- **Private endpoints for each storage resource:** Ensure the storage account has private endpoints for both the Data Lake Storage resource and the Blob Storage resource. Some operations might be redirected between endpoints, requiring both to be present.
- **Source and destination access:** When copying blobs between storage accounts, the client must have network access to both accounts. If you're using private link for only one account, ensure network access to the other account as well.

For general troubleshooting, see [Troubleshoot Azure Private Endpoint connectivity problems](#).

Encryption scopes

When you use encryption scopes, review the following items.

- **Encryption scope override setting:** If a container enforces a default encryption scope and prohibits overrides, uploading a blob with a different encryption scope results in a 403 error. You can set the override prohibition only when you create the container. To learn more, see [Create a container with a default encryption scope](#). To check if overrides are prohibited,

examine the container properties. See [Manage blob containers using PowerShell](#) or [Manage blob containers using Azure CLI](#).

- **Disabled key:** If an encryption scope uses a customer-managed key and you disable the key, all access attempts to data in that container fail with a 403 error. See [Revoke access to a storage account that uses customer-managed keys](#).

Disabled accounts

A 403 error can occur if you attempt to access data in a storage account or subscription that is disabled. Common reasons for a disabled subscription include expired credit, reaching a spending limit, overdue bills, exceeding your credit card limit, or the Account Administrator canceling the subscription.

Azure Databricks and Azure RBAC role assignments

A 403 error can occur if your subscription includes an Azure Databricks namespace. Roles scoped to the subscription don't grant access to blob data. Assign roles at the resource group, storage account, or container level instead. To reassign a role, go to the storage account in the Azure portal, select **Access control (IAM)**, and add the role assignment at the container or storage account scope.

See also

- [Auditing control plane operations](#)
- [Auditing data plane operations](#)

403 Access Denied authorization error when the sticky bit is enabled in ADLS Gen2

This article helps you understand the sticky bit and provides information about how to check this setting when you configure it in Azure Data Lake Storage (ADLS) Gen2 and get issues.

What is the sticky bit in ADLS Gen2?

ADLS Gen2 users often need to manage permissions for different users, and one way to do this is by using an access control list (ACL). ACL is a POSIX-like access control system with a specific setting called the sticky bit that can cause authorization failures. For more information about the permission control mode and the sticky bit, see [Access control lists \(ACLs\) in Azure Data Lake Storage Gen2](#).

The sticky bit is an advanced feature that isn't necessary in the ACL setting of the ADLS Gen2 storage account. Instead, you can use the [mask feature](#) to limit the maximum permissions for named users, named groups, and the owning group. This works similarly to the sticky bit and is easily configured in the Azure portal.

403 Access Denied authorization error

Consider the following scenario:

- An ADLS Gen2 storage account has a container called *container* and a folder path called *folder/child-folder*.
- You use ACL as an authorization method.
- In the ACL setting of the ADLS Gen2 storage account, you're configured with Execute (X) permission on the root directory and *folder* and with Write and Execute (WX) permission on *child-folder*.
- The sticky bit is enabled on *child-folder*.
- You try to create or upload a new file, for example, *test.txt*, into the ADLS Gen2 Storage Account folder path *container/folder/child-folder/*.

In this scenario, you get a 403 Access Denied authorization error.

This error occurs for two reasons:

- You don't have enough permissions to access the folder path.
- You have enough permissions, but enabling the sticky bit causes you not to be the owner of this folder path.

Identify whether the sticky bit causes a 403 Access Denied error

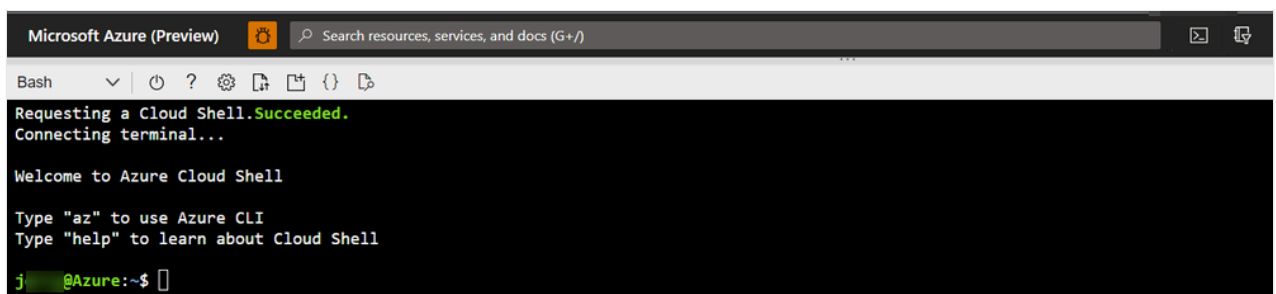
Verify the ACL setting of the folder and the parent folders, and then compare it with the [Common scenarios related to ACL permissions](#). If the permissions are enough, then the 403 error may be caused by the sticky bit.

Verify the sticky bit setting by using Azure CLI

There are many ways to check this setting, such as a REST API call, PowerShell command, and Azure CLI. We recommend the Azure CLI option because it doesn't require any extra software to be installed, and the command is easy to understand.

To verify the sticky bit setting by using Azure CLI, follow these steps:

1. Sign in to the Azure portal with your account. Make sure that this account has the Storage Blob Data Owner role assignment on the ADLS Gen2 storage account.
2. Select **Cloud Shell** from the Azure portal.



3. Use the following command to get the ACL and the sticky bit setting of the *container/folder* directory:

Azure CLI

```
az storage fs access show -p folder -f container --account-name account --auth-mode login
```

To check the ACL and the sticky bit setting of the root directory, which is the container level ACL and the sticky bit setting, use the following command:

Azure CLI

```
az storage fs access show -p / -f container --account-name account --auth-mode login
```

Here's a sample output:

```
jerry [ ~ ]$ az storage fs access show -p test1 -f testacl --account-name jerrygen2 --auth-mode login
{
  "acl": "user::rwx,user:82...:r-x,group::r-x,mask::rwx,other::rwx",
  "group": "$superuser",
  "owner": "$superuser",
  "permissions": "rwxrwxrwx+"
}
```

In the response JSON body, focus on `permissions`. It normally contains 9 or 10 bits with an extra "+" symbol. For more information about these letters, see [Users and identities](#).

The previous example indicates that all user permissions are enabled, and the sticky bit is enabled. For more information about how to read this permission notation, see [Notation of traditional Unix permissions](#).

The ninth letter has four possible values: "-", "x", "t", and "T". If the value of this letter is "t" or "T", it means that the sticky bit is enabled. The "t" is "x" with the sticky bit enabled, and "T" is "-" with the sticky bit enabled.

"rwxrwxrwt" can be explained by the following:

- r,w, and x permissions are enabled for the Owner.
- r,w, and x permissions are enabled for the Owning group.
- r,w, and x permissions are enabled for Other users, and the sticky bit is enabled.

To understand it better, here's another example for "rwxr-xr-T":

- r,w, and x permissions are enabled for the Owner.
- r and x permissions are enabled for the Owning group.
- Only r permission is enabled for Other users, and the sticky bit is enabled.

According to [Short forms for permissions](#), short form permission is calculated for every group of three letters ("r" as 4, "w" as 2, and "x" as 1). So, "rw-rwx--x" will be equal to 4+2+0, 4+2+1, 0+0+1, 671. Based on this calculation rule, you only need to add the

fourth letter at the beginning. If the sticky bit is enabled, set it as 1. If the sticky bit is disabled, set it as 0.

Here are some examples:

- `rw-rwxrwt => 1777`
- `rw-r-xr-T => 1754`
- `rw-rwx--x => 0671`

Disable/Enable the sticky bit setting

To disable/enable the sticky bit setting, set permissions to expected values.

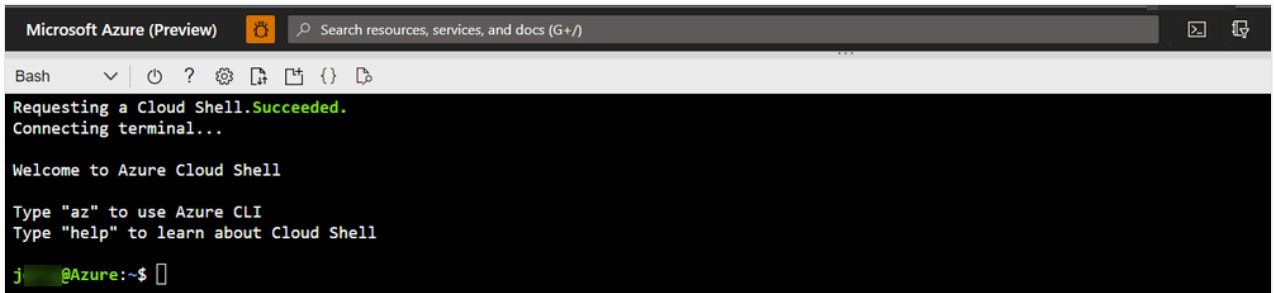
The Azure account used to modify this setting must have the Storage Blob Data Owner role on the target ADLS Gen2 storage account. There are many possible ways to modify the sticky bit setting. Here are the supported SDKs:

 Expand table

SDK	Supported version	More information
REST API	2019-12-12 and later API versions	Path
Az PowerShell module	5.6.0 and later versions	Use PowerShell to manage ACLs in Azure Data Lake Storage Gen2
Azure CLI	2.38.0 and later versions	Use Azure CLI to manage ACLs in Azure Data Lake Storage Gen2
.NET SDK	12.14.0 and later versions	Use .NET to manage ACLs in Azure Data Lake Storage Gen2
Java SDK	12.11.0 and later versions	Use Java to manage ACLs in Azure Data Lake Storage Gen2
Python SDK	12.8.0 and later versions	Use Python to manage ACLs in Azure Data Lake Storage Gen2
JavaScript SDK	12.11.0-beta.1 and later versions	Use JavaScript SDK in Node.js to manage ACLs in Azure Data Lake Storage Gen2
HDFS CLI	3.3.3 and later versions	Using the HDFS CLI with Data Lake Storage Gen2 and Apache Hadoop 3.3.3 – HDFS Permissions Guide

Here's an example of disabling/enabling the sticky bit setting with Azure CLI.

1. Sign in to the Azure portal with your account that has the Storage Blob Data Owner role assignment on the target ADLS Gen2 storage account.
2. Select **Cloud Shell** from the Azure portal.



```
Microsoft Azure (Preview) Search resources, services, and docs (G+/)
Bash
Requesting a Cloud Shell. Succeeded.
Connecting terminal...

Welcome to Azure Cloud Shell

Type "az" to use Azure CLI
Type "help" to learn about Cloud Shell

j @Azure:~$
```

3. To set the ACL and the sticky bit setting of the *container/folder* directory to the "rwxrwxrwt" permissions and to enable the sticky bit, use the following command:

Azure CLI

```
az storage fs access set --permissions rwxrwxrwt -p folder -f container --account-name account --auth-mode login
```

To modify the setting of the root directory, which is the container level ACL and the sticky bit setting, use the following command:

Azure CLI

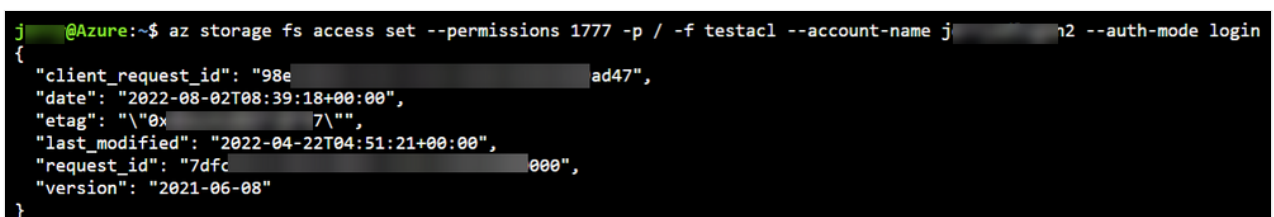
```
az storage fs access set --permissions rwxrwxrwt -p / -f container --account-name account --auth-mode login
```

The {permission notation} in the previous command can accept both long and short-form notations. This means that the following command is also qualified:

Azure CLI

```
az storage fs access set --permissions 1776 -p folder -f container --account-name account --auth-mode login
```

Here's an example output:



```
j @Azure:~$ az storage fs access set --permissions 1777 -p / -f testacl --account-name j[redacted]n2 --auth-mode login
{
  "client_request_id": "98e[redacted]ad47",
  "date": "2022-08-02T08:39:18+00:00",
  "etag": "\"0x[redacted]7\"",
  "last_modified": "2022-04-22T04:51:21+00:00",
  "request_id": "7dfc[redacted]000",
  "version": "2021-06-08"
}
```


How to use PerfInsights to diagnose high CPU or disk usage issues

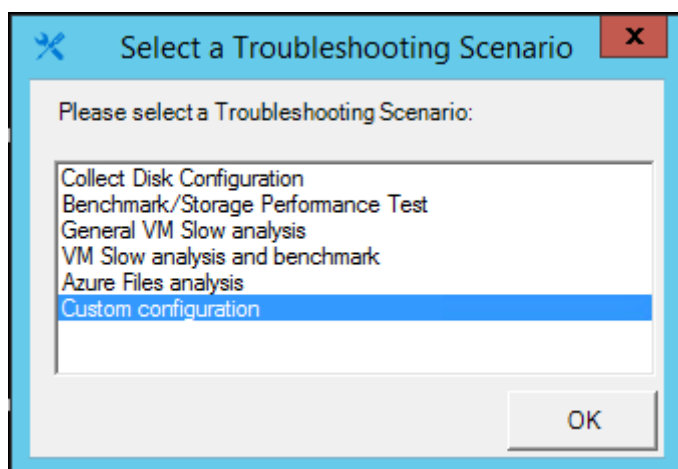
Original product version: Azure

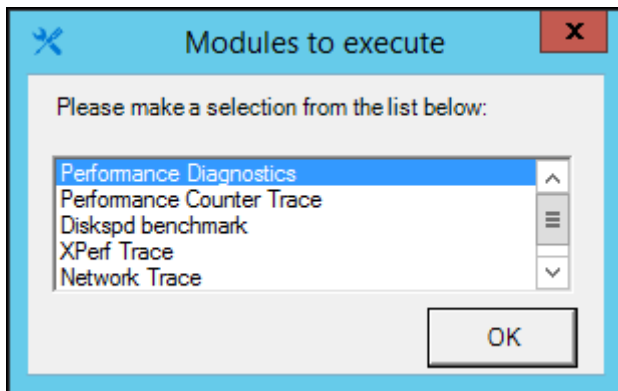
Original KB number: 4021853

This article describes how to use PerfInsights to diagnose high CPU or disk usage issues in Microsoft Azure.

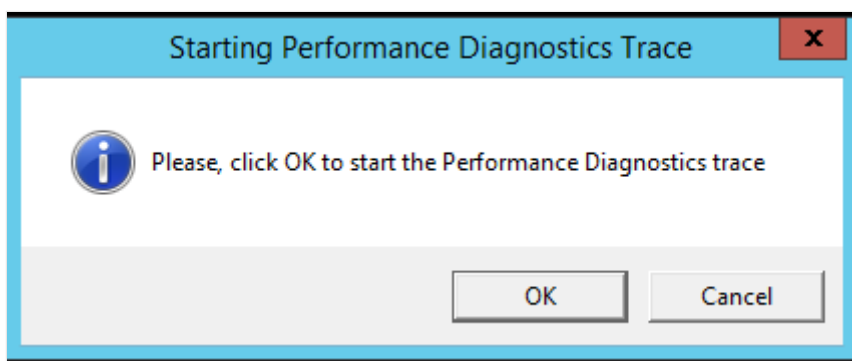
Use PerfInsights to collect log data

1. Follow the general PerfInsights User's Guide to start PerfInsights, and then follow these steps:
 - a. Enter the Microsoft Support request number.
 - b. In the **Select a Troubleshooting Scenario** dialog box, select **Custom configuration**.
 - c. In the **Modules to Execute** dialog box, select **Performance Diagnostics**.
 - d. Click **OK** to continue.

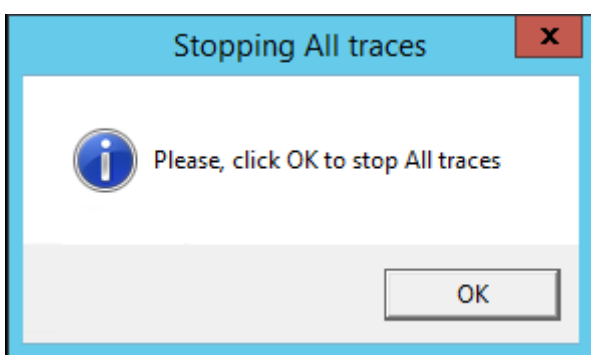




2. Wait for PerfInsights to collect system information. After that job is complete, the **Starting Performance Diagnostics Trace** window appears. Click **OK** to continue if the issue persists.



3. The **Stopping All traces** windows appears. We recommend that you wait several minutes to let the tool capture enough data for diagnostics. Then, click **OK** to stop the diagnostics. The final diagnostics result will be included in the CollectedData_<DateTime>.zip file.



Analyze the diagnostics report for performance issues

1. Follow the general PerfInsights User's Guide to open the PerfInsights Report html file.

If any high CPU or disk usage data was captured during the Performance Diagnostics job, warning messages appear on the **Findings** tab. The messages contain links to the **Top**

CPU Consumer and Top Disk Consumer tabs.

Findings	Disk Configuration	Tasklist	Netstat	Storage Reliability Counters	Events	Filter driver	Performance Diagnostics
Overview							
VMName							FLIANG-TESTVM3
OSVersion							6.3.9600
TempDrive							D:
ScriptVersion							3.2.0
CommandLine							no
Scenario							Custom configuration
ExecutionTime							4/24/2017 5:24:52 PM
HasSQLInstance							false
Messages							
KB3153887 is not installed, but please consider installing it: KB3153887							
Throughput optimization not enabled, please consider to enable it: Optimize network throughput for VMs							
Volume I: has allocation unit size equal to 4K. For generic scenarios it is preferable to have 64K. Link to Best Practices							
Volume G: has allocation unit size equal to 4K. For generic scenarios it is preferable to have 64K. Link to Best Practices							
Volume F: has allocation unit size equal to 4K. For generic scenarios it is preferable to have 64K. Link to Best Practices							
Volume H: has allocation unit size equal to 4K. For generic scenarios it is preferable to have 64K. Link to Best Practices							
High Disk usage periods were detected. Look Top Disk Consumers tab for further details							
High CPU usage periods were detected. Look Top CPU Consumers tab for further details							

2. You can use the **Top CPU Consumers** tab to identify the periods in which high CPU usage occurred and the top processes show the most CPU consumption. This tab includes the following information:

- **StartTime** and **EndTime**. The start time and end time of the analysis period are shown as UTC time. Additionally, there are some notes about how the data was collected and evaluated.
- **High CPU Usage Periods**. This table lists all high CPU usage that occurred between the start and end times. For example, the following table shows one period of high CPU usage at **30%+**. The nested table in the **ProcessorsHighCPUUsageBreakdown** column shows the usage of each logical processor.

High CPU Usage Periods						
The table below lists the periods in which overall CPU usage of all processors were high(30%+) or any of processor was busy(80%+)						
StartTime	EndTime	AllProcessorsCPUUsageMS	AllProcessorsCPUUsage%	ProcessorsHighCPUUsageBreakdown		
4/24/2017 5:27:48 PM	4/24/2017 5:28:13 PM	37483	37.45	ProcessorNumber	CPUUsageMS	CPUUsage%
				0	8390	33.53
				1	9188	36.72
				2	9906	39.59
				3	9999	39.96

- Top long-running CPU consumers. This table shows which processes were using CPU. For example, **ieexplore.exe** (4964) contributed 39.05% CPU usage to the period of high CPU usage period. If the top consumers are your own services or programs, you must use a more powerful profiler tool to do deep analysis. If the top consumers are owned by third parties, you might want to disable or uninstall that service or program, or contact the manufacturer for customer support.

Top long running CPU consumers						
The table below lists the top long running CPU consumers during Performance Diagnostics analysis period. Sorted by TotalCPUUsageMS in descending order.						
ProcessName	ProcessId	ServicesHosted	TotalCPUUsageMS	TotalCPUUsage%	ProcessesHighCPUUsageBreakdown	
ieexplore.exe	4964		5860	39.05	CPUUsageMS	StartTime
					5860	4/24/2017 5:27:53 PM
ieexplore.exe	2024		5608	112.10	CPUUsageMS	StartTime
					5608	4/24/2017 5:28:03 PM
ieexplore.exe	4664		4579	22.88	CPUUsageMS	StartTime
					4579	4/24/2017 5:27:48 PM
lsass.exe	624	KeyIso	1796	5.98	CPUUsageMS	StartTime
					1796	4/24/2017 5:27:48 PM
ieexplore.exe	5352		1422	14.21	CPUUsageMS	StartTime
					1422	4/24/2017 5:27:58 PM

- Top spike CPU consumers. This table shows the processes that had high spike usage but did not consume CPU for a long time.

3. You can use the **Top Disk Consumers** tab to identify the periods in which high physical disk usage occurred and the top processes that show the most disk consumption. This tab includes the following information:

- StartTime and EndTime. The start time and end time of the analysis period are shown as UTC time. Additionally, there are some notes about how the data was collected and evaluated.
- High Disk Usage Periods. This table lists all high disk usage that occurred between the start and end times. For example, the disk usage of PhysicalDisk3 was high during the period between StartTime and EndTime. The nested table in the **DisksHighUsageBreakdown** column shows the usage of each physical disk.

High Disk Usage Periods									
The table below lists the periods in which any of physical disks was busy (IOPS >= 500 OR Throughput >= 100MB/s OR Utilization >= 90%)									
StartTime	EndTime	DisksHighUsageBreakdown							
4/24/2017 5:27:18 PM	4/24/2017 5:27:43 PM	DiskNumber	AvgIOPS	Throughput (KB/sec)	ReadCount	BytesReadKB	WriteCount	BytesWriteKB	MaxQueueDepth
		PhysicalDisk3	2,275.59	7,486.78	57045	93840	0	362520	64
		PhysicalDisk0	1.28	0.00	0	0	32	200	0
		PhysicalDisk1	0.96	0.00	0	0	24	136	0
		PhysicalDisk2	0.00	0.00	0	0	0	0	0
		PhysicalDisk4	0.00	0.00	0	0	0	0	0
		PhysicalDisk5	0.00	0.00	0	0	0	0	0

- Top long-running CPU consumers. This table shows which processes were accessing the disk. For example, diskspd.exe(5172) issued 47,037 disk IOs, and its average IOPS was 2,344. If the top consumers are your own services or programs, you must investigate why they issued the large number of disk IOs. If the top consumers are owned by third parties, you might want to disable or uninstall those services or programs, or contact the manufacturer for customer support.

Top long running Disk consumers											
The table below lists the top long running disk consumers during Performance Diagnostics analysis period. Sorted by TotalDiskIoCount in descending order.											
ProcessName	ProcessId	ServicesHosted	TotalDiskIoCount	TotalDiskIoSizeKB	AvgIOPS	ProcessesHighDiskUsageBreakdown					
diskspd.exe	5172		47037	376296	2,344.87	ReadCount	BytesReadKB	WriteCount	BytesWriteKB	StartTime	EndTime
						47037	376296	0	0	4/24/2017 5:27:18 PM	4/24/2017 5:27:38 PM
iexplore.exe	5868		3921	6092	156.72	ReadCount	BytesReadKB	WriteCount	BytesWriteKB	StartTime	EndTime
						2055	2853	1866	3239	4/24/2017 5:27:48 PM	4/24/2017 5:28:13 PM
iexplore.exe	2024		2838	12249	567.30	ReadCount	BytesReadKB	WriteCount	BytesWriteKB	StartTime	EndTime
						1612	9570	1226	2679	4/24/2017 5:28:03 PM	4/24/2017 5:28:08 PM

- Top spike CPU consumers. This table shows the processes that had high spike usage but did not consume CPU for a long time.

Troubleshoot availability issues in Azure storage accounts

This article helps you investigate changes in availability (such as the number of failed requests). These changes in availability can often be identified by monitoring storage metrics in Azure Monitor. For general information about using metrics and logs in Azure Monitor, see the following articles:

- [Monitoring Azure Blob Storage](#)
- [Monitoring Azure Files](#)
- [Monitoring Azure Queue Storage](#)
- [Monitoring Azure Table storage](#)

Monitoring availability

You should monitor the availability of the storage services in your storage account by monitoring the value of the **Availability** metric. The **Availability** metric contains a percentage value. It's calculated by taking the total billable requests value and dividing it by the number of applicable requests, including those requests that produced unexpected errors.

Any value less than 100% indicates that some storage requests are failing. You can see why they're failing by examining the **ResponseType** dimension for error types such as **ServerTimeoutError**. You should expect to see **Availability** fall temporarily below 100% for reasons such as transient server timeouts while the service moves partitions to better load-balance requests; the retry logic in your client application should handle such intermittent conditions. The **Availability** metric will only be available for time periods when transactions also occur on the account.

You can use features in Azure Monitor to alert you if **Availability** for a service falls below a threshold that you specify.

Metrics show an increase in throttling errors

Throttling errors occur when you exceed the scalability targets of a storage service. The storage service throttles to ensure that no single client or tenant can use the service at the expense of others. For more information, see [Scalability and performance targets for standard storage](#)

[accounts](#) for details on scalability targets for storage accounts and performance targets for partitions within storage accounts.

If the **ClientThrottlingError** or **ServerBusyError** value of the **ResponseType** dimension shows an increase in the percentage of requests that are failing with a throttling error, you need to investigate one of two scenarios:

- Transient increase in throttling errors
- Permanent increase in throttling errors

An increase in throttling errors often occurs at the same time as an increase in the number of storage requests or when you're initially load testing your application. This may also manifest itself in the client as "503 Server Busy" or "500 Operation Timeout" HTTP status messages from storage operations.

Important

Increased throttling doesn't always indicate that a storage account has reached its overall scalability limits. Throttling can occur when a workload concentrates a high volume of requests against a small set of partition keys, creating a hot partition. In these situations, applications might experience increased latency or receive HTTP 503 (Server Busy) and HTTP 500 (Operation Timeout) responses. To reduce throttling, implement an exponential backoff retry policy and design workloads so requests are distributed across partitions. Avoid naming patterns that concentrate traffic on a narrow range of blob names, and review hot partition mitigation guidance if throttling continues. For more information, see [Mitigate hot partitions in Azure Blob Storage](#).

Transient increase in throttling errors

If you're seeing spikes in throttling errors that coincide with periods of high activity for the application, you implement an exponential (not linear) back-off strategy for retries in your client. Back-off retries reduce the immediate load on the partition and help your application smooth out spikes in traffic. For more information about how to implement retry policies using the Storage Client Library, see the [RetryOptions.MaxRetries](#) property.

Note

You may also see spikes in throttling errors that do not coincide with periods of high activity for the application. The most likely cause is the storage service moving partitions to improve

load balancing.

Permanent increase in throttling errors

If you're seeing a consistently high value for throttling errors following a permanent increase in your transaction volumes or when you're performing your initial load tests on your application, then you need to evaluate how your application is using storage partitions and whether it's approaching the scalability targets for a storage account. For example, if you're seeing throttling errors on a queue (which counts as a single partition), then you consider using additional queues to spread the transactions across multiple partitions. If you're seeing throttling errors on a table, consider using a different partitioning scheme to spread your transactions across multiple partitions by using a wider range of partition key values. One common cause of this issue is the prepend/append anti-pattern, where you select the date as the partition key, and then all data on a particular day is written to one partition (under load, this can result in a write bottleneck). Consider a different partitioning design or evaluate whether using blob storage might be a better solution. Also, check whether throttling is occurring due to spikes in your traffic and investigate ways of smoothing your pattern of requests.

If you distribute your transactions across multiple partitions, you must still be aware of the scalability limits set for the storage account. For example, if you used 10 queues, each processing the maximum of 2,000 1KB messages per second, you'll be at the overall limit of 20,000 messages per second for the storage account. If you need to process more than 20,000 entities per second, consider using multiple storage accounts. You should also bear in mind that the size of your requests and entities impacts when the storage service throttles your clients. If you have larger requests and entities, you may be throttled sooner.

Inefficient query design can also cause you to hit the scalability limits for table partitions. For example, a query with a filter that only selects one percent of the entities in a partition but that scans all the entities in a partition will need to access each entity. Every entity read will count toward the total number of transactions in that partition. Therefore, you can easily reach the scalability targets.

Note

Your performance testing should reveal any inefficient query designs in your application.

Metrics show an increase in timeout errors

Timeout errors occur when the **ResponseType** dimension is equal to **ServerTimeoutError** or **ClientTimeout**.

Your metrics show an increase in timeout errors for one of your storage services. At the same time, the client receives a high volume of "500 Operation Timeout" HTTP status messages from storage operations.

 Note

You may see timeout errors temporarily as the storage service load balances requests by moving a partition to a new server.

The server timeouts (**ServerTimeOutError**) are caused by an error on the server. The client timeouts (**ClientTimeout**) happen because an operation on the server has exceeded the timeout specified by the client. For example, a client using the Storage Client Library can set a timeout for an operation.

Server timeouts indicate a problem with the storage service that requires further investigation. You can use metrics to see if you're hitting the scalability limits for the service and to identify any spikes in traffic that might be causing this problem. If the problem is intermittent, it may be due to load-balancing activity in the service. If the problem is persistent and isn't caused by your application hitting the scalability limits of the service, you should raise a support issue. For client timeouts, you must decide if the timeout is set to an appropriate value in the client and either change the timeout value set in the client or investigate how you can improve the performance of the operations in the storage service, for example, by optimizing your table queries or reducing the size of your messages.

Metrics show an increase in network errors

Network errors occur when the **ResponseType** dimension is equal to **NetworkError**. These occur when a storage service detects a network error when the client makes a storage request.

The most common cause of this error is a client disconnecting before a timeout expires in the storage service. Investigate the code in your client to understand why and when the client disconnects from the storage service. You can also use third-party network analysis tools to investigate network connectivity issues from the client.

See also

- [Troubleshoot client application errors](#)
 - [Troubleshoot performance issues](#)
 - [Monitor, diagnose, and troubleshoot your Azure Storage](#)
-

Last updated on 07/28/2026

Troubleshoot client application errors in Azure storage accounts

This article helps you investigate client application errors by using metrics, [client side logs](#), and resource logs in Azure Monitor.

Diagnosing errors

Users of your application may notify you of errors reported by the client application. Azure Monitor also records counts of different response types (**ResponseType** dimensions) from your storage services, such as **NetworkError**, **ClientTimeoutError**, or **AuthorizationError**. While Azure Monitor only records counts of different error types, you can obtain more detail about individual requests by examining server-side, client-side, and network logs. Typically, the HTTP status code returned by the storage service will give an indication of why the request failed.

① Note

Remember that you should expect to see some intermittent errors. For example, errors due to transient network conditions or application errors.

The following resources are useful for understanding storage-related status and error codes:

- [Common REST API error codes](#)
- [Blob Service error codes](#)
- [Queue Service error codes](#)
- [Table Service error codes](#)
- [File Service error codes](#)

The client is receiving HTTP 403 (Forbidden) messages

If your client application is throwing HTTP 403 (Forbidden) errors, a likely cause is that the client is using an expired Shared Access Signature (SAS) when it sends a storage request (although other possible causes include clock skew, invalid keys, and empty headers).

The Storage Client Library for .NET enables you to collect client-side log data that relates to storage operations performed by your application. For more information, see [Client-side Logging with the .NET Storage Client Library](#).

The following table shows a sample from the client-side log generated by the Storage Client Library that illustrates this issue occurring:

 Expand table

Source	Verbosity	Verbosity	Client request ID	Operation text
Microsoft.Azure.Storage	Information	3	85d077ab-...	Starting operation with location Primary per location mode PrimaryOnly.
Microsoft.Azure.Storage	Information	3	85d077ab-...	Starting synchronous request to <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/Synchronous_and_Asynchronous_Requests#Synchronous_request>
Microsoft.Azure.Storage	Information	3	85d077ab-...	Waiting for response.
Microsoft.Azure.Storage	Warning	2	85d077ab-...	Exception thrown while waiting for response: The remote server returned an error: (403) Forbidden.
Microsoft.Azure.Storage	Information	3	85d077ab-...	Response received. Status code = 403, Request ID = <Request ID>, Content-MD5 = , ETag = .
Microsoft.Azure.Storage	Warning	2	85d077ab-...	Exception thrown during the operation: The remote server returned an error: (403) Forbidden..
Microsoft.Azure.Storage	Information	3	85d077ab-...	Checking if the operation should be retried. Retry count = 0, HTTP status code = 403, Exception = The remote server returned an error: (403) Forbidden..
Microsoft.Azure.Storage	Information	3	85d077ab-...	The next location has been set to Primary, based on the location mode.

Source	Verbosity	Verbosity	Client request ID	Operation text
Microsoft.Azure.Storage	Error	1	85d077ab-...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (403) Forbidden.

In this scenario, you should investigate why the SAS token is expiring before the client sends the token to the server:

- Typically, you shouldn't set a start time when you create a SAS for a client to use immediately. If there are small clock differences between the host generating the SAS using the current time and the storage service, then it's possible for the storage service to receive a SAS that isn't yet valid.
- Don't set a very short expiry time on a SAS. Again, small clock differences between the host generating the SAS and the storage service can lead to a SAS expiring earlier than anticipated.
- Does the version parameter in the SAS key (for example, `sv=2015-04-05`) match the version of the Storage Client Library you're using? We recommend that you always use the latest version of the storage client library.
- If you regenerate your storage access keys, any existing SAS tokens may be invalidated. This issue may arise if you generate SAS tokens with a long expiry time for client applications to cache.

If you're using the Storage Client Library to generate SAS tokens, then it's easy to build a valid token. However, if you're using the Storage REST API and constructing the SAS tokens by hand, see [Delegating Access with a Shared Access Signature](#).

The client is receiving HTTP 404 (Not found) messages

If the client application receives an HTTP 404 (Not found) message from the server, this implies that the object the client was attempting to use (such as an entity, table, blob, container, or queue) doesn't exist in the storage service. There are a number of possible reasons for this, such as:

- The client or another process previously deleted the object.
- A Shared Access Signature (SAS) authorization issue.
- Client-side JavaScript code doesn't have permission to access the object.
- Network failure.

The client or another process previously deleted the object

In scenarios where the client is attempting to read, update, or delete data in a storage service, it's easy to identify in the storage resource logs a previous operation that deleted the object in question from the storage service. Often, the log data shows that another user or process deleted the object. The Azure Monitor logs (server-side) show when a client deleted an object.

In the scenario where a client is attempting to insert an object, it may not be immediately obvious why this results in an HTTP 404 (Not found) response, given that the client is creating a new object. However, if the client is creating a blob, it must be able to find the blob container. If the client is creating a message, it must be able to find a queue. And if the client is adding a row, it must be able to find the table.

You can use the client-side log from the Storage Client Library to better understand when the client sends specific requests to the storage service.

The following client-side log generated by the Storage Client library illustrates the problem when the client can't find the container for the blob it's creating. This log includes details of the following storage operations:

 Expand table

Request ID	Operation
07b26a5d-...	<code>DeleteIfExists</code> method to delete the blob container. This operation includes a <code>HEAD</code> request to check for the existence of the container.
e2d06d78...	<code>CreateIfNotExists</code> method to create the blob container. This operation includes a <code>HEAD</code> request that checks for the existence of the container. The <code>HEAD</code> returns a 404 message but continues.
de8b1c3c-...	<code>UploadFromStream</code> method to create the blob. The <code>PUT</code> request fails with a 404 message

Log entries:

Request ID	Operation Text
07b26a5d-...	Starting synchronous request to https://domemalldist.blob.core.windows.net/azuremblobcontainer.
07b26a5d-...	StringToSign = HEAD.....x-ms-client-request-id:07b26a5d-....x-ms-date:Tue, 03 Jun 2014 10:33:11 GMT.x-ms-version:2014-02-14./domemalldist/azuremblobcontainer.restype:container.
07b26a5d-...	Waiting for response.
07b26a5d-...	Response received. Status code = 200, Request ID = eeead849-...Content-MD5 = , ETag = "0x8D14D2DC63D059B";.
07b26a5d-...	Response headers were processed successfully, proceeding with the rest of the operation.
07b26a5d-...	Downloading response body.
07b26a5d-...	Operation completed successfully.
07b26a5d-...	Starting synchronous request to https://domemalldist.blob.core.windows.net/azuremblobcontainer.
07b26a5d-...	StringToSign = DELETE.....x-ms-client-request-id:07b26a5d-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azuremblobcontainer.restype:container.
07b26a5d-...	Waiting for response.
07b26a5d-...	Response received. Status code = 202, Request ID = 6ab2a4cf-..., Content-MD5 = , ETag = .
07b26a5d-...	Response headers were processed successfully, proceeding with the rest of the operation.
07b26a5d-...	Downloading response body.
07b26a5d-...	Operation completed successfully.
e2d06d78-...	Starting asynchronous request to https://domemalldist.blob.core.windows.net/azuremblobcontainer.
e2d06d78-...	StringToSign = HEAD.....x-ms-client-request-id:e2d06d78-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azuremblobcontainer.restype:container.
e2d06d78-...	Waiting for response.
de8b1c3c-...	Starting synchronous request to https://domemalldist.blob.core.windows.net/azuremblobcontainer/blobCreated.txt.
de8b1c3c-...	StringToSign = PUT...64.qCmF+TQLPhq/YVK50mP9ZQ==.....x-ms-blob-type:BlockBlob.x-ms-client-request-id:de8b1c3c-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azuremblobcontainer/blobCreated.txt.
de8b1c3c-...	Preparing to write request data.
e2d06d78-...	Exception thrown while waiting for response: The remote server returned an error: (404) Not Found..
e2d06d78-...	Response received. Status code = 404, Request ID = 353ae3bc-..., Content-MD5 = , ETag = .
e2d06d78-...	Response headers were processed successfully, proceeding with the rest of the operation.
e2d06d78-...	Downloading response body.
e2d06d78-...	Operation completed successfully.
e2d06d78-...	Starting asynchronous request to https://domemalldist.blob.core.windows.net/azuremblobcontainer.
e2d06d78-...	StringToSign = PUT...0.....x-ms-client-request-id:e2d06d78-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azuremblobcontainer.restype:container.
e2d06d78-...	Waiting for response.
de8b1c3c-...	Writing request data.
de8b1c3c-...	Waiting for response.
e2d06d78-...	Exception thrown while waiting for response: The remote server returned an error: (409) Conflict..
e2d06d78-...	Response received. Status code = 409, Request ID = c27da20e-..., Content-MD5 = , ETag = .
e2d06d78-...	Downloading error response body.
de8b1c3c-...	Exception thrown while waiting for response: The remote server returned an error: (404) Not Found..

Request ID	Operation Text
de8b1c3c-...	Response received. Status code = 404, Request ID = 0eaeab3e-..., Content-MD5 = , ETag = .
de8b1c3c-...	Exception thrown during the operation: The remote server returned an error: (404) Not Found..
de8b1c3c-...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (404) Not Found..
e2d06d78-...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (409) Conflict..

In this example, the log shows that the client is interleaving requests from the `CreateIfNotExists` method (request ID e2d06d78...) with the requests from the `UploadFromStream` method (de8b1c3c-...). This interleaving happens because the client application is invoking these methods asynchronously. Modify the asynchronous code in the client to ensure that it creates the container before attempting to upload any data to a blob in that container. Ideally, you should create all your containers in advance.

A Shared Access Signature (SAS) authorization issue

If the client application attempts to use a SAS key that doesn't include the necessary permissions for the operation, the storage service returns an HTTP 404 (Not found) message to the client. At the same time, in Azure Monitor metrics, you'll also see an **AuthorizationError** for the **ResponseType** dimension.

Investigate why your client application is attempting to perform an operation for which it hasn't been granted permissions.

Client-side JavaScript code doesn't have permission to access the object

If you're using a JavaScript client and the storage service is returning HTTP 404 messages, check for the following JavaScript errors in the browser:

SEC7120: Origin http://localhost:56309 not found in Access-Control-Allow-Origin header.
 SCRIPT7002: XMLHttpRequest: Network Error 0x80070005, Access is denied.

Note

You can use the F12 Developer Tools in Internet Explorer to trace the messages exchanged between the browser and the storage service when you're troubleshooting client-side JavaScript issues.

These errors occur because the web browser implements the [same origin policy](#) security restriction that prevents a web page from calling an API in a different domain from the domain the page comes from.

To work around the JavaScript issue, you can configure Cross-Origin Resource Sharing (CORS) for the storage service the client is accessing. For more information, see [Cross-Origin Resource Sharing \(CORS\) Support for Azure Storage Services](#).

The following code sample shows how to configure your blob service to allow JavaScript running in the Contoso domain to access a blob in your blob storage service:

.NET v12 SDK

C#

```
var connectionString = Constants.connectionString;

BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);

BlobServiceProperties sp = blobServiceClient.GetProperties();

// Set the service properties.
sp.DefaultServiceVersion = "2013-08-15";
BlobCorsRule bcr = new BlobCorsRule();
bcr.AllowedHeaders = "*";

bcr.AllowedMethods = "GET,POST";
bcr.AllowedOrigins = "http://www.contoso.com";
bcr.ExposedHeaders = "x-ms-*";
bcr.MaxAgeInSeconds = 5;
sp.Cors.Clear();
```

```
sp.Cors.Add(bcr);
blobServiceClient.SetProperties(sp);
```

Network failure

In some circumstances, lost network packets can lead to the storage service returning HTTP 404 messages to the client. For example, when your client application is deleting an entity from the table service, you see the client throw a storage exception reporting an "HTTP 404 (Not Found)" status message from the table service. When you investigate the table in the table storage service, you see that the service did delete the entity as requested.

The exception details in the client include the request ID (7e84f12d...) assigned by the table service for the request: you can use this information to locate the request details in the storage resource logs in Azure Monitor by searching in [Fields that describe how the operation was authenticated](#) of log entries. You could also use the metrics to identify when failures such as this occur and then search the log files based on the time the metrics recorded this error. This log entry shows that the delete failed with an "HTTP (404) Client Other Error" status message. The same log entry also includes the request ID generated by the client in the `client-request-id` column (813ea74f...).

The server-side log also includes another entry with the same `client-request-id` value (813ea74f...) for a successful delete operation for the same entity and from the same client. This successful delete operation took place shortly before the failed delete request.

The most likely cause of this scenario is that the client sent a delete request for the entity to the table service, which succeeded but didn't receive an acknowledgment from the server (perhaps due to a temporary network issue). The client then automatically retried the operation (using the same `client-request-id`), and this retry failed because the entity had already been deleted.

If this problem occurs frequently, you should investigate why the client is failing to receive acknowledgments from the table service. If the problem is intermittent, you should trap the "HTTP (404) Not Found" error and log it in the client, but allow the client to continue.

The client is receiving HTTP 409 (Conflict) messages

When a client deletes blob containers, tables, or queues, there's a brief period before the name becomes available again. If the code in your client application deletes and then immediately recreates a blob container using the same name, the `CreateIfNotExists` method eventually fails with the HTTP 409 (Conflict) error.

The client application should use unique container names whenever it creates new containers if the delete/recreate pattern is common.

Metrics show low PercentSuccess or analytics log entries have operations with transaction status of ClientOtherErrors

A `ResponseType` dimension equal to a value of `Success` captures the percent of operations that were successful based on their HTTP Status Code. Operations with status codes of 2XX count as successful, whereas operations with status codes in 3XX, 4XX, and 5XX ranges are counted as unsuccessful and lower the Success metric value. In storage resource logs, these operations are recorded with a transaction status of `ClientOtherError`.

These operations have completed successfully and therefore don't affect other metrics, such as availability. Some examples of operations that execute successfully but that can result in unsuccessful HTTP status codes include:

- **ResourceNotFound** (Not Found 404), for example, from a GET request to a blob that doesn't exist.
- **ResourceAlreadyExists** (Conflict 409), for example, from a `CreateIfNotExists` operation where the resource already exists.
- **ConditionNotMet** (Not Modified 304), for example, from a conditional operation such as when a client sends an `ETag` value and an HTTP `If-None-Match` header to request an image only if it has been updated since the last operation.

You can find a list of common REST API error codes that the storage services return on the page [Common REST API Error Codes](#).

See also

- [Monitoring Azure Blob Storage](#)
- [Monitoring Azure Files](#)
- [Monitoring Azure Queue Storage](#)
- [Monitoring Azure Table storage](#)
- [Troubleshoot performance issues](#)

- [Troubleshoot availability issues](#)
 - [Monitor, diagnose, and troubleshoot your Azure Storage](#)
-

Last updated on 04/09/2024

Troubleshoot performance in Azure storage accounts

This article helps you investigate unexpected changes in behavior (such as slower-than-usual response times). These changes in behavior can often be identified by monitoring storage metrics in Azure Monitor. For general information about using metrics and logs in Azure Monitor, see the following articles:

- [Monitoring Azure Blob Storage](#)
- [Monitoring Azure Files](#)
- [Monitoring Azure Queue Storage](#)
- [Monitoring Azure Table storage](#)

Monitoring performance

To monitor the performance of the storage services, you can use the following metrics:

- The **SuccessE2ELatency** and **SuccessServerLatency** metrics show the average time the storage service or API operation type is taking to process requests. **SuccessE2ELatency** is a measure of end-to-end latency that includes the time taken to read the request and send the response in addition to the time taken to process the request (therefore, it includes network latency once the request reaches the storage service). **SuccessServerLatency** is a measure of just the processing time and therefore excludes any network latency related to communicating with the client.
- The **Egress** and **Ingress** metrics show the total amount of data, in bytes, coming in to and going out of your storage service or through a specific API operation type.
- The **Transactions** metric shows the total number of requests that the storage service or API operation is receiving. **Transactions** is the total number of requests the storage service receives.

You can monitor for unexpected changes in any of these values. These changes could indicate an issue that requires further investigation.

In the [Azure portal](#) , you can add alert rules that notify you when any of the performance metrics for this service fall below or exceed a threshold that you specify.

Diagnose performance issues

The performance of an application can be subjective, especially from a user perspective. Therefore, it's important to have baseline metrics available to help you identify where there might be a performance issue. Many factors might affect the performance of an Azure storage service from the client application perspective. These factors might operate in the storage service, the client, or the network infrastructure. Therefore, it's important to have a strategy for identifying the origin of the performance issue.

After you've identified the likely location of the cause of the performance issue from the metrics, you can then use the log files to find detailed information to diagnose and troubleshoot the problem further.

Metrics show high `SuccessE2ELatency` and low `SuccessServerLatency`

In some cases, you might find that `SuccessE2ELatency` is higher than the `SuccessServerLatency`. The storage service only calculates the metric `SuccessE2ELatency` for successful requests and, unlike `SuccessServerLatency`, includes the time the client takes to send the data and receive acknowledgment from the storage service. Therefore, a difference between `SuccessE2ELatency` and `SuccessServerLatency` could be either due to the client application being slow to respond or due to conditions on the network.

ⓘ Note

You can also view `E2ELatency` and `ServerLatency` for individual storage operations in the Storage Logging log data.

Investigating client performance issues

Possible reasons for the client responding slowly include having limited available connections or threads or being low on resources such as CPU, memory, or network bandwidth. You may be able to resolve the issue by modifying the client code to be more efficient (for example, by using asynchronous calls to the storage service) or by using a larger Virtual Machine with more cores and more memory.

For the table and queue services, the Nagle algorithm can also cause high `SuccessE2ELatency` compared to `SuccessServerLatency`. For more information, see the post [Nagle's Algorithm is Not Friendly towards Small Requests](#). You can disable the Nagle algorithm in code by using the `ServicePointManager` class in the `System.Net` namespace. You should do this before you make any calls to the table or queue services in your application since this doesn't affect connections

that are already open. The following example comes from the `Application_Start` method in a worker role:

C#

```
var connectionString = Constants.connectionString;
QueueServiceClient queueServiceClient = new QueueServiceClient(connectionString);
ServicePoint queueServicePoint =
ServicePointManager.FindServicePoint(queueServiceClient.Uri);
queueServicePoint.UseNagleAlgorithm = false;
```

You should check the client-side logs to see how many requests your client application is submitting and check for general .NET related performance bottlenecks in your client, such as CPU, .NET garbage collection, network utilization, or memory. As a starting point for troubleshooting .NET client applications, see [Debugging, Tracing, and Profiling](#).

Investigating network latency issues

Typically, high end-to-end latency caused by the network is due to transient conditions. You can investigate both transient and persistent network issues, such as dropped packets, by using tools like Wireshark.

Metrics show low SuccessE2ELatency and low SuccessServerLatency but the client is experiencing high latency

In this scenario, the most likely cause is a delay in the storage request reaching the storage service. You should investigate why requests from the client aren't making it through to the blob service.

One possible reason for the client delaying sending requests is that there are a limited number of available connections or threads.

Also, check whether the client is performing multiple retries, and investigate the reason if it is. To determine whether the client is performing multiple retries, you can:

- Examine logs. If multiple retries are happening, you'll see multiple operations with the same client request IDs.
- Examine the client logs. Verbose logging will indicate that a retry has occurred.
- Debug your code, and check the properties of the `OperationContext` object associated with the request. If the operation has retried, the `RequestResults` property will include

multiple unique requests. You can also check the start and end times for each request.

If there are no issues in the client, you should investigate potential network issues such as packet loss. You can use tools such as Wireshark to investigate network issues.

Metrics show high `SuccessServerLatency`

In the case of high `SuccessServerLatency` for blob download requests, you should use the Storage logs to see if there are repeated requests for the same blob (or set of blobs). For blob upload requests, you should investigate what block size the client is using (for example, blocks less than 64 K in size can result in overheads unless the reads are also in less than 64 K chunks) and if multiple clients are uploading blocks to the same blob in parallel. You should also check the per-minute metrics for spikes in the number of requests that result in exceeding the per-second scalability targets.

If you see high `SuccessServerLatency` for blob download requests when there are repeated requests for the same blob or set of blobs, consider caching these blobs using Azure Cache or the Azure Content Delivery Network (CDN). For upload requests, you can improve the throughput by using a larger block size. For queries to tables, it's also possible to implement client-side caching on clients that perform the same query operations and where the data doesn't change frequently.

High `SuccessServerLatency` values can also be a symptom of poorly designed tables or queries that result in scan operations or that follow the append/prepend anti-pattern.

ⓘ Note

You can find a comprehensive performance checklist from [Microsoft Azure Storage Performance and Scalability Checklist](#).

You're experiencing unexpected delays in message delivery on a queue

If you're experiencing a delay between the time an application adds a message to a queue and the time it becomes available to read from the queue, then take the following steps to diagnose the issue:

- Verify that the application is successfully adding the messages to the queue. Check that the application is not retrying the `AddMessage` method several times before succeeding.

- Verify there is no clock skew between the worker role that adds the message to the queue and the worker role that reads the message from the queue. A clock skew makes it appear as if there is a delay in processing.
- Check if the worker role that reads the messages from the queue is failing. If a queue client calls the `GetMessage` method but fails to respond with an acknowledgment, the message will remain invisible on the queue until the `invisibilityTimeout` period expires. At this point, the message becomes available for processing again.
- Check if the queue length is growing over time. This can occur if you don't have sufficient workers available to process all of the messages that other workers are placing in the queue. Also, check the metrics to see if delete requests are failing and the dequeue count on messages, which might indicate repeated failed attempts to delete the message.
- Examine the Storage logs for any queue operations that have higher than expected **E2ELatency** and **ServerLatency** values over a longer period of time than usual.

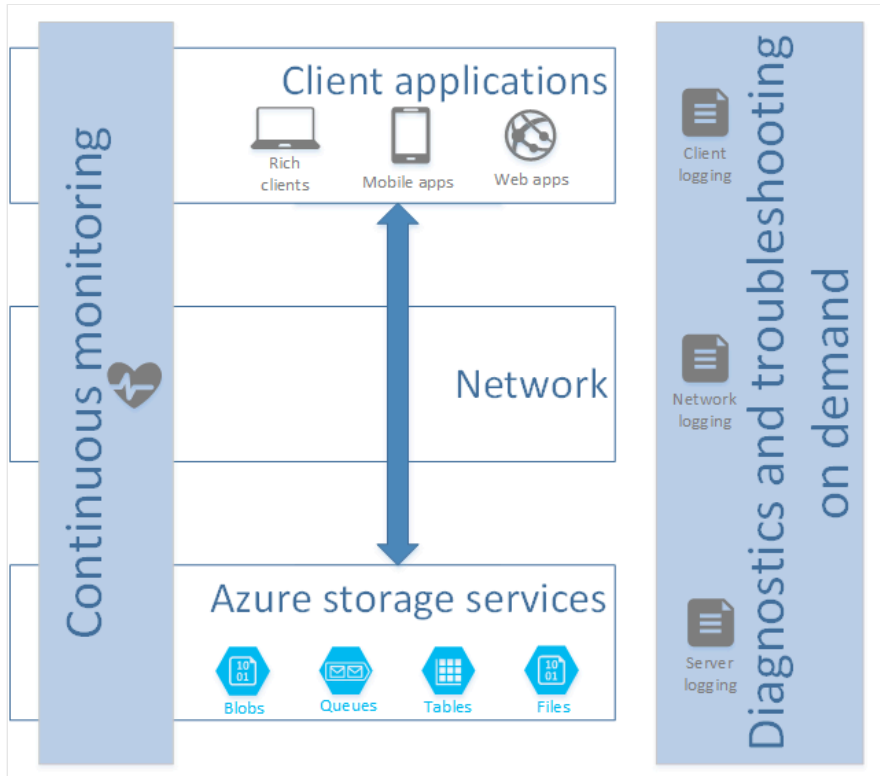
See also

- [Troubleshoot client application errors](#)
- [Troubleshoot availability issues](#)
- [Monitor, diagnose, and troubleshoot your Azure Storage](#)

Last updated on 04/09/2024

Monitor, diagnose, and troubleshoot Microsoft Azure Storage (classic)

This guide shows you how to use features such as Azure Storage Analytics, client-side logging in the Azure Storage Client Library, and other third-party tools to identify, diagnose, and troubleshoot Azure Storage-related issues.



This guide is intended to be read primarily by developers of online services that use Azure Storage Services and IT Pros responsible for managing such online services. The goals of this guide are:

- To help you maintain the health and performance of your Azure Storage accounts.
- To provide you with the necessary processes and tools to help you decide whether an issue or problem in an application relates to Azure Storage.
- To provide you with actionable guidance for resolving problems related to Azure Storage.

ⓘ Note

This article is based on using Storage Analytics metrics and logs referred to as *Classic metrics and logs*. We recommend that you use Azure Storage metrics and logs in Azure Monitor instead of Storage Analytics logs. To learn more, see any of the following articles:

- [Monitoring Azure Blob Storage](#)
- [Monitoring Azure Files](#)
- [Monitoring Azure Queue Storage](#)
- [Monitoring Azure Table storage](#)

Overview

Diagnosing and troubleshooting issues in a distributed application hosted in a cloud environment can be more complex than in traditional environments. Applications can be deployed in a PaaS or IaaS infrastructure, on-premises, on a mobile device, or in some combination of these environments. Typically, your application's network traffic may traverse public and private networks, and your application may use multiple storage technologies such as Microsoft Azure Storage Tables, Blobs, Queues, or Files in addition to other data stores such as relational and document databases.

To manage such applications successfully, you should monitor them proactively and understand how to diagnose and troubleshoot all aspects of them and their dependent technologies. As a user of Azure Storage services, you should continuously monitor the Storage services your application uses for any unexpected changes in behavior (such as slower-than-usual response times) and use logging to collect more detailed

data and to analyze a problem in depth. The diagnostics information you obtain from monitoring and logging will help you determine the root cause of the issue your application encountered. Then you can troubleshoot the issue and determine the appropriate steps to remediate it. Azure Storage is a core Azure service and forms an important part of the majority of solutions that customers deploy to the Azure infrastructure. Azure Storage includes capabilities to simplify monitoring, diagnosing, and troubleshooting storage issues in your cloud-based applications.

How this guide is organized

The [Monitoring your storage service](#) section describes how to monitor the health and performance of your Azure Storage services using Azure Storage Analytics Metrics (Storage Metrics).

The [Diagnosing storage issues](#) section describes how to diagnose issues using Azure Storage Analytics Logging (Storage Logging). It also describes how to enable client-side logging using the facilities in one of the client libraries, such as the Storage Client Library for .NET or the Azure SDK for Java.

The [End-to-end tracing](#) section describes how you can correlate the information contained in various log files and metrics data.

The [Troubleshooting guidance](#) section provides troubleshooting guidance for some of the common storage-related issues you might encounter.

The [Appendices](#) section includes information about using other tools, such as Wireshark and Netmon for analyzing network packet data, and Fiddler for analyzing HTTP/HTTPS messages.

Monitoring your storage service

If you are familiar with Windows performance monitoring, you can think of Storage Metrics as being an Azure Storage equivalent of Windows Performance Monitor counters. In Storage Metrics, you will find a comprehensive set of metrics (counters in Windows Performance Monitor terminology), such as service availability, the total number of requests to service, or the percentage of successful requests to service. For a full list of the available metrics, see [Storage Analytics Metrics Table Schema](#). You can specify whether you want the storage service to collect and aggregate metrics every hour or every minute. For more information about how to enable metrics and monitor your storage accounts, see [Enabling storage metrics and viewing metrics data](#).

You can choose which hourly metrics you want to display in the [Azure portal](#) and configure rules that notify administrators by email whenever an hourly metric exceeds a particular threshold. For more information, see [Receive Alert Notifications](#).

We recommend you review [Azure Monitor for Storage](#) (preview). It's a feature of Azure Monitor that offers comprehensive monitoring of your Azure Storage accounts by delivering a unified view of your Azure Storage services performance, capacity, and availability. It doesn't require you to enable or configure anything, and you can immediately view these metrics from the pre-defined interactive charts and other visualizations included.

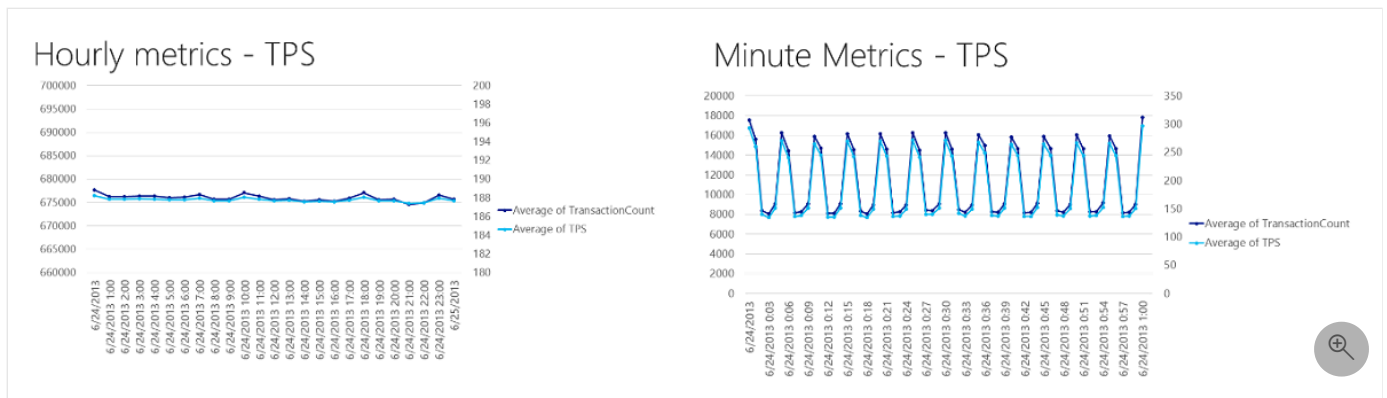
The storage service tries its best to collect metrics but may not record every storage operation.

In the Azure portal, you can view metrics such as availability, total requests, and average latency numbers for a storage account. A notification rule has also been set up to alert an administrator if availability drops below a certain level. From viewing this data, one possible area for investigation is the table service success percentage being below 100% (for more information, see the [Metrics show low PercentSuccess or analytics log entries have operations with transaction status of ClientOtherErrors](#) section).

You should continuously monitor your Azure applications to ensure they're healthy and performing as expected by:

- Establishing some baseline metrics for the application that will enable you to compare current data and identify any significant changes in the behavior of Azure storage and your application. The values of your baseline metrics will, in many cases, be application specific, and you should establish them when you're performance testing your application.
- Recording minute metrics and using them to monitor actively for unexpected errors and anomalies, such as spikes in error counts or request rates.
- Recording hourly metrics and using them to monitor average values such as average error counts and request rates.
- Investigating potential issues using diagnostics tools as discussed later in the [Diagnosing storage issues](#) section.

The charts in the following image illustrate how the averaging that occurs for hourly metrics can hide spikes in activity. The hourly metrics appear to show a steady rate of requests, while the minute metrics reveal the fluctuations that are really taking place.



The remainder of this section describes what metrics you should monitor and why.

Monitoring service health

You can use the [Azure portal](#) to view the health of the Storage service (and other Azure services) in all the Azure regions around the world. Monitoring enables you to see immediately if an issue outside your control is affecting the Storage service in the region you use for your application.

The [Azure portal](#) can also provide notifications of incidents that affect the various Azure services.

Note

This information was previously available, along with historical data, on the [Azure Service Dashboard](#). For more information about Application Insights for Azure DevOps, see [Appendix 5: Monitoring with Application Insights for Azure DevOps](#).

Monitoring capacity

Storage Metrics only stores capacity metrics for the blob service because blobs typically account for the largest proportion of stored data (at the time of writing, it's not possible to use Storage Metrics to monitor the capacity of your tables and queues). You can find this data in the `$MetricsCapacityBlob` table if you have enabled monitoring for the Blob service. Storage Metrics records this data once per day, and you can use the value of the `RowKey` to determine whether the row contains an entity that relates to user data (value `data`) or analytics data (value `analytics`). Each stored entity contains information about the amount of storage used (`Capacity` measured in bytes) and the current number of containers (`ContainerCount`) and blobs (`ObjectCount`) in use in the storage account. For more information about the capacity metrics stored in the `$MetricsCapacityBlob` table, see [Storage Analytics Metrics Table Schema](#).

Note

You should monitor these values for an early warning that you're approaching the capacity limits of your storage account. In the Azure portal, you can add alert rules to notify you if aggregate storage use exceeds or falls below the thresholds that you specify.

To estimate the size of various storage objects such as blobs, see the blog post [Understanding Azure Storage Billing – Bandwidth, Transactions, and Capacity](#).

Monitoring availability

You should monitor the availability of the storage services in your storage account by monitoring the value in the `Availability` column in the hourly or minute metrics tables — `$MetricsHourPrimaryTransactionsBlob`, `$MetricsHourPrimaryTransactionsTable`, `$MetricsHourPrimaryTransactionsQueue`, `$MetricsMinutePrimaryTransactionsBlob`, `$MetricsMinutePrimaryTransactionsTable`, `$MetricsMinutePrimaryTransactionsQueue`, `$MetricsCapacityBlob`. The `Availability` column contains a percentage value that indicates the availability of the service or the API operation represented by the row (the `RowKey` shows if the row contains metrics for the service as a whole or for a specific API operation).

Any value less than 100% indicates that some storage requests are failing. You can see why they're failing by examining the other columns in the metrics data that show the numbers of requests with different error types, such as `ServerTimeoutError`. You should expect to see `Availability` fall temporarily below 100% for reasons such as transient server timeouts while the service moves partitions to better load-

balance requests; the retry logic in your client application should handle such intermittent conditions. The article [Storage Analytics Logged Operations and Status Messages](#) lists the transaction types that Storage Metrics includes in its `Availability` calculation.

In the [Azure portal](#), you can add alert rules to notify you if `Availability` for a service falls below a threshold that you specify.

The [Troubleshooting guidance](#) section of this guide describes some common storage service issues related to availability.

Monitoring performance

To monitor the performance of the storage services, you can use the following metrics from the hourly and minute metrics tables.

- The values in the `AverageE2ELatency` and `AverageServerLatency` columns show the average time the storage service or API operation type is taking to process requests. `AverageE2ELatency` is a measure of end-to-end latency that includes the time taken to read the request and send the response in addition to the time taken to process the request (therefore includes network latency once the request reaches the storage service); `AverageServerLatency` is a measure of just the processing time and therefore excludes any network latency related to communicating with the client. See the [Metrics show high AverageE2ELatency and low AverageServerLatency](#) section later in this guide for a discussion of why there might be a significant difference between these two values.
- The values in the `TotalIngress` and `TotalEgress` columns show the total amount of data, in bytes, coming in to and going out of your storage service or through a specific API operation type.
- The values in the `TotalRequests` column show the total number of requests the storage service or API operation is receiving. `TotalRequests` is the total number of requests the storage service receives.

Typically, you will monitor for unexpected changes in any of these values, as this indicates you have an issue that requires investigation.

In the [Azure portal](#), you can add alert rules to notify you if any performance metrics for this service fall below or exceed a threshold that you specify.

The [Troubleshooting guidance](#) section of this guide describes some common storage service issues related to performance.

Diagnosing storage issues

There are a number of ways that you might become aware of a problem or issue in your application, including:

- A major failure that causes the application to crash or stop working.
- Significant changes from baseline values in the metrics you're monitoring as described in the previous section [Monitoring your storage service](#).
- Reports from users of your application that some particular operation didn't complete as expected or that some feature is not working.
- Errors generated within your application that appear in log files or through some other notification method.

Typically, issues related to Azure storage services fall into one of four broad categories:

- Your application has a performance issue, either reported by your users or revealed by changes in the performance metrics.
- There is a problem with the Azure Storage infrastructure in one or more regions.
- Your application is encountering an error, either reported by your users or revealed by an increase in one of the error count metrics you monitor.
- During development and testing, you may be using the local storage emulator; you may encounter some issues that relate specifically to the usage of the storage emulator.

The following sections outline the steps you should follow to diagnose and troubleshoot issues in each of these four categories. The [Troubleshooting guidance](#) section later in this guide provides more detail for some common issues you may encounter.

Service health issues

Service health issues are typically outside of your control. The [Azure portal](#) provides information about any ongoing issues with Azure services, including storage services. If you opted for Read-Access Geo-Redundant Storage when you created your storage account, then if your data becomes unavailable in the primary location, your application can switch temporarily to the read-only copy in the secondary location. To read from the secondary, your application must be able to switch between using the primary and secondary storage locations and be able to work in a reduced functionality mode with read-only data. The Azure Storage Client libraries allow you to define a retry policy that can read from secondary storage in case a read from primary storage fails. Your application also needs to be aware that the data in the secondary location is eventually consistent. For more information, see the blog post [Azure Storage Redundancy Options and Read Access Geo Redundant Storage](#).

Performance issues

The performance of an application can be subjective, especially from a user perspective. Therefore, it is important to have baseline metrics available to help you identify where there might be a performance issue. Many factors might affect the performance of an Azure storage service from the client application perspective. These factors might operate in the storage service, the client, or the network infrastructure; therefore, it's important to have a strategy for identifying the origin of the performance issue.

After you have identified the likely location of the cause of the performance issue from the metrics, you can then use the log files to find detailed information to diagnose and troubleshoot the problem further.

The [Troubleshooting guidance](#) section later in this guide provides more information about some common performance-related issues you may encounter.

Diagnosing errors

Users of your application may notify you of errors reported by the client application. Storage Metrics also records counts of different error types from your storage services, such as `NetworkError`, `ClientTimeoutError`, or `AuthorizationError`. While Storage Metrics only records counts of different error types, you can obtain more detail about individual requests by examining server-side, client-side, and network logs. Typically, the HTTP status code returned by the storage service will give an indication of why the request failed.

Note

Remember that you should expect to see some intermittent errors: for example, errors due to transient network conditions or application errors.

The following resources are useful for understanding storage-related status and error codes:

- [Common REST API error codes](#)
- [Blob Service error codes](#)
- [Queue Service error codes](#)
- [Table Service error codes](#)
- [File Service error codes](#)

Storage emulator issues

The Azure SDK includes a storage emulator you can run on a development workstation. This emulator simulates most of the behavior of the Azure storage services and is useful during development and testing, enabling you to run applications that use Azure storage services without the need for an Azure subscription and an Azure storage account.

The [Troubleshooting guidance](#) section of this guide describes some common issues encountered using the storage emulator.

Storage logging tools

Storage Logging provides server-side logging of storage requests in your Azure storage account. For more information about how to enable server-side logging and access the log data, see [Enabling Storage Logging and Accessing Log Data](#).

The Storage Client Library for .NET enables you to collect client-side log data that relates to storage operations performed by your application. For more information, see [Client-side Logging with the .NET Storage Client Library](#).

Note

In some circumstances (such as SAS authorization failures), a user may report an error for which you can find no request data in the server-side Storage logs. You can use the logging capabilities of the Storage Client Library to investigate if the cause of the issue is on the client or use network monitoring tools to investigate the network.

Using network logging tools

You can capture the traffic between the client and server to provide detailed information about the data the client and server are exchanging and the underlying network conditions. Useful network logging tools include:

- [Fiddler](#) is a free web debugging proxy that enables you to examine the headers and payload data of HTTP and HTTPS request and response messages. For more information, see [Appendix 1: Using Fiddler to capture HTTP and HTTPS traffic](#).
- [Microsoft Network Monitor \(Netmon\)](#) and [Wireshark](#) are free network protocol analyzers that enable you to view detailed packet information for a wide range of network protocols. For more information about Wireshark, see [Appendix 2: Using Wireshark to capture network traffic](#).
- If you want to perform a basic connectivity test to check that your client machine can connect to the Azure storage service over the network, you cannot do this using the standard ping tool on the client. However, you can use the [tcping tool](#) to check connectivity.

In many cases, the log data from Storage Logging and the Storage Client Library will be sufficient to diagnose an issue, but in some scenarios, you may need the more detailed information that these network logging tools can provide. For example, using Fiddler to view HTTP and HTTPS messages enables you to view header and payload data sent to and from the storage services, which would enable you to examine how a client application retries storage operations. Protocol analyzers such as Wireshark operate at the packet level enabling you to view TCP data, which would enable you to troubleshoot lost packets and connectivity issues.

End-to-end tracing

End-to-end tracing using a variety of log files is a useful technique for investigating potential issues. You can use the date/time information from your metrics data to indicate where to start looking in the log files for detailed information to help you troubleshoot the issue.

Correlating log data

When viewing logs from client applications, network traces, and server-side storage logging, it is critical to be able to correlate requests across the different log files. The log files include a number of different fields that are useful as correlation identifiers. The client request ID is the most useful field to use to correlate entries in the different logs. However, sometimes, it can be useful to use either the server request ID or timestamps. The following sections provide more details about these options.

Client request ID

The Storage Client Library automatically generates a unique client request ID for every request.

- In the client-side log that the Storage Client Library creates, the client request ID appears in the `Client Request ID` field of every log entry relating to the request.
- In a network trace such as one captured by Fiddler, the client request ID is visible in request messages as the `x-ms-client-request-id` HTTP header value.
- In the server-side Storage Logging log, the client request ID appears in the Client request ID column.

Note

It's possible for multiple requests to share the same client request ID because the client can assign this value (although the Storage Client Library assigns a new value automatically). When the client retries, all attempts share the same client request ID. In the case of a batch sent from the client, the batch has a single client request ID.

Server request ID

The storage service automatically generates server request IDs.

- In the server-side Storage Logging log, the server request ID appears in the `Request ID header` column.
- In a network trace such as one captured by Fiddler, the server request ID appears in response messages as the `x-ms-request-id` HTTP header value.
- In the client-side log that the Storage Client Library creates, the server request ID appears in the `Operation Text` column for the log entry showing details of the server response.

Note

The storage service always assigns a unique server request ID to every request it receives, so every retry attempt from the client and every operation included in a batch has a unique server request ID.

The code sample below demonstrates how to use a custom client request ID.

C#

```
var connectionString = Constants.connectionString;

BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);

BlobContainerClient blobContainerClient = blobServiceClient.GetBlobContainerClient("demcontainer");

BlobClient blobClient = blobContainerClient.GetBlobClient("testImage.jpg");

string clientRequestId = String.Format("{0} {1} {2} {3}", HOSTNAME, APPNAME, USERID, Guid.NewGuid().ToString());

using (HttpPipeline.CreateClientRequestIdScope(clientRequestId))
{
    BlobDownloadInfo download = blobClient.Download();

    using (FileStream downloadFileStream = File.OpenWrite("C:\\testImage.jpg"))
    {
        download.Content.CopyTo(downloadFileStream);
        downloadFileStream.Close();
    }
}
```

Timestamps

You can also use timestamps to locate related log entries, but be careful of any clock skew between the client and server that may exist. Search plus or minus 15 minutes for matching server-side entries based on the timestamp on the client. Remember that the blob metadata for the blobs containing metrics indicates the time range for the metrics stored in the blob. This time range is useful if you have many metrics blobs for the same minute or hour.

Troubleshooting guidance

This section will help you with the diagnosis and troubleshooting of some of the common issues your application may encounter when using the Azure storage services. Use the list below to locate the information relevant to your specific issue.

Troubleshooting decision tree

Does your issue relate to the performance of one of the storage services?

- [Metrics show high AverageE2ELatency and low AverageServerLatency.](#)
- [Metrics show low AverageE2ELatency and low AverageServerLatency but the client is experiencing high latency.](#)
- [Metrics show high AverageServerLatency.](#)
- [You are experiencing unexpected delays in message delivery on a queue.](#)

Does your issue relate to the availability of one of the storage services?

- [Metrics show an increase in PercentThrottlingError.](#)
- [Metrics show an increase in PercentTimeoutError.](#)
- [Metrics show an increase in PercentNetworkError.](#)

Is your client application receiving an HTTP 4XX (such as 404) response from a storage service?

- [The client is receiving HTTP 403 \(Forbidden\) messages.](#)
- [The client is receiving HTTP 404 \(Not found\) messages.](#)
- [The client is receiving HTTP 409 \(Conflict\) messages.](#)

[Metrics show low PercentSuccess or analytics log entries have operations with transaction status of ClientOtherErrors.](#)

Capacity metrics show an unexpected increase in storage capacity usage.

Your issue arises from using the storage emulator for development or testing.

You are encountering problems installing the Azure SDK for .NET.

You have a different issue with a storage service.

Metrics show high AverageE2ELatency and low AverageServerLatency

The illustration below from the [Azure portal](#) monitoring tool shows an example where the **AverageE2ELatency** is significantly higher than the **AverageServerLatency**.

AVERAGESERVERLATENCY		AVERAGEE2ELATENCY		
1.27		30.16		
METRIC	AVERAGE	MINIMUM	MAXIMUM	TOTAL
AverageE2ELatency	49.47 ms	0 ms	139.65 ms	--
AverageServerLatency	2.49 ms	0 ms	4.59 ms	--

The storage service only calculates the metric **AverageE2ELatency** for successful requests and, unlike **AverageServerLatency**, includes the time the client takes to send the data and receive acknowledgment from the storage service. Therefore, a difference between **AverageE2ELatency** and **AverageServerLatency** could be either due to the client application being slow to respond or due to conditions on the network.

Note

You can also view **E2ELatency** and **ServerLatency** for individual storage operations in the Storage Logging log data.

Investigating client performance issues

Possible reasons for the client responding slowly include having a limited number of available connections or threads or being low on resources such as CPU, memory, or network bandwidth. You may be able to resolve the issue by modifying the client code to be more efficient (for example, by using asynchronous calls to the storage service) or by using a larger Virtual Machine (with more cores and more memory).

For the table and queue services, the Nagle algorithm can also cause high **AverageE2ELatency** compared to **AverageServerLatency**. For more information, see [Nagle's Algorithm is Not Friendly towards Small Requests](#). You can disable the Nagle algorithm in code by using the `ServicePointManager` class in the `System.Net` namespace. You should do this before you make any calls to the table or queue services in your application since this doesn't affect connections that are already open. The following example comes from the `Application_Start` method in a worker role.

.NET v12 SDK

C#

```
var connectionString = Constants.connectionString;

QueueServiceClient queueServiceClient = new QueueServiceClient(connectionString);

ServicePoint queueServicePoint = ServicePointManager.FindServicePoint(queueServiceClient.Uri);
queueServicePoint.UseNagleAlgorithm = false;
```

You should check the client-side logs to see how many requests your client application is submitting and check for general .NET-related performance bottlenecks in your client, such as CPU, .NET garbage collection, network utilization, or memory. As a starting point for troubleshooting .NET client applications, see [Debugging, Tracing, and Profiling](#).

Investigating network latency issues

Typically, high end-to-end latency caused by the network is due to transient conditions. You can investigate both transient and persistent network issues, such as dropped packets, by using tools like Wireshark.

For more information about using Wireshark to troubleshoot network issues, see [Appendix 2: Using Wireshark to capture network traffic](#).

Metrics show low `AverageE2ELatency` and low `AverageServerLatency` but the client is experiencing high latency

In this scenario, the most likely cause is a delay in the storage requests reaching the storage service. You should investigate why requests from the client are not making it through to the blob service.

One possible reason for the client delaying sending requests is that there are a limited number of available connections or threads.

Also, check whether the client is performing multiple retries, and investigate the reason if it is. To determine whether the client is performing multiple retries, you can:

- Examine the Storage Analytics logs. If multiple retries are happening, you will see multiple operations with the same client request ID but with different server request IDs.
- Examine the client logs. Verbose logging will indicate that a retry has occurred.
- Debug your code, and check the properties of the `OperationContext` object associated with the request. If the operation has retried, the `RequestResults` property will include multiple unique server request IDs. You can also check the start and end times for each request. For more information, see the code sample in the section [Server request ID](#).

If there are no issues in the client, you should investigate potential network issues such as packet loss. You can use tools such as Wireshark to investigate network issues.

For more information about using Wireshark to troubleshoot network issues, see [Appendix 2: Using Wireshark to capture network traffic](#).

Metrics show high `AverageServerLatency`

In the case of high `AverageServerLatency` for blob download requests, you should use the Storage Logging logs to see if there are repeated requests for the same blob (or set of blobs). For blob upload requests, you should investigate what block size the client is using (for example, blocks less than 64 K in size can result in overheads unless the reads are also in less than 64 K chunks) and if multiple clients are uploading blocks to the same blob in parallel. You should also check the per-minute metrics for spikes in the number of requests that result in exceeding the per-second scalability targets. For more information, see [Metrics show an increase in `PercentTimeoutError`](#).

If you see high `AverageServerLatency` for blob download requests when there are repeated requests for the same blob or set of blobs, then consider caching these blobs using Azure Cache or the Azure Content Delivery Network (CDN). For upload requests, you can improve the throughput by using a larger block size. For queries to tables, it's also possible to implement client-side caching on clients that perform the same query operations and where the data doesn't change frequently.

High `AverageServerLatency` values can also be a symptom of poorly designed tables or queries that result in scan operations or that follow the append/prepend anti-pattern. For more information, see [Metrics show an increase in `PercentThrottlingError`](#).

Note

You can find a comprehensive performance checklist here: [Microsoft Azure Storage Performance and Scalability Checklist](#).

You are experiencing unexpected delays in message delivery on a queue

If you're experiencing a delay between the time an application adds a message to a queue and the time it becomes available to read from the queue, then take the following steps to diagnose the issue:

- Verify that the application is successfully adding the messages to the queue. Check that the application isn't retrying the `AddMessage` method several times before succeeding. The Storage Client Library logs will show any repeated retries of storage operations.
- Verify there is no clock skew between the worker role that adds the message to the queue and the worker role that reads the message from the queue. A clock skew makes it appear as if there's a delay in processing.
- Check if the worker role that reads the messages from the queue is failing. If a queue client calls the `GetMessage` method but fails to respond with an acknowledgment, the message will remain invisible on the queue until the `invisibilityTimeout` period expires. At this

point, the message becomes available for processing again.

- Check if the queue length is growing over time. This can occur if you don't have sufficient workers available to process all of the messages that other workers are placing in the queue. Also, check the metrics to see if delete requests are failing and the dequeue count on messages, which might indicate repeated failed attempts to delete the message.
- Examine the Storage Logging logs for any queue operations that have higher than expected **E2ELatency** and **ServerLatency** values over a longer period of time than usual.

Metrics show an increase in PercentThrottlingError

Throttling errors occur when you exceed the scalability targets of a storage service. The storage service throttles to ensure that no single client or tenant can use the service at the expense of others. For more information, see [Scalability and performance targets for standard storage accounts](#) for details on scalability targets for storage accounts and performance targets for partitions within storage accounts.

If the **PercentThrottlingError** metric shows an increase in the percentage of requests that are failing with a throttling error, you need to investigate one of two scenarios:

- [Transient increase in PercentThrottlingError](#)
- [Permanent increase in PercentThrottlingError error](#)

An increase in **PercentThrottlingError** often occurs at the same time as an increase in the number of storage requests or when you are initially load-testing your application. This may also manifest itself in the client as "503 Server Busy" or "500 Operation Timeout" HTTP status messages from storage operations.

Transient increase in PercentThrottlingError

If you're seeing spikes in the value of **PercentThrottlingError** that coincide with periods of high activity for the application, you can implement an exponential (not linear) back-off strategy for retries in your client. Back-off retries reduce the immediate load on the partition and help your application smooth out spikes in traffic. For more information about how to implement retry policies using the Storage Client Library, see the [Microsoft.Azure.Storage.RetryPolicies namespace](#).

① Note

You may also see spikes in the value of **PercentThrottlingError** that do not coincide with periods of high activity for the application. The most likely cause is the storage service moving partitions to improve load balancing.

Permanent increase in PercentThrottlingError

If you're seeing a consistently high value for **PercentThrottlingError** following a permanent increase in your transaction volumes or when you're performing your initial load tests on your application, then you need to evaluate how your application is using storage partitions and whether it's approaching the scalability targets for a storage account. For example, if you're seeing throttling errors on a queue (which counts as a single partition), then consider using additional queues to spread the transactions across multiple partitions. If you're seeing throttling errors on a table, you need to consider using a different partitioning scheme to spread your transactions across multiple partitions by using a wider range of partition key values. One common cause of this issue is the prepend/append anti-pattern where you select the date as the partition key, and then all data on a particular day is written to one partition: under load, this can result in a write bottleneck. Either consider a different partitioning design or evaluate whether using blob storage might be a better solution. Also, check whether throttling is occurring as a result of spikes in your traffic and investigate ways of smoothing your pattern of requests.

If you distribute your transactions across multiple partitions, you must still be aware of the scalability limits set for the storage account. For example, if you used ten queues, each processing the maximum of 2,000 1KB messages per second, you will be at the overall limit of 20,000 messages per second for the storage account. If you need to process more than 20,000 entities per second, consider using multiple storage accounts. You should also bear in mind that the size of your requests and entities has an impact on when the storage service throttles your clients. If you have larger requests and entities, you may be throttled sooner.

Inefficient query design can also cause you to hit the scalability limits for table partitions. For example, a query with a filter that only selects one percent of the entities in a partition but that scans all the entities in a partition will need to access each entity. Every entity read will count towards the total number of transactions in that partition; therefore, you can easily reach the scalability targets.

① Note

Your performance testing should reveal any inefficient query designs in your application.

Metrics show an increase in PercentTimeoutError

Your metrics show an increase in **PercentTimeoutError** for one of your storage services. At the same time, the client receives a high volume of "500 Operation Timeout" HTTP status messages from storage operations.

Note

You may see timeout errors temporarily as the storage service load balances requests by moving a partition to a new server.

The **PercentTimeoutError** metric is an aggregation of the following metrics: **ClientTimeoutError**, **AnonymousClientTimeoutError**, **SASClientTimeoutError**, **ServerTimeoutError**, **AnonymousServerTimeoutError**, and **SASServerTimeoutError**.

The server timeouts are caused by an error on the server. The client timeouts happen because an operation on the server has exceeded the timeout specified by the client; for example, a client using the Storage Client Library can set a timeout for an operation by using the `ServerTimeout` property of the `QueueRequestOptions` class.

Server timeouts indicate a problem with the storage service that requires further investigation. You can use metrics to see if you're hitting the scalability limits for the service and to identify any spikes in traffic that might be causing this problem. If the problem is intermittent, it may be due to load-balancing activity in the service. If the problem is persistent and isn't caused by your application hitting the scalability limits of the service, you should raise a support issue. For client timeouts, you must decide if the timeout is set to an appropriate value in the client and either change the timeout value set in the client or investigate how you can improve the performance of the operations in the storage service, for example, by optimizing your table queries or reducing the size of your messages.

Metrics show an increase in PercentNetworkError

Your metrics show an increase in **PercentNetworkError** for one of your storage services. The **PercentNetworkError** metric is an aggregation of the following metrics: **NetworkError**, **AnonymousNetworkError**, and **SASNetworkError**. These occur when the storage service detects a network error when the client makes a storage request.

The most common cause of this error is a client disconnecting before a timeout expires in the storage service. Investigate the code in your client to understand why and when the client disconnects from the storage service. You can also use Wireshark or Tcping to investigate network connectivity issues from the client. These tools are described in the [Appendices](#).

The client is receiving HTTP 403 (Forbidden) messages

If your client application is throwing HTTP 403 (Forbidden) errors, a likely cause is that the client is using an expired Shared Access Signature (SAS) when it sends a storage request (although other possible causes include clock skew, invalid keys, and empty headers). If an expired SAS key is the cause, you won't see any entries in the server-side Storage Logging log data. The following table shows a sample from the client-side log generated by the Storage Client Library that illustrates this issue occurring:

 Expand table

Source	Verbosity	Verbosity	Client request ID	Operation text
Microsoft.Azure.Storage	Information	3	85d077ab-...	Starting operation with location Primary per location mode PrimaryOnly.
Microsoft.Azure.Storage	Information	3	85d077ab-...	Starting synchronous request to https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/Synchronous_and_Asynchronous_Requests#Synchronous_request.
Microsoft.Azure.Storage	Information	3	85d077ab-...	Waiting for response.
Microsoft.Azure.Storage	Warning	2	85d077ab-...	Exception thrown while waiting for response: The remote server returned an error: (403) Forbidden.
Microsoft.Azure.Storage	Information	3	85d077ab-...	Response received. Status code = 403, Request ID = <Request ID>, Content-MD5 = , ETag = .
Microsoft.Azure.Storage	Warning	2	85d077ab-...	Exception thrown during the operation: The remote server returned an error: (403) Forbidden..

Source	Verbosity	Verbosity	Client request ID	Operation text
Microsoft.Azure.Storage	Information	3	85d077ab ~...	Checking if the operation should be retried. Retry count = 0, HTTP status code = 403, Exception = The remote server returned an error: (403) Forbidden..
Microsoft.Azure.Storage	Information	3	85d077ab ~...	The next location has been set to Primary, based on the location mode.
Microsoft.Azure.Storage	Error	1	85d077ab ~...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (403) Forbidden.

In this scenario, you should investigate why the SAS token is expiring before the client sends the token to the server:

- Typically, you shouldn't set a start time when you create a SAS for a client to use immediately. If there are small clock differences between the host generating the SAS using the current time and the storage service, then it's possible for the storage service to receive a SAS that's not yet valid.
- Don't set a very short expiry time on a SAS. Again, small clock differences between the host generating the SAS and the storage service can lead to a SAS seemingly expiring earlier than anticipated.
- Does the version parameter in the SAS key (for example, `sv=2015-04-05`) match the version of the Storage Client Library you are using? We recommend that you always use the latest version of the [Storage Client Library](#).
- If you regenerate your storage access keys, any existing SAS tokens may be invalidated. This issue may arise if you generate SAS tokens with a long expiry time for client applications to cache.

If you're using the Storage Client Library to generate SAS tokens, then it's easy to build a valid token. However, if you're using the Storage REST API and constructing the SAS tokens by hand, see [Delegating Access with a Shared Access Signature](#).

The client is receiving HTTP 404 (Not found) messages

If the client application receives an HTTP 404 (Not found) message from the server, this implies that the object the client was attempting to use (such as an entity, table, blob, container, or queue) doesn't exist in the storage service. There are a number of possible reasons for this, such as:

- [The client or another process previously deleted the object.](#)
- [A Shared Access Signature \(SAS\) authorization issue.](#)
- [Client-side JavaScript code doesn't have permission to access the object.](#)
- [Network failure.](#)

The client or another process previously deleted the object

In scenarios where the client is attempting to read, update, or delete data in a storage service, it's usually easy to identify in the server-side logs a previous operation that deleted the object in question from the storage service. Often, the log data shows that another user or process deleted the object. In the server-side Storage Logging log, the operation-type and requested-object-key columns show when a client deleted an object.

In the scenario where a client is attempting to insert an object, it may not be immediately obvious why this results in an HTTP 404 (Not found) response, given that the client is creating a new object. However, if the client is creating a blob, it must be able to find the blob container. If the client is creating a message, it must be able to find a queue. And if the client is adding a row, it must be able to find the table.

You can use the client-side log from the Storage Client Library to gain a more detailed understanding of when the client sends specific requests to the storage service.

The following client-side log generated by the Storage Client library illustrates the problem when the client cannot find the container for the blob it's creating. This log includes details of the following storage operations:

 Expand table

Request ID	Operation
07b26a5d-...	<code>DeleteIfExists</code> method to delete the blob container. Note that this operation includes a <code>HEAD</code> request to check for the existence of the container.
e2d06d78...	<code>CreateIfNotExists</code> method to create the blob container. Note that this operation includes a <code>HEAD</code> request that checks for the existence of the container. The <code>HEAD</code> returns a 404 message but continues.

Request ID	Operation
de8b1c3c-...	<code>UploadFromStream</code> method to create the blob. The <code>PUT</code> request fails with a 404 message.

Log entries:

 Expand table

Request ID	Operation text
07b26a5d-...	Starting synchronous request to <code>https://domemalldist.blob.core.windows.net/azurembblobcontainer</code> .
07b26a5d-...	<code>StringToSign = HEAD.....x-ms-client-request-id:07b26a5d-....x-ms-date:Tue, 03 Jun 2014 10:33:11 GMT.x-ms-version:2014-02-14./domemalldist/azurembblobcontainer.restype:container.</code>
07b26a5d-...	Waiting for response.
07b26a5d-...	Response received. Status code = 200, Request ID = eeead849-..., Content-MD5 = , ETag = <code>"0x8D14D2DC63D059B"</code> ;
07b26a5d-...	Response headers were processed successfully, proceeding with the rest of the operation.
07b26a5d-...	Downloading response body.
07b26a5d-...	Operation completed successfully.
07b26a5d-...	Starting synchronous request to <code>https://domemalldist.blob.core.windows.net/azurembblobcontainer</code> .
07b26a5d-...	<code>StringToSign = DELETE.....x-ms-client-request-id:07b26a5d-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azurembblobcontainer.restype:container.</code>
07b26a5d-...	Waiting for response.
07b26a5d-...	Response received. Status code = 202, Request ID = 6ab2a4cf-..., Content-MD5 = , ETag = .
07b26a5d-...	Response headers were processed successfully, proceeding with the rest of the operation.
07b26a5d-...	Downloading response body.
07b26a5d-...	Operation completed successfully.
e2d06d78-...	Starting asynchronous request to <code>https://domemalldist.blob.core.windows.net/azurembblobcontainer</code> .
e2d06d78-...	<code>StringToSign = HEAD.....x-ms-client-request-id:e2d06d78-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azurembblobcontainer.restype:container.</code>
e2d06d78-...	Waiting for response.
de8b1c3c-...	Starting synchronous request to <code>https://domemalldist.blob.core.windows.net/azurembblobcontainer/blobCreated.txt</code> .
de8b1c3c-...	<code>StringToSign = PUT...64.qCmF+TQLPhq/YK50mP9ZQ==.....x-ms-blob-type:BlockBlob.x-ms-client-request-id:de8b1c3c-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azurembblobcontainer/blobCreated.txt.</code>
de8b1c3c-...	Preparing to write request data.
e2d06d78-...	Exception thrown while waiting for response: The remote server returned an error: (404) Not Found..
e2d06d78-...	Response received. Status code = 404, Request ID = 353ae3bc-..., Content-MD5 = , ETag = .
e2d06d78-...	Response headers were processed successfully, proceeding with the rest of the operation.
e2d06d78-...	Downloading response body.
e2d06d78-...	Operation completed successfully.
e2d06d78-...	Starting asynchronous request to <code>https://domemalldist.blob.core.windows.net/azurembblobcontainer</code> .
e2d06d78-...	<code>StringToSign = PUT...0.....x-ms-client-request-id:e2d06d78-....x-ms-date:Tue, 03 Jun 2014 10:33:12 GMT.x-ms-version:2014-02-14./domemalldist/azurembblobcontainer.restype:container.</code>
e2d06d78-...	Waiting for response.
de8b1c3c-...	Writing request data.
de8b1c3c-...	Waiting for response.

Request ID	Operation text
e2d06d78-...	Exception thrown while waiting for response: The remote server returned an error: (409) Conflict..
e2d06d78-...	Response received. Status code = 409, Request ID = c27da20e-..., Content-MD5 = , ETag = .
e2d06d78-...	Downloading error response body.
de8b1c3c-...	Exception thrown while waiting for response: The remote server returned an error: (404) Not Found..
de8b1c3c-...	Response received. Status code = 404, Request ID = 0eaeab3e-..., Content-MD5 = , ETag = .
de8b1c3c-...	Exception thrown during the operation: The remote server returned an error: (404) Not Found..
de8b1c3c-...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (404) Not Found..
e2d06d78-...	Retry policy did not allow for a retry. Failing with The remote server returned an error: (409) Conflict..

In this example, the log shows that the client is interleaving requests from the `CreateIfNotExists` method (request ID e2d06d78...) with the requests from the `UploadFromStream` method (de8b1c3c-...). This interleaving happens because the client application is invoking these methods asynchronously. Modify the asynchronous code in the client to ensure that it creates the container before attempting to upload any data to a blob in that container. Ideally, you should create all your containers in advance.

A Shared Access Signature (SAS) authorization issue

If the client application attempts to use a SAS key that doesn't include the necessary permissions for the operation, the storage service returns an HTTP 404 (Not found) message to the client. At the same time, you will also see a non-zero value for `SASAuthorizationError` in the metrics.

The following table shows a sample server-side log message from the Storage Logging log file:

 Expand table

Name	Value
Request start time	2014-05-30T06:17:48.4473697Z
Operation type	GetBlobProperties
Request status	SASAuthorizationError
HTTP status code	404
Authentication type	Sas
Service type	Blob
Request URL	https://domemaildist.blob.core.windows.net/azureimblobcontainer/blobCreatedViaSAS.txt
	?sv=2014-02-14&sr=c&si=mypolicy&sig=XXXXX&api-version=2014-02-14
Request ID header	<Request ID header>
Client request ID	<Client request ID>

Investigate why your client application is attempting to perform an operation for which it hasn't been granted permissions.

Client-side JavaScript code doesn't have permission to access the object

If you're using a JavaScript client and the storage service is returning HTTP 404 messages, you check for the following JavaScript errors in the browser:

- SEC7120: Origin http://localhost:56309 not found in Access-Control-Allow-Origin header.
- SCRIPT7002: XMLHttpRequest: Network Error 0x80070005, Access is denied.

 **Note**

You can use the F12 Developer Tools in Internet Explorer to trace the messages exchanged between the browser and the storage service when you're troubleshooting client-side JavaScript issues.

These errors occur because the web browser implements the [same origin policy](#) security restriction that prevents a web page from calling an API in a different domain from the domain the page comes from.

To work around the JavaScript issue, you can configure Cross-Origin Resource Sharing (CORS) for the storage service the client is accessing. For more information, see [Cross-Origin Resource Sharing \(CORS\) Support for Azure Storage Services](#).

The following code sample shows how to configure your blob service to allow JavaScript running in the Contoso domain to access a blob in your blob storage service:

.NET v12 SDK

C#

```
var connectionString = Constants.connectionString;

BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);

BlobServiceProperties sp = blobServiceClient.GetProperties();

// Set the service properties.
sp.DefaultServiceVersion = "2013-08-15";
BlobCorsRule bcr = new BlobCorsRule();
bcr.AllowedHeaders = "*";

bcr.AllowedMethods = "GET,POST";
bcr.AllowedOrigins = "http://www.contoso.com";
bcr.ExposedHeaders = "x-ms-*";
bcr.MaxAgeInSeconds = 5;
sp.Cors.Clear();
sp.Cors.Add(bcr);
blobServiceClient.SetProperties(sp);
```

Network failure

In some circumstances, lost network packets can lead to the storage service returning HTTP 404 messages to the client. For example, when your client application is deleting an entity from the table service, you see the client throw a storage exception reporting an "HTTP 404 (Not Found)" status message from the table service. When you investigate the table in the table storage service, you see that the service did delete the entity as requested.

The exception details in the client include the request ID (7e84f12d...) assigned by the table service for the request. You can use this information to locate the request details in the server-side storage logs by searching in the `request-id-header` column in the log file. You could also use the metrics to identify when failures such as this occur and then search the log files based on the time the metrics recorded this error. This log entry shows that the delete failed with an "HTTP (404) Client Other Error" status message. The same log entry also includes the request ID generated by the client in the `client-request-id` column (813ea74f...).

The server-side log also includes another entry with the same `client-request-id` value (813ea74f...) for a successful delete operation for the same entity and from the same client. This successful delete operation took place very shortly before the failed delete request.

The most likely cause of this scenario is that the client sent a delete request for the entity to the table service, which succeeded but did not receive an acknowledgment from the server (perhaps due to a temporary network issue). The client then automatically retried the operation (using the same `client-request-id`), and this retry failed because the entity had already been deleted.

If this problem occurs frequently, you should investigate why the client is failing to receive acknowledgments from the table service. If the problem is intermittent, you should trap the "HTTP (404) Not Found" error and log it in the client, but allow the client to continue.

The client is receiving HTTP 409 (Conflict) messages

The following table shows an extract from the server-side log for two client operations: `DeleteIfExists` followed immediately by `CreateIfNotExists` using the same blob container name. Each client operation results in two requests sent to the server, first a `GetContainerProperties` request to check if the container exists, followed by the `DeleteContainer` or `CreateContainer` request.

Timestamp	Operation	Result	Container name	Client request ID
05:10:13.7167225	GetContainerProperties	200	mmcont	c9f52c89-...
05:10:13.8167325	DeleteContainer	202	mmcont	c9f52c89-...
05:10:13.8987407	GetContainerProperties	404	mmcont	bc881924-...
05:10:14.2147723	CreateContainer	409	mmcont	bc881924-...

The code in the client application deletes and then immediately recreates a blob container using the same name: the `CreateIfNotExists` method (Client request ID bc881924-...) eventually fails with the HTTP 409 (Conflict) error. When a client deletes blob containers, tables, or queues, there's a brief period before the name becomes available again.

The client application should use unique container names whenever it creates new containers if the delete/recreate pattern is common.

Metrics show low PercentSuccess or analytics log entries have operations with transaction status of ClientOtherErrors

The **PercentSuccess** metric captures the percent of operations that were successful based on their HTTP Status Code. Operations with status codes of 2XX count as successful, whereas operations with status codes in 3XX, 4XX, and 5XX ranges are counted as unsuccessful and lower the **PercentSuccess** metric value. In the server-side storage log files, these operations are recorded with a transaction status of **ClientOtherErrors**.

It's important to note that these operations have been completed successfully and therefore don't affect other metrics, such as availability. Some examples of operations that execute successfully but that can result in unsuccessful HTTP status codes include:

- ResourceNotFound (Not Found 404), for example, from a `GET` request to a blob that doesn't exist.
- ResourceAlreadyExists (Conflict 409), for example, from a `CreateIfNotExists` operation where the resource already exists.
- ConditionNotMet (Not Modified 304), for example, from a conditional operation such as when a client sends an `ETag` value and an HTTP `If-None-Match` header to request an image only if it has been updated since the last operation.

You can find a list of common REST API error codes that the storage services return on the page [Common REST API error codes](#).

Capacity metrics show an unexpected increase in storage capacity usage

If you see sudden, unexpected changes in capacity usage in your storage account, you can investigate the reasons by first looking at your availability metrics; for example, an increase in the number of failed delete requests might lead to an increase in the amount of blob storage you're using as application-specific cleanup operations you might have expected to be freeing up space may not be working as expected (for example, because the SAS tokens used for freeing up space have expired).

Your issue arises from using the storage emulator for development or test

You typically use the storage emulator during development and testing to avoid the requirement for an Azure storage account. The common issues that can occur when you're using the storage emulator are:

- [Feature "X" is not working in the storage emulator.](#)
- [Error "The value for one of the HTTP headers is not in the correct format" when using the storage emulator.](#)
- [Running the storage emulator requires administrative privileges.](#)

Feature "X" is not working in the storage emulator

The storage emulator doesn't support all of the features of the Azure storage services, such as the file service. For more information, see [Use the Azure Storage Emulator for Development and Testing](#).

For those features that the storage emulator doesn't support, use the Azure storage service in the cloud.

Error "The value for one of the HTTP headers is not in the correct format" when using the storage emulator

You are testing your application that uses the Storage Client Library against the local storage emulator, and method calls such as `CreateIfNotExists` fail with the error message "The value for one of the HTTP headers is not in the correct format." This indicates that the version of the storage emulator you're using doesn't support the version of the storage client library you're using. The Storage Client Library adds the header `x-ms-version` to all the requests it makes. If the storage emulator doesn't recognize the value in the `x-ms-version` header, it rejects the request.

You can use the Storage Library Client logs to see the value of the `x-ms-version` header it's sending. You can also see the value of the `x-ms-version` header if you use Fiddler to trace the requests from your client application.

This scenario typically occurs if you install and use the latest version of the Storage Client Library without updating the storage emulator. You should either install the latest version of the storage emulator or use cloud storage instead of the emulator for development and testing.

Running the storage emulator requires administrative privileges

You're prompted for administrator credentials when you run the storage emulator. This only occurs when you're initializing the storage emulator for the first time. After you have initialized the storage emulator, you don't need administrative privileges to run it again.

For more information, see [Use the Azure Storage Emulator for Development and Testing](#). You can also initialize the storage emulator in Visual Studio, which will also require administrative privileges.

You are encountering problems installing the Azure SDK for .NET

When you try to install the SDK, it fails while trying to install the storage emulator on your local machine. The installation log contains one of the following messages:

- CAQuietExec: Error: Unable to access SQL instance
- CAQuietExec: Error: Unable to create database

The cause is an issue with the existing LocalDB installation. By default, the storage emulator uses LocalDB to persist data when it simulates the Azure storage services. You can reset your LocalDB instance by running the following commands in a command-prompt window before trying to install the SDK.

Console

```
sqllocaldb stop v11.0
sqllocaldb delete v11.0
delete %USERPROFILE%\WAStorageEmulatorDb3*. *
sqllocaldb create v11.0
```

The `delete` command removes any old database files from previous installations of the storage emulator.

You have a different issue with a storage service

If the previous troubleshooting sections don't include the issue you're having with a storage service, you should adopt the following approach to diagnosing and troubleshooting your issue.

- Check your metrics to see if there's any change from your expected baseline behavior. From the metrics, you may be able to determine whether the issue is transient or permanent and which storage operations the issue is affecting.
- You can use the metrics information to help you search your server-side log data for more detailed information about any errors that are occurring. This information may help you troubleshoot and resolve the issue.
- If the information in the server-side logs isn't sufficient to troubleshoot the issue successfully, you can use the Storage Client Library client-side logs to investigate the behavior of your client application and tools such as Fiddler and Wireshark to investigate your network.

For more information about using Fiddler, see [Appendix 1: Using Fiddler to capture HTTP and HTTPS traffic](#).

For more information about using Wireshark, see [Appendix 2: Using Wireshark to capture network traffic](#).

Appendices

The appendices describe several tools that you may find useful when you're diagnosing and troubleshooting issues with Azure Storage (and other services). These tools aren't part of Azure Storage, and some are third-party products. As such, the tools discussed in these appendices

aren't covered by any support agreement you may have with Microsoft Azure or Azure Storage. Therefore, as part of your evaluation process, you should examine the licensing and support options available from the providers of these tools.

Appendix 1: Using Fiddler to capture HTTP and HTTPS traffic

[Fiddler](#) is a useful tool for analyzing the HTTP and HTTPS traffic between your client application and the Azure storage service you're using.

ⓘ Note

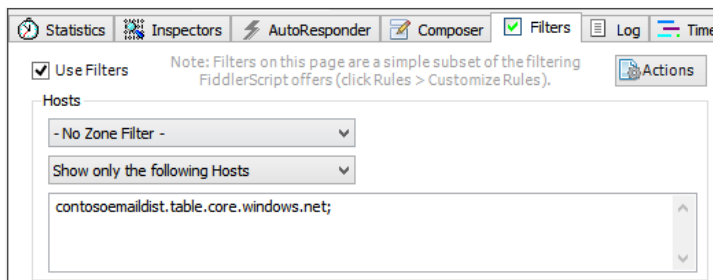
Fiddler can decode HTTPS traffic. You should read the Fiddler documentation carefully to understand how it does this and its security implications.

This appendix provides a brief walkthrough of how to configure Fiddler to capture traffic between the local machine where you have installed Fiddler and the Azure storage services.

After you have launched Fiddler, it will begin capturing HTTP and HTTPS traffic on your local machine. The following are some useful commands for controlling Fiddler:

- Stop and start capturing traffic. On the main menu, go to **File** and then select **Capture Traffic** to toggle capturing on and off.
- Save captured traffic data. On the main menu, go to **File**, select **Save**, and then select **All Sessions**. This enables you to save the traffic in a Session Archive file. You can reload a Session Archive later for analysis or send it if requested to Microsoft support.

To limit the amount of traffic that Fiddler captures, you can use filters that you configure in the **Filters** tab. The following screenshot shows a filter that captures only traffic sent to the `contosoemaildist.table.core.windows.net` storage endpoint:

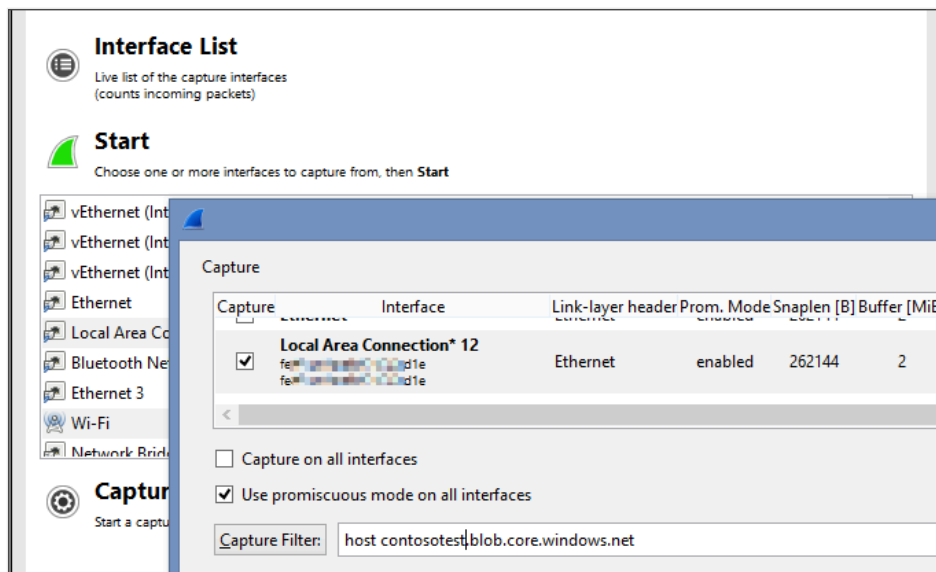


Appendix 2: Using Wireshark to capture network traffic

[Wireshark](#) is a network protocol analyzer that enables you to view detailed packet information for a wide range of network protocols.

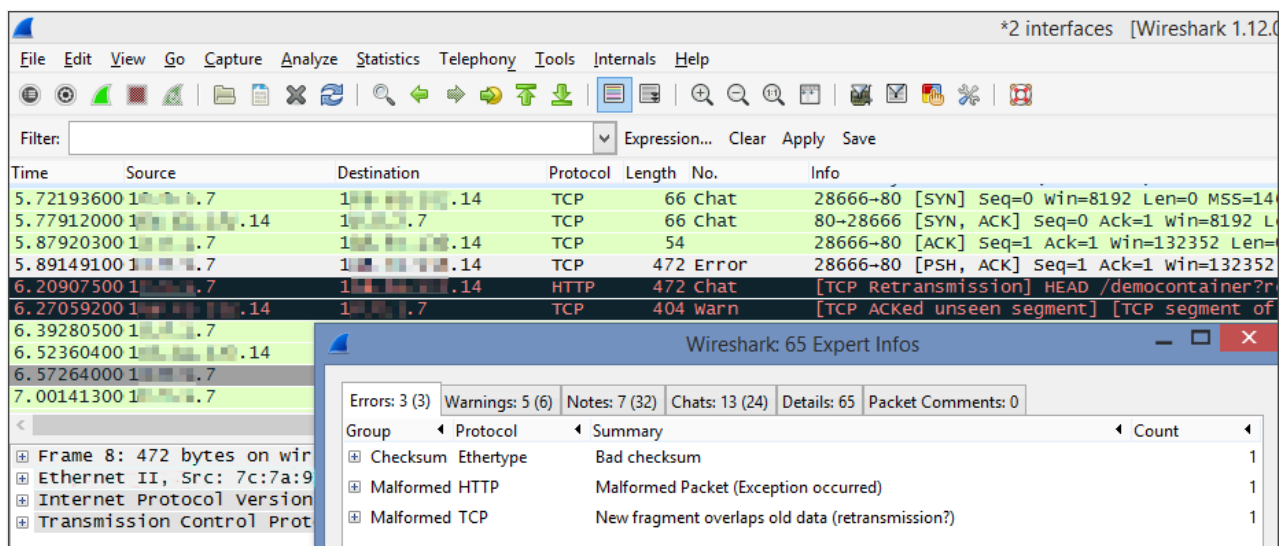
The following procedure shows you how to capture detailed packet information for traffic from the local machine where you installed Wireshark to the table service in your Azure storage account.

1. Launch Wireshark on your local machine.
2. In the **Start** section, select the local network interface or interfaces that are connected to the internet.
3. Select **Capture Options**.
4. Add a filter to the **Capture Filter** textbox. For example, `host contosoemaildist.table.core.windows.net` will configure Wireshark to capture only packets sent to or from the table service endpoint in the *contosoemaildist* storage account. Check out the [complete list of Capture Filters](#).

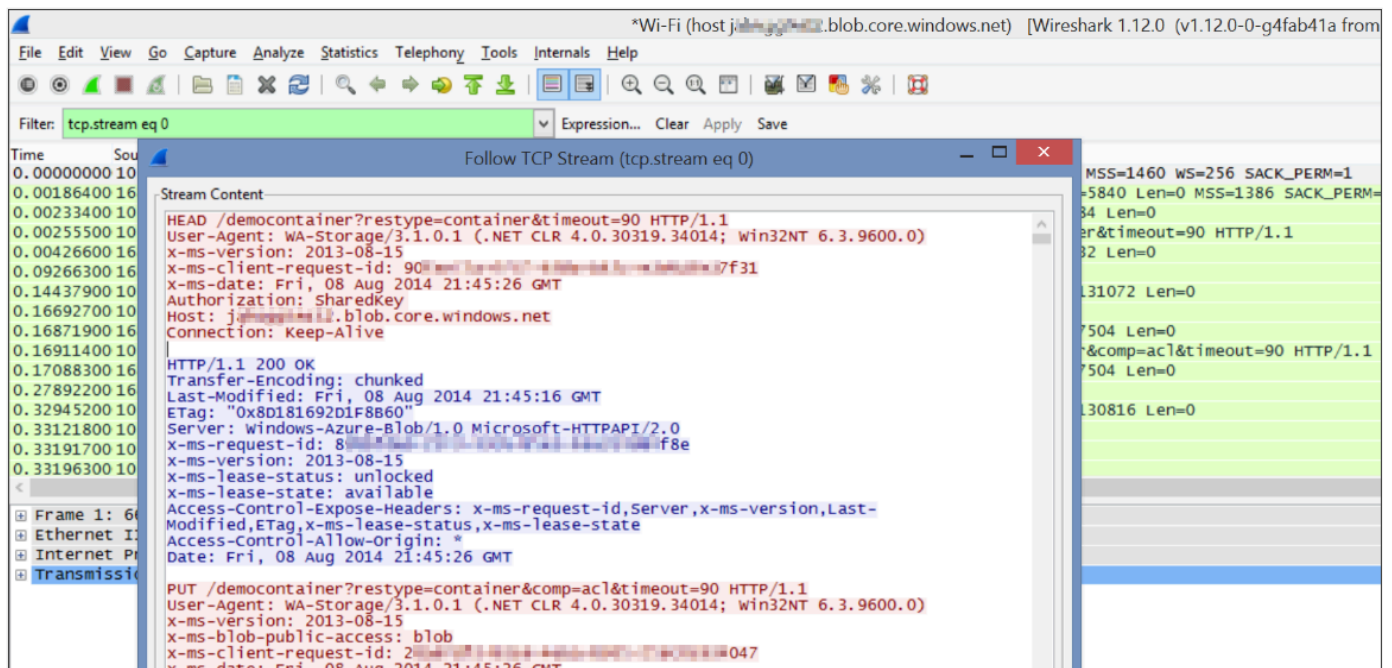


5. Select **Start**. Wireshark will now capture all the packets sent to or from the table service endpoint as you use your client application on your local machine.
6. When you have finished, select **Capture > Stop** on the main menu.
7. To save the captured data in a Wireshark Capture file, select **File > Save** on the main menu.

Wireshark will highlight any errors that exist in the **packetlist** window. You can also use the **Expert Info** window (select **Analyze > Expert Info**) to view a summary of errors and warnings.



You can also choose to view the TCP data as the application layer sees it by right-clicking on the TCP data and selecting **Follow TCP Stream**. This is useful if you captured your dump without a capture filter. For more information, see [Following TCP Streams](#).



Note

For more information about using Wireshark, see the [Wireshark Users Guide](#).

Appendix 4: Using Excel to view metrics and log data

Many tools enable you to download the Storage Metrics data from Azure table storage in a delimited format that makes it easy to load the data into Excel for viewing and analysis. Storage Logging data from Azure Blob Storage is already in a delimited format that you can load into Excel. However, you'll need to add appropriate column headings based on the information at [Storage Analytics Log Format](#) and [Storage Analytics Metrics Table Schema](#).

To import your Storage Logging data into Excel after you download it from blob storage:

- On the **Data** menu, select **From Text**.
- Browse to the log file you want to view and select **Import**.
- In step 1 of the **Text Import Wizard**, select **Delimited**.

In step 1 of the **Text Import Wizard**, select **Semicolon** as the only delimiter and choose double-quote as the **Text qualifier**. Then select **Finish** and choose where to place the data in your workbook.

Appendix 5: Monitoring with Application Insights for Azure DevOps

You can also use the Application Insights feature for Azure DevOps as part of your performance and availability monitoring. This tool can:

- Make sure your web service is available and responsive. Whether your app is a web site or a device app that uses a web service, it can test your URL every few minutes from locations worldwide and let you know if there's a problem.
- Quickly diagnose any performance issues or exceptions in your web service. Find out if CPU or other resources are being stretched, get stack traces from exceptions, and easily search through log traces. If the app's performance drops below acceptable limits, Microsoft can send you an email. You can monitor both .NET and Java web services.

You can find more information at [What is Application Insights](#).

Next steps

For more information about analytics in Azure Storage, see these resources:

- [Monitor a storage account in the Azure portal](#)
- [Storage analytics](#)
- [Storage analytics metrics](#)
- [Storage analytics metrics table schema](#)

- [Storage analytics logs](#)
- [Storage analytics log format](#)

Third-party information disclaimer

The third-party products that this article discusses are manufactured by companies that are independent of Microsoft. Microsoft makes no warranty, implied or otherwise, about the performance or reliability of these products.

Last updated on 04/09/2024

Troubleshoot latency using Storage Analytics logs

Diagnosing and troubleshooting are key skills for building and supporting client applications with Azure Storage.

Because of the distributed nature of an Azure application, diagnosing and troubleshooting both errors and performance issues may be more complex than in traditional environments.

The following steps demonstrate how to identify and troubleshoot latency issues using Azure Storage Analytics logs and optimize the client application.

Recommended steps

1. Download the [Storage Analytics logs](#).
2. Use the following PowerShell script to convert the raw format logs into tabular format:

PowerShell

```
$Columns =  
    (  
        "version-number",  
        "request-start-time",  
        "operation-type",  
        "request-status",  
        "http-status-code",  
        "end-to-end-latency-in-ms",  
        "server-latency-in-ms",  
        "authentication-type",  
        "requester-account-name",  
        "owner-account-name",  
        "service-type",  
        "request-url",  
        "requested-object-key",  
        "request-id-header",  
        "operation-count",  
        "requester-ip-address",  
        "request-version-header",  
        "request-header-size",  
        "request-packet-size",  
        "response-header-size",  
        "response-packet-size",  
        "request-content-length",  
        "request-md5",  
        "server-md5",  
        "etag-identifier",  
        "last-modified-time",  
        "conditions-used",  
        "user-agent-header",
```

3. The script will launch a GUI window where you can filter the information by columns, as shown below.

4. Narrow down the log entries based on **operation-type**, and look for the log entry created during the issue's time frame.

5. During the time when the issue occurred, the following values are important:

- `Operation-type` = `GetBlob`
- `request-status` = `SASNetworkError`
- `End-to-End-Latency-In-Ms` = 8,453
- `Server-Latency-In-Ms` = 391

End-to-End Latency is calculated using the following equation:

- $\text{End-to-End Latency} = \text{Server-Latency} + \text{Client Latency}$

Calculate the Client Latency using the log entry:

- $\text{Client Latency} = \text{End-to-End Latency} - \text{Server-Latency}$

Example: $8,453 - 391 = 8,062$ ms

The following table provides information about the high-latency OperationType and RequestStatus results:

[Expand table](#)

Blob Type	RequestStatus= Success	RequestStatus= (SAS)NetworkError	Recommendation
GetBlob	Yes	No	GetBlob Operation: RequestStatus = Success
GetBlob	No	Yes	GetBlob Operation: RequestStatus = (SAS)NetworkError
PutBlob	Yes	No	Put Operation: RequestStatus = Success
PutBlob	No	Yes	Put Operation: RequestStatus = (SAS)NetworkError

Status results

GetBlob Operation: RequestStatus = Success

Check the following values as mentioned in step 5 of the [Recommended steps](#) section:

- End-to-End Latency
- Server-Latency
- Client-Latency

In a `GetBlob Operation` with `RequestStatus = Success`, if `Max Time` is spent in `Client-Latency`, this indicates that Azure Storage is spending a large volume of time writing data to the client. This delay indicates a Client-Side issue.

Recommendation:

- Investigate the code in your client.

- Use Wireshark, Microsoft Message Analyzer, or Tcping to investigate network connectivity issues from the client.

GetBlob Operation: RequestStatus = (SAS)NetworkError

Check the following values as mentioned in step 5 of the [Recommended steps](#) section:

- End-to-End Latency
- Server-Latency
- Client-Latency

In a `GetBlob Operation` with `RequestStatus = (SAS)NetworkError`, if `Max Time` is spent in `Client-Latency`, the most common issue is that the client is disconnecting before a timeout expires in the storage service.

Recommendation:

- Investigate the code in your client to understand why and when the client disconnects from the storage service.
- Use Wireshark, Microsoft Message Analyzer, or Tcping to investigate network connectivity issues from the client.

Put Operation: RequestStatus = Success

Check the following values as mentioned in step 5 of the [Recommended steps](#) section:

- End-to-End Latency
- Server-Latency
- Client-Latency

In a `Put Operation` with `RequestStatus = Success`, if `Max Time` is spent in `Client-Latency`, this indicates that the Client is taking more time to send data to the Azure Storage. This delay indicates a Client-Side Issue.

Recommendation:

- Investigate the code in your client.
- Use Wireshark, Microsoft Message Analyzer, or Tcping to investigate network connectivity issues from the client.

Put Operation: RequestStatus = (SAS)NetworkError

Check the following values as mentioned in step 5 of the "Recommended steps" section:

- End-to-End Latency
- Server-Latency
- Client-Latency

In a PutBlob Operation with RequestStatus = (SAS)NetworkError, if Max Time is spent in Client-Latency, the most common issue is that the client is disconnecting before a timeout expires in the storage service.

Recommendation:

- Investigate the code in your client to understand why and when the client disconnects from the storage service.
- Use Wireshark, Microsoft Message Analyzer, or Tcping to investigate network connectivity issues from the client.

Third-party information disclaimer

The third-party products that this article discusses are manufactured by companies that are independent of Microsoft. Microsoft makes no warranty, implied or otherwise, about the performance or reliability of these products.

Last updated on 04/09/2024

Troubleshoot Azure Storage Explorer

ⓘ Note

Was this article helpful? Your input is important to us. Please use the **Feedback** button on this page to let us know how well this article worked for you or how we can improve it.

Summary

Microsoft Azure Storage Explorer is a standalone app that makes it easy to work with Azure Storage data on Windows, macOS, and Linux. The app can connect to storage accounts hosted on Azure, national clouds, and Azure Stack.

This guide summarizes solutions for issues that are commonly seen in Storage Explorer.

Azure permissions issues

[Azure role-based access control \(RBAC\)](#) enables highly granular access management of Azure resources by combining sets of permissions into roles. Here are some strategies to get Azure RBAC working optimally in Storage Explorer.

How do I access my resources in Storage Explorer?

If you can't access storage resources through Azure RBAC, you might not have the right roles assigned. The following sections describe the permissions Storage Explorer currently requires for access to your storage resources. Contact your Azure account admin if you're not sure you have the right roles or permissions.

"Read: List/Get Storage Accounts" permissions issue

You must have permission to list storage accounts. To get this permission, you must be assigned the Reader role.

List storage account keys

Storage Explorer can also use account keys to authenticate requests. You can get access to account keys through more powerful roles, such as the Contributor role.

ⓘ Note

Access keys grant unrestricted permissions to anyone who holds them. As a result, don't hand out these keys to account users. If you need to revoke access keys, you can regenerate them from the [Azure portal](#) [↗].

Data roles

You must be assigned at least one role that grants access to read data from resources. For example, if you want to list or download blobs, you need, at the least, the Storage Blob Data Reader role.

Why do I need a management layer role to see my resources in Storage Explorer?

Azure Storage has two layers of access: management and data. You access subscriptions and storage accounts through the management layer. You access containers, blobs, and other data resources through the data layer. For example, if you want to get a list of your storage accounts from Azure, you send a request to the management endpoint. If you want a list of blob containers in an account, you send a request to the appropriate service endpoint.

Azure roles can grant you permissions for management or data layer access. The Reader role, for example, grants read-only access to management layer resources.

Strictly speaking, the Reader role provides no data layer permissions and isn't necessary for accessing the data layer.

Storage Explorer makes it easy to access your resources by gathering the necessary information to connect to your Azure resources. For example, to display your blob containers, Storage Explorer sends a "list containers" request to the blob service endpoint. To get that endpoint, Storage Explorer searches the list of subscriptions and storage accounts you have access to. To find your subscriptions and storage accounts, Storage Explorer also needs access to the management layer.

If you don't have a role that grants any management layer permissions, Storage Explorer can't get the information it needs to connect to the data layer.

What if I can't get the management layer permissions I need from my admin?

If you want to access blob containers, Azure Data Lake Storage Gen2 containers or directories, or queues, you can attach to those resources by using your Azure credentials.

1. Open the **Connect** dialog box.
2. Select the resource type you want to connect to.
3. Select **Sign in using Microsoft Entra ID > Next**.
4. Select the user account and tenant associated with the resource you're attaching to, and then select **Next**.
5. Type the URL to the resource and then type a unique display name for the connection. Select **Next > Connect**.

Currently, there's no Azure RBAC-related solution for other resource types. As a workaround, you can request a shared access signature URL and then attach it to your resource:

1. Open the **Connect** dialog box.
2. Select the resource type you want to connect to.
3. Select **Shared access signature (SAS) > Next**.
4. Type the shared access signature URL you received and enter a unique display name for the connection. Select **Next > Connect**.

For more information about how to attach to resources, see [Attach to an individual resource](#).

Recommended Azure built-in roles

Several Azure built-in roles provide the permissions needed to use Storage Explorer. Some of those roles include:

- [Owner](#): Manage everything, including access to resources.
- [Contributor](#): Manage everything, excluding access to resources.
- [Reader](#): Read and list resources.
- [Storage Account Contributor](#): Full management of storage accounts.
- [Storage Blob Data Owner](#): Full access to Azure Storage blob containers and data.
- [Storage Blob Data Contributor](#): Read, write, and delete Azure Storage containers and blobs.
- [Storage Blob Data Reader](#): Read and list Azure Storage containers and blobs.

Note

The Owner, Contributor, and Storage Account Contributor roles grant account key access.

TLS/SSL certificate issues

This section discusses TLS/SSL certificate issues.

Understand TLS/SSL certificate issues

Make sure you understand the [TLS/SSL certificates section](#) in the Storage Explorer networking documentation before you continue.

Use system proxy

If you're only using features that support the **use system proxy** setting, try using that setting. To learn more about the **system proxy** setting, see [Network connections in Storage Explorer](#).

Import TLS/SSL certificates

If you have a copy of the self-signed certificates, you can instruct Storage Explorer to trust them:

1. Obtain a Base-64 encoded X.509 (.cer) copy of the certificate.
2. Go to **Edit > SSL Certificates > Import Certificates**. Then use the file picker to find, select, and open the .cer file.

This error might also occur if there are multiple certificates (root and intermediate). To fix this error, you must import all certificates.

Find TLS/SSL certificates

If you don't have a copy of the self-signed certificates, ask your IT admin for help.

To find certificates, follow these steps:

1. Install OpenSSL:
 - [Windows](#): Any of the light versions is sufficient.
 - Mac: OpenSSL is included with your operating system.
 - Linux: OpenSSL is included with your operating system.
2. Run OpenSSL:
 - Windows: Open the installation directory, select `/bin/`, and then double-click `openssl.exe`.
 - Mac: Run `openssl` from a terminal.

- Linux: Run `openssl` from a terminal.
3. Run the command `openssl s_client -showcerts -connect <hostname>:443` for any of the Microsoft or Azure host names that your storage resources are behind. For more information, see this [list of host names frequently accessed by Storage Explorer](#).
 4. Look for self-signed certificates. If the subject (`"s:"`) and issuer (`"i:"`) are the same, the certificate is most likely self-signed.
 5. When you find the self-signed certificates, for each one, copy and paste everything from `--`
`---BEGIN CERTIFICATE-----` to `-----END CERTIFICATE-----` into a new .cer file.
 6. Open Storage Explorer and go to **Edit > SSL Certificates > Import Certificates**. Then use the file picker to find, select, and open the .cer files you created.

Disable TLS/SSL certificate validation

If you can't find any self-signed certificates by following these steps, contact us through the feedback tool. You can also open Storage Explorer from the command line with the `--ignore-certificate-errors` flag. When opened with this flag, Storage Explorer ignores certificate errors.

Important

Use of this flag isn't recommended.

Sign-in issues

This section discusses sign-in issues you might encounter.

Understand sign-in

Make sure you understand the [Sign in to Storage Explorer](#) documentation before you continue.

Frequently having to reenter credentials

Having to reenter credentials is most likely the result of Conditional Access policies set by your Microsoft Entra admin. When Storage Explorer asks you to reenter credentials from the account panel, you should see an **Error details** link. Select it to see why Storage Explorer is asking you to reenter credentials. Conditional Access policy errors that require reentering of credentials might look something like these errors:

The refresh token has expired.

You must use multifactor authentication to access.

Your admin made a configuration change.

To reduce the frequency of having to reenter credentials because of errors like the preceding ones, talk to your Microsoft Entra admin.

Other conditional access policy issues

If other conditional access policies need to be satisfied for your account, try using either the **Authentication Broker** or **Default Web Browser** value for the **Sign in with** setting. For information on that setting, see [Changing where sign-in happens](#).

Issues with authentication broker on Windows

The Windows operating system uses the Web Account Manager (WAM) as its authentication broker. If you're using the **Authentication Broker** sign-in method on Windows and experience problems, see [this guide](#) for steps to resolve common errors.

Browser complains about HTTP redirect or insecure connection during sign-in

When Storage Explorer performs sign-in in your web browser, it redirects to localhost at the end of the sign-in process. Browsers sometimes raise a warning or error that the redirect is being performed with HTTP instead of HTTPS. Some browsers might also try to force the redirect to be performed with HTTPS. If either of these problems happens, depending on your browser, you have options:

- Ignore the warning.
- Add an exception for localhost.
- Disable force HTTPS, either globally or just for localhost.

If you can't do any of those options, you can also [change where sign-in happens](#) to integrated sign-in to avoid using your browser altogether.

Unable to acquire token, tenant is filtered out

Sometimes you see an error message that says a token can't be acquired because a tenant is filtered out. This error means you're trying to access a resource that's in a tenant you filtered out. To include the tenant, go to the **Account Panel**. Make sure the check box for the tenant specified in the error is selected. For more information on filtering tenants in Storage Explorer, see [Managing accounts](#).

Authentication library didn't start properly

If, on startup, you see an error message that says Storage Explorer's authentication library didn't start properly, make sure your installation environment meets all [prerequisites](#). Not meeting prerequisites is the most likely cause of this error message.

If you believe that your installation environment meets all prerequisites, [open an issue on GitHub](#) . When you open your issue, make sure to include:

- Your OS
- Your version of Storage Explorer
- Your [authentication logs](#) from an unsuccessful launch of Storage Explorer. Verbose authentication logging is automatically enabled after this type of error occurs.

Blank window when you use integrated sign-in

If you choose to use **Integrated Sign-in** and you see a blank sign-in window, you likely need to switch to a different sign-in method. Blank sign-in dialog boxes most often occur when an Active Directory Federation Services server prompts Storage Explorer to perform a redirect that's unsupported by Electron.

To change to a different sign-in method, change the **Sign in with** setting under **Settings > Application > Sign-in**. For information on the different types of sign-in methods, see [Changing where sign in happens](#).

Reauthentication loop or user principal name (UPN) change

If you're in a reauthentication loop or changed the UPN of one of your accounts, try these steps:

1. Open Storage Explorer.
2. Go to **Help > Reset**.
3. Make sure at least **Authentication** is selected. Clear other items you don't want to reset.
4. Select **Reset**.
5. Restart Storage Explorer and try to sign in again.

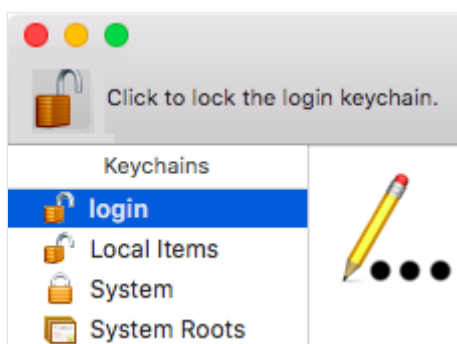
If you continue to have problems after you reset, try these steps:

1. Open Storage Explorer.
2. Remove all accounts and then close Storage Explorer.
3. Delete the `.IdentityService` folder from your machine. On Windows, find the folder at `C:\Users\<username>\AppData\Local`. For macOS and Linux, find the folder at the root of your user directory.
4. If you're running macOS or Linux, also delete the `Microsoft.Developer.IdentityService` entry from your operating system's keystore. On the macOS, the keystore is the Gnome Keychain application. In Linux, the application is typically called *Keyring*, but the name might differ depending on your distribution.
5. Restart Storage Explorer and try to sign in again.

macOS: Keychain errors or no sign-in window

macOS Keychain can sometimes enter a state that causes problems for the Storage Explorer authentication library. To get Keychain out of this state, follow these steps:

1. Close Storage Explorer.
2. Open Keychain by selecting `Command` + `Space`, type "keychain" in the search box, and select `Enter`.
3. Select the **login** keychain.
4. Select the **padlock** to lock the keychain. After the process finishes, the **padlock** appears locked. It might take a few seconds, depending on what apps you have open.



5. Open Storage Explorer.
6. You're prompted with a message like "Service hub wants to access the Keychain." Type your Mac admin account password and select **Always Allow**. Or select **Allow** if **Always Allow** isn't available.

7. Try to sign in.

Linux: No application window or password manager errors at startup

If you start Storage Explorer on a Linux system, you might encounter one of the following problems:

- No application window appears.
- Errors about the system's password manager occur.

Storage Explorer uses the system's credential manager to protect your data, including sign-in credentials and SAS connections. If it doesn't detect a compatible credential manager application, Storage Explorer can't start. If your system doesn't have a local credential management tool installed, you can install a tool compatible with `libsecret`. For example, on Linux systems that use the GNOME desktop environment, you can install [Seahorse](#).

Storage Explorer usually creates a default keyring if it doesn't exist at startup. However, in some cases, a default keyring isn't created, resulting in no application window or password manager service errors. To resolve the problems, manually set a default keyring.

If you're using Seahorse and there are no existing keyrings, or you want to create a new one, follow these steps to create a default keyring:

1. Start the "Passwords and Keys" application.
2. Select the + button, and then select **Password keyring**.
3. Set a name and a password for the new keyring.
4. Right-click the new keyring and select **Set as default**.

If you use the Storage Explorer snap, you must also make sure Storage Explorer is connected to your system's password manager. To connect the password manager, run the following command:

Bash

```
snap connect storage-explorer:password-manager-service :password-manager-service
```

Default browser doesn't open

If your default browser doesn't open when you try to sign in, try all of the following techniques:

- Restart Storage Explorer.
- Open your browser manually before you start to sign in.
- Try using **Integrated Sign-In**. For instructions, see [Changing where sign-in happens](#).

Other sign-in problems

If none of the preceding instructions apply to your sign-in problem or if they don't resolve your sign-in problem, [open an issue on GitHub](#).

Missing subscriptions and broken tenants

If you can't retrieve your subscriptions after you successfully sign in, try the following troubleshooting methods:

- Verify that your account has access to the subscriptions you expect. You can verify your access by signing in to the portal for the Azure environment you're trying to use.
- Make sure you sign in to the correct Azure environment (Azure, Azure China 21Vianet, Azure US Government, or Custom Environment).
- Make sure you configured the Storage Explorer proxy correctly if you're behind a proxy server.
- Remove and add back the account.
- Check which error messages are reported for the tenants that are failing if there's a **More information** or **Error details** link. If you're not sure how to respond to the error messages, [open an issue in GitHub](#).

Problem interacting with your OS credential store during an AzCopy transfer

If you see this message on Windows, the Windows Credential Manager is most likely full. To make room in the Windows Credential Manager, follow these steps:

1. Close Storage Explorer.
2. On the **Start** menu, search for **Credential Manager** and open it.
3. Go to **Windows Credentials**.
4. Under **Generic Credentials**, look for entries associated with programs you no longer use and delete them. You can also look for entries like `azcopy/aadtoken/<some number>` and delete those entries.

If the message continues to appear after completing the preceding steps, or if you encounter this message on platforms other than Windows, you can [open an issue on GitHub](#).

Can't remove an attached storage account or resource

If you can't remove an attached account or storage resource through the UI, you can manually delete all attached resources by deleting the following folders:

 [Expand table](#)

Platform	Folder path
Windows	C:\Users\<username>\AppData\StorageExplorer
macOS	~/Library/Application Support/StorageExplorer
Linux	~/.config/StorageExplorer

Close Storage Explorer before you delete these folders.

 Note

If you imported any TLS/SSL certificates, back up the contents of the `certs` directory. Later, you can use the backup to reimport your TLS/SSL certificates.

Proxy issues

Storage Explorer supports connecting to Azure Storage resources via a proxy server. If you experience any issues when you connect to Azure via proxy, here are some suggestions.

Storage Explorer only supports basic authentication with proxy servers. Other authentication methods, such as New Technology LAN Manager (NTLM), aren't supported.

 Note

Storage Explorer doesn't support proxy autoconfig files for configuring proxy settings.

Verify Storage Explorer proxy settings

The **Application > Proxy > Proxy configuration** setting determines which source Storage Explorer gets the proxy configuration from.

If you select **Use environment variables**, make sure to set the `HTTPS_PROXY` or `HTTP_PROXY` environment variables. Environment variables are case-sensitive, so be sure to set the correct variables. If these variables are undefined or invalid, Storage Explorer doesn't use a proxy. Restart Storage Explorer after you modify any environment variables.

If you select **Use app proxy settings**, make sure the in-app proxy settings are correct.

Steps for diagnosing issues

If you're still experiencing issues, try these troubleshooting methods:

1. If you can connect to the internet without using your proxy, verify that Storage Explorer works without proxy settings enabled. If Storage Explorer connects successfully, there might be an issue with your proxy server. Work with your admin to identify the problems.
2. Verify that other applications that use the proxy server work as expected.
3. Verify that you can connect to the portal for the Azure environment you're trying to use.
4. Verify that you can receive responses from your service endpoints. Enter one of your endpoint URLs into your browser. If you can connect, you should receive an `InvalidQueryParameterValue` or a similar XML response.
5. Check whether someone else using Storage Explorer with the same proxy server can connect. If they can, you might have to contact your proxy server admin.

Tools for diagnosing problems

A networking tool, such as Fiddler, can help you diagnose problems.

1. Configure your networking tool as a proxy server running on the local host. If you need to continue working behind an actual proxy, you might need to configure your networking tool to connect through the proxy.
2. Check the port number used by your networking tool.
3. Configure Storage Explorer proxy settings to use the local host and the networking tool's port number, such as `localhost:8888`.

When set correctly, your networking tool logs network requests that Storage Explorer makes to management and service endpoints.

If your networking tool doesn't appear to be logging Storage Explorer traffic, try testing your tool with a different application. For example, enter the endpoint URL for one of your storage

resources, such as `https://contoso.blob.core.windows.net/` in a web browser. You should receive a response similar to this code sample:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>InvalidQueryParameterValue</Code>
  <Message>Value for one of the query parameters specified in the request URI is
invalid.
    RequestId:<RequestId> Time:2017-04-10T21:42:17.3863214Z</Message>
  <QueryParameterName>comp</QueryParameterName>
  <QueryParameterValue/>
  <Reason/>
</Error>
```

The response suggests the resource exists, even though you can't access it.

If your networking tool only shows traffic from other applications, you might need to adjust the proxy settings in Storage Explorer. Otherwise, you might need to adjust your tool's settings.

Contact proxy server admin

If your proxy settings are correct, you might need to contact your proxy server admin to:

- Make sure your proxy doesn't block traffic to Azure management or resource endpoints.
- Verify the authentication protocol used by your proxy server. Storage Explorer only supports basic authentication protocols. Storage Explorer doesn't support NTLM proxies.

"Unable to Retrieve Children" error message

If you're connected to Azure through a proxy, verify that your proxy settings are correct.

If the owner of a subscription or account granted you access to a resource, verify that you have "read" or "list" permissions for that resource.

Connection string doesn't have complete configuration settings

If you receive this error message, you might not have the necessary permissions to get the keys for your storage account. To check, go to the portal and find your storage account. Right-click the node for your storage account and select **Open in Portal**. Then, go to the **Access Keys** pane. If you don't have permission to view keys, you might see a "You don't have access" message. To

work around this problem, get either an account name and key or an account shared access signature and use it to attach the storage account.

If you do see the account keys, file an issue in GitHub so we can help you resolve the problem.

Adding custom connections results in a `TypeError`

The local credential manager might store corrupted connection data if you receive the following error message when you try to add a custom connection:

```
Error occurred while adding new connection: TypeError: Cannot read property 'version' of undefined
```

To work around this problem, try deleting and adding back your corrupted local connections:

1. Start Storage Explorer. From the menu, go to **Help > Toggle Developer Tools**.
2. In the opened window, on the **Application** tab, go to **Local Storage > file://** on the left side.
3. Depending on the type of connection you're having an issue with, look for its key. Then copy its value into a text editor. The value is an array of your custom connection names, such as:

- Storage accounts
 - `StorageExplorer_CustomConnections_Accounts_v1`
- Blob containers
 - `StorageExplorer_CustomConnections_Blobs_v1`
 - `StorageExplorer_CustomConnections_Blobs_v2`
- File shares
 - `StorageExplorer_CustomConnections_Files_v1`
- Queues
 - `StorageExplorer_CustomConnections_Queues_v1`
- Tables
 - `StorageExplorer_CustomConnections_Tables_v1`

4. After you save your current connection names, set the value in **Developer Tools** to `[]`.

To preserve the connections that aren't corrupted, use the following steps to locate the corrupted connections. If you don't mind losing all existing connections, skip these steps and follow the platform-specific instructions to clear your connection data.

1. From a text editor, add back each connection name to **Developer Tools**. Then check whether the connection is still working.
2. If a connection is working correctly, it isn't corrupted; you can safely leave it there. If a connection isn't working, remove its value from **Developer Tools**, and record it so you can add it back later.
3. Repeat for each connection.

After removing connection names, you must clear their corrupted data. Then you can add the connections back by using the standard connect steps in Storage Explorer.

Windows

1. On the **Start** menu, search for **Credential Manager** and open it.
2. Go to **Windows Credentials**.
3. Under **Generic Credentials**, look for entries that have the `<connection_type_key>/<corrupted_connection_name>` key. An example is `StorageExplorer_CustomConnections_Accounts_v1/account1`.
4. Delete and add back these connections.

If you still encounter this error after you run these steps, or if you want to share what you suspect corrupted the connections, [open an issue](#) on our GitHub page.

Issues with a shared access signature URL

If you connect to a service through a shared access signature URL and experience an error, verify the following conditions:

- The URL provides the necessary permissions to read or list resources.
- The URL isn't expired.
- The access policy isn't revoked if the shared access signature URL is based on an access policy.

If you accidentally attach by using an invalid shared access signature URL and now can't detach, follow these steps:

1. When you're running Storage Explorer, select **F12** to open the **Developer Tools** window.
2. On the **Application** tab, select **Local Storage** > **file://** on the left side.
3. Find the key associated with the service type of the shared access signature URI. For example, if the bad shared access signature URI is for a blob container, look for the key

named `StorageExplorer_AddStorageServiceSAS_v1_blob`.

- The value of the key is a JSON array. Find the object associated with the bad URI, and delete it.
- Select `Ctrl + R` to reload Storage Explorer.

Storage Explorer dependencies

Windows

Storage Explorer comes packaged with all dependencies it needs to run on Windows.

ⓘ Note

Some components need command line access. If restricted, Storage Explorer might not work as expected. If you encounter issues, contact your system administrator.

Patch Storage Explorer for newer versions of .NET Core

For Storage Explorer 1.7.0 or earlier versions, you might need to patch the version of .NET Core that Storage Explorer uses:

- Go to the [NuGet](#) page and download version 1.5.43 of StreamJsonRpc from the **Download package** link on the right side.
- After you download the package, change its file extension from `.nupkg` to `.zip`.
- Unzip the package.
- Open the `streamjsonrpc.1.5.43/lib/netstandard1.1` folder.
- Copy `StreamJsonRpc.dll` to the following locations in the Storage Explorer folder:
 - `StorageExplorer/resources/app/ServiceHub/Services/Microsoft.Developer.IdentitySer vice`
 - `StorageExplorer/resources/app/ServiceHub/Hosts/ServiceHub.Host.Core.CLR.x64`

"Open In Explorer" button in the Azure portal doesn't work

If the **Open In Explorer** button in the Azure portal doesn't work, make sure you're using a compatible browser. The following browsers are tested for compatibility:

- Microsoft Edge
- Mozilla Firefox
- Google Chrome
- Microsoft Internet Explorer

Gather logs

When you report an issue to GitHub, you might be asked to gather certain logs to help diagnose your issue.

Storage Explorer logs

Storage Explorer logs various events to its own application logs. You can easily access these logs by selecting **Help > Open Logs Directory**. By default, Storage Explorer logs at a low level of verbosity. Use the **Trace** log level when troubleshooting, as it's the most verbose level.

To change the verbosity level, go to **Settings** (the **gear** symbol in the vertical toolbar) > **Application > Logging > Log Level** and select the desired log level. Restart Storage Explorer, and then reproduce the issue.

Logs are split into folders for each session of Storage Explorer that you run. To share log files, place them in a zip archive. Use different folders for files from different sessions.

Authentication logs

For issues related to sign-in or Storage Explorer's authentication library, you most likely need to gather authentication logs. Storage Explorer stores authentication logs at the following locations:

 Expand table

Platform	Folder path
Windows	C:\Users\<username>\AppData\Local\Temp\servicehub\logs
macOS	~/ServiceHub/logs
Linux	~/ServiceHub/logs

Generally, follow these steps to gather the logs:

1. Go to **Settings** (the **gear** symbol in the vertical toolbar) > **Application** > **Sign-in**. Select **Verbose Authentication Logging**. If Storage Explorer fails to start because of an issue with its authentication library, this step is done for you.
2. Close Storage Explorer.
3. (Optional) Clear out existing logs from the `logs` folder. This step reduces the amount of information you have to send.
4. Open Storage Explorer and reproduce your issue.
5. Close Storage Explorer.
6. Zip the contents of the `logs` folder.

AzCopy logs

If you're having trouble transferring data, you might need to get the AzCopy logs. By default, AzCopy logs at a low level of verbosity. The **Trace** log level is recommended when troubleshooting as it's the most verbose level.

To change the verbosity level, go to **Settings** (the **gear** symbol in the vertical toolbar) > **Transfers** > **AzCopy** > **Log Level** and set the desired log level. Restart Storage Explorer, then reproduce the issue you're having.

You can find AzCopy logs through two different methods:

- For failed transfers still in the Activity Log, select **Go to AzCopy Log File**.
- For transfers that failed in the past, go to the AzCopy logs folder. This folder can be found at:

[Expand table](#)

Platform	Folder path
Windows	<code>C:\Users\<your username>\.azcopy</code>
macOS	<code>~/ .azcopy</code>
Linux	<code>~/ .azcopy</code>

Network logs

For some problems, you need to provide logs of the network calls that Storage Explorer makes. On Windows, you can get network logs by using Fiddler.

ⓘ Note

Fiddler traces might contain passwords you entered or sent in your browser during the gathering of the trace. Make sure to read the instructions on how to sanitize a Fiddler trace. Don't upload Fiddler traces to GitHub. Wait for instructions on how to securely send your Fiddler trace.

Part 1: Install and configure Fiddler

1. Install Fiddler.
2. Start Fiddler.
3. Go to **Tools > Options**.
4. Select the **HTTPS** tab.
5. Make sure **Capture CONNECTs** and **Decrypt HTTPS traffic** are selected.
6. Select **Actions**.
7. Select **Trust Root Certificate** and then select **Yes** in the next dialog box.
8. Start Storage Explorer.
9. Go to **Settings** (the **gear** symbol in the vertical toolbar) > **Application > Proxy**.
10. Change the proxy source dropdown to **Use system proxy (preview)**.
11. Restart Storage Explorer.
12. You should start seeing network calls from a `storageexplorer:` process show up in Fiddler.

Part 2: Reproduce the problem

1. Close all apps other than Fiddler.
2. Clear the Fiddler log by using the **X** in the top left corner, near the **View** menu.
3. (Optional) Let Fiddler run for a few minutes. If you see network calls that aren't related to Storage Explorer, right-click them and select **Filter Now > Hide <process name>**.
4. Start or restart Storage Explorer.
5. Reproduce the problem.
6. Select **File > Save > All Sessions**. Save it in an easy-to-remember location.
7. Close Fiddler and Storage Explorer.

Part 3: Sanitize the Fiddler trace

1. Double-click the Fiddler trace (.saz file).
2. Select `Ctrl` + `F`.

3. In the dialog box that appears, make sure the following options are set: **Search = Requests and responses** and **Examine = Headers and bodies**.
4. Search for any passwords you used while you collected the Fiddler trace and any entries that are highlighted. Right-click and select **Remove > Selected sessions**.
5. You can skip sending the .saz file if any of the following conditions are true:
 - You definitely entered passwords into your browser while you collected the trace, but you don't find any entries when you use `Ctrl + F`.
 - You don't want to change your passwords.
 - The passwords you used are for other accounts.
6. Save the trace again with a new name.
7. (Optional) Delete the original trace.

Storage Explorer can't find Azurite containers

If Storage Explorer can't find your Azurite Docker or Podman containers, follow these steps to troubleshoot the issue:

1. Verify you have the latest version of Storage Explorer installed

Make sure you're using the [latest version of Storage Explorer](#). You can check for updates by going to **Help > Check for Updates**.

2. Verify container engine is installed

Ensure that the container engine is installed and running on your machine.

Bash

```
# If using Docker
docker --version

# If using Podman
podman --version
```

If the engine isn't running, restart container engine services and try again.

3. Verify communication channel locations

As of version 1.41.0, Storage Explorer needs to communicate with the container engine via designated named pipes (Windows) or Unix domain sockets (Linux and macOS).

By default, Storage Explorer looks in the following locations for these communication channels:

[Expand table](#)

Platform	Locations
Windows	\\.\pipe\docker_engine \\.\pipe\podman-machine-default
macOS	/var/run/docker.sock
Linux	/var/run/docker.sock ~/docker/run/docker.sock

Verify the location of the container engines you plan to use. If they aren't located in the default locations, modify the Azurite Communication Channels setting (`services.storageAccounts.containerSocketPaths`) to include the absolute paths to the correct locations.

4. Set up for Linux snap

The snap sandbox environment prevents Storage Explorer from accessing system files. You can work around snap limitations by opening a socket that forwards communications to the container engine using `socat`:

1. Verify the location of the container engine socket (for example, `/var/run/docker.sock`).
2. Create a forwarding socket within an accessible location, like your home directory:

Bash

```
socat UNIX-LISTEN:$HOME/docker.sock,fork UNIX-CONNECT:/var/run/docker.sock &
```

1. Launch Storage Explorer.
2. Modify the Azurite Communication Channels setting to include the absolute path to the forwarding socket (in this case, `/home/<username>/docker.sock`).
3. Refresh the `Emulators & Attached` node.

ⓘ Note

- The forwarding socket must be opened in a location that Storage Explorer can access, like your home directory.

- The forwarding socket must not be a hidden file (for example, its name can't start with a dot).
- You must enter an absolute path to the forwarding socket for the Azurite Communication Channels setting. You can't use expanding shell characters, like the "~" character
- If you terminate the background process launched by `socat`, Storage Explorer won't be able to communicate with the container engines to find Azurite instances.

If you're using Docker, you might also need to join the `docker` group.

Bash

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

Once added to the `docker` group, restart your machine. For more information, see [Linux post-installation steps for Docker Engine](#).

5. Verify the container engine context

Make sure you're using the correct context that's managing your containers.

Bash

```
# If using Docker
docker context ls
docker context use <context name>

# If using Podman
podman context ls
podman context use <context name>
```

6. Check Azurite container status

Verify that your Azurite containers are running. Ensure that your Azurite containers are listed and their status is `Up`. Start any containers that aren't running.

Bash

```
# If using Docker
docker container list --all
```

Ensure that your Azurite containers are listed and their status is **Up**. If a container isn't running, you can start it by using:

Bash

```
docker start <container name>

# If using Podman
podman container list --all
podman start <container name>
```

7. Verify Azurite ports

Ensure that your Azurite containers port configurations are correct.

Bash

```
# If using Docker
docker inspect <container name>

# If using Podman
podman inspect <container name>
```

Look for the `NetworkSettings.Ports` section in the output to verify the port mappings. For example, assuming an Azurite container's blob endpoint is configured to listen on port 10000, and that container port is mapped to the host machine's port 10010, you see an entry like this:

JSON

```
"10000/tcp": [
  {
    "HostIp": "0.0.0.0",
    "HostPort": "10010"
  }
]
```

8. Verify custom accounts are valid

If you're using custom accounts, make sure the account names and keys are valid and correctly configured in Storage Explorer.

Bash

```
# If using Docker
docker exec <container name> printenv AZURITE_ACCOUNTS
```

```
# If using Podman
podman exec <container name> printenv AZURITE_ACCOUNTS
```

9. Check network settings

Ensure that no network issues prevent Storage Explorer from connecting to the Azurite container. Verify that your firewall or antivirus software isn't blocking the connection.

Restart Azurite container

If the issue persists, try restarting the Azurite container.

Bash

```
# If using Docker
docker restart <container name>

# If using Podman
podman restart <container name>
```

Recreate Azurite container

If none of the preceding steps resolve the issue, try recreating the Azurite container. For example:

Bash

```
# If using Docker
docker stop <container name>
docker rm <container name>
docker run -d \
  --name <container-name> \
  -e AZURITE_ACCOUNTS="<custom account name>:<custom key>" \
  -p <blob-host-port>:10000 \
  -p <queue-host-port>:10001 \
  -p <table-host-port>:10002 \
  mcr.microsoft.com/azure-storage/azurite

# If using Podman
podman stop <container name>
podman rm <container name>
podman run -d \
  --name <container-name> \
  -e AZURITE_ACCOUNTS="<custom account name>:<custom key>" \
  -p <blob-host-port>:10000 \
  -p <queue-host-port>:10001 \
  -p <table-host-port>:10002 \
  mcr.microsoft.com/azure-storage/azurite
```

Storage Explorer doesn't run within FSLogix session

If FSLogix manages your user profile, Storage Explorer might be unable to run due to an `EEXIST` error. The error occurs due to a known issue with certain versions of FSLogix. If you encounter the error, make sure you're using the latest version of FSLogix. If the issue still occurs, you can work around it by following these steps:

1. Open the Group Policy Management Console (gpmc.msc).
2. Create or edit a Group Policy Object (GPO) that targets your AVD session hosts.
3. Go to **Computer Configuration > Preferences > Windows Settings > Registry**.
4. Add a new Registry Item with the following settings:

 Expand table

Action	Update
Hive	HKEY_LOCAL_MACHINE
Key Path	SYSTEM\CurrentControlSet\Services\frxdrvvt
Value Name	SupportedFeatures
Value Type	REG_DWORD
Value Data	11 (0xB)

5. Apply the GPO and reboot the session hosts.

Tip

You can also add registry items to the following key paths:

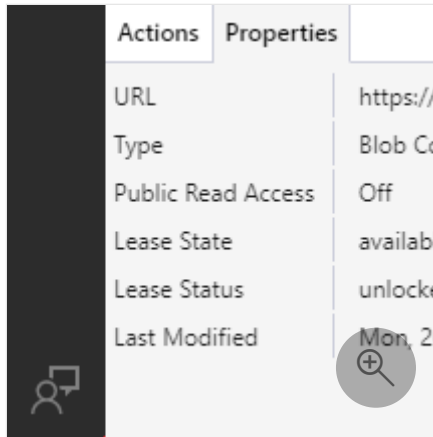
- SYSTEM\CurrentControlSet\Services\frxdrv
- SYSTEM\CurrentControlSet\Services\frxccd

Next steps

If none of these solutions work for you, use one of the following methods:

- [Create a support ticket](#).

- [Open an issue on GitHub](#) by selecting the **Report issue to GitHub** button in the lower-left corner.



Third-party information disclaimer

The third-party products that this article discusses are manufactured by companies that are independent of Microsoft. Microsoft makes no warranty, implied or otherwise, about the performance or reliability of these products.

Last updated on 03/19/2026